

Машинное обучение (краткий курс)

Судаков Владимир Анатольевич,
Профессор кафедры 806
sudakov@ws-dss.com

2026

Содержание

- Определение, классификация
- Линейные модели
 - регрессия
 - классификация
- Ядерные методы
- Деревья решений
- Графовые модели
- EM-алгоритм

Литература

- Кристофер Бишоп. Распознавание образов и машинное обучение (PRML book)
- Кэвин Мэрфи. Вероятностное машинное обучение: Введение
- Джоэл Грас. Data science. Наука о данных с нуля
- Машинное обучение (курс лекций, К.В.Воронцов)
- Машинное обучение и анализ данных (курс лекций, А.Г.Дьяконов)
- Стюарт Рассел, Питер Норвиг. Искусственный интеллект: современный подход (AIMA-2)
- Судаков В.А., Титов Ю.П. Методы искусственного интеллекта в информационных системах: Учебник — М.: МИРЭА, 2024.
- Саттон Ричард С., Барто Эндрю Г. Обучение с подкреплением
- <https://github.com/sudakov/ai-lab>
- <https://github.com/sudakov/math-for-ds/tree/main/Bayes>

Некоторые популярные на сегодня задачи ИИ

- Обработка естественного языка
- Анализ изображений



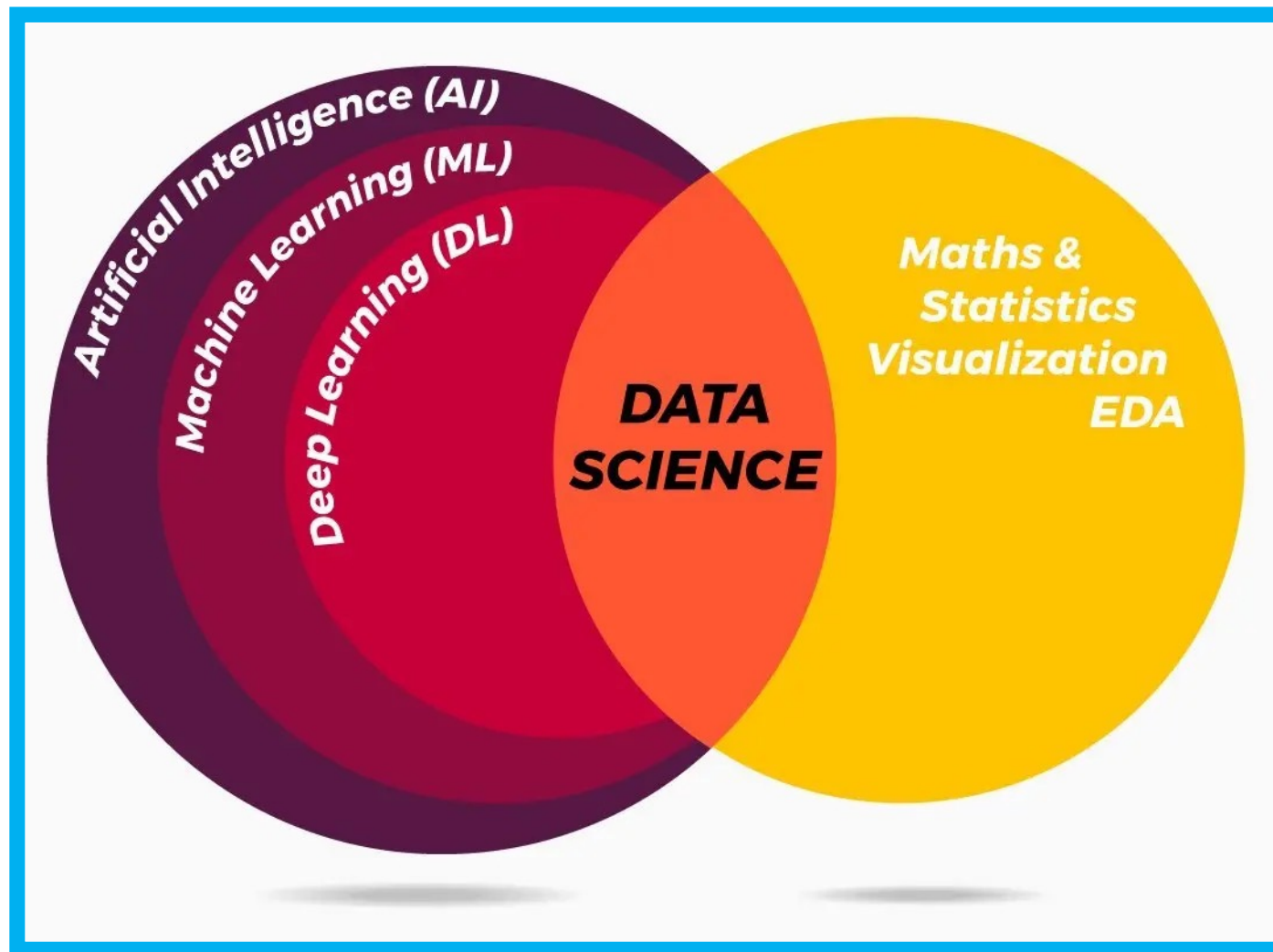
Машинное обучение.

Определение

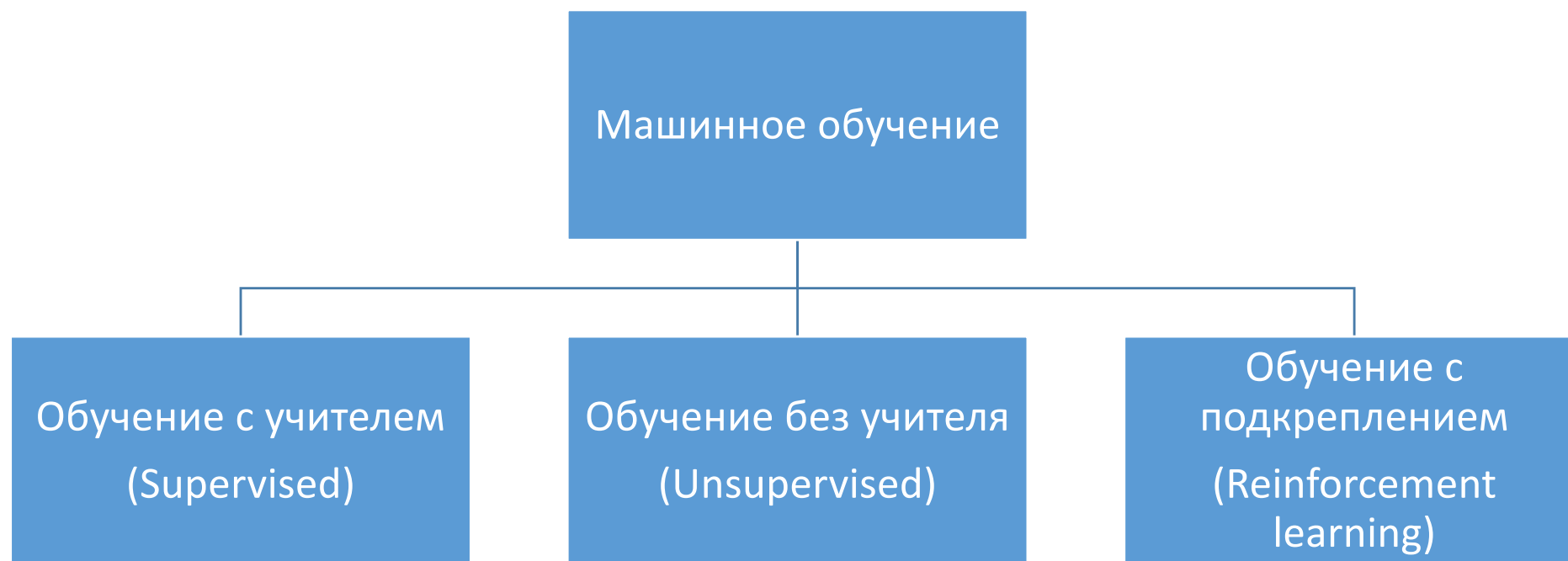
Говорят, что *компьютерная программа обучается на опыте E* относительно некоторого класса задач T и меры качества P , если ее качество на задачах, принадлежащих T , измеренное в соответствии с P , улучшается с увеличением опыта E .

Алгоритмы машинного обучения создают модель на основе выборочных данных, известных как «обучающие данные», чтобы делать прогнозы или предлагать решения, не будучи явно запрограммированными на это.

Взаимосвязи между науками

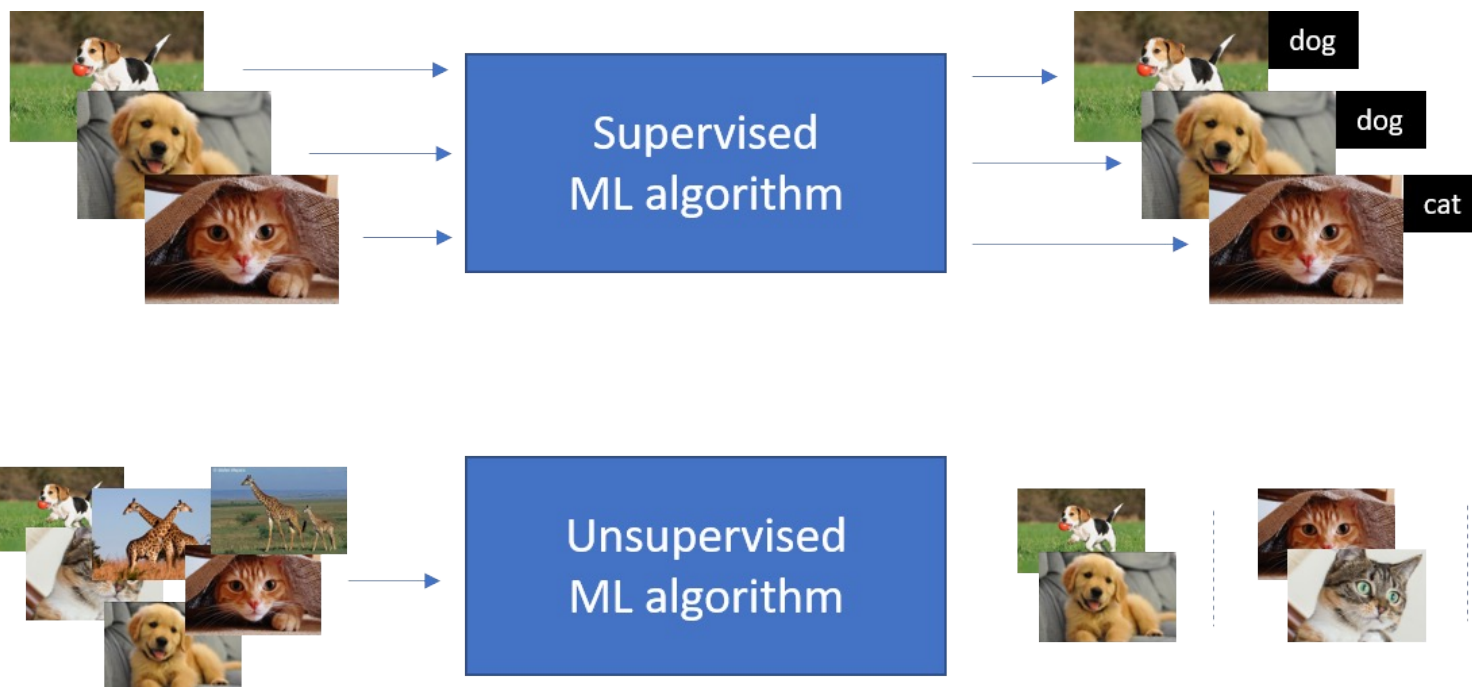


Машинное обучение



Отличия

Обучение с учителем (supervised) vs
Обучение без учителя (unsupervised)



Вопрос для обсуждения

Многие методы Data Science и Machine Learning появились достаточно давно - 50-70 года прошлого века, но активно использоваться в бизнесе начали только сейчас

С чем это связано?

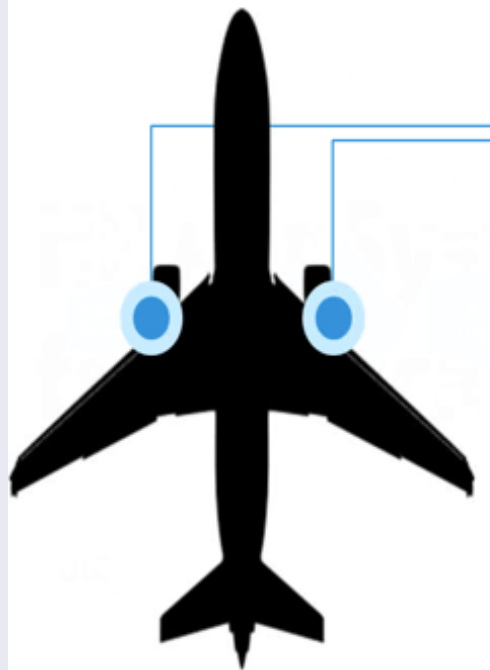
Что такого случилось?

Что есть сейчас и чего не было тогда?

Пример больших данных

Data in USA cross-country commercial flights

Sensor data from a cross-country flight



20 TB × 2 × 6 × 28,537 × 365

20 terabytes of
information per
engine every hour

twin-engine
Boeing 737

six-hour, cross-
country flight from
New York to Los
Angeles

of commercial
flights in the sky in
the United States on
any given day.

days in a year

= **2,499,841,200 TB**

Некоторые задачи машинного обучения

Регрессия

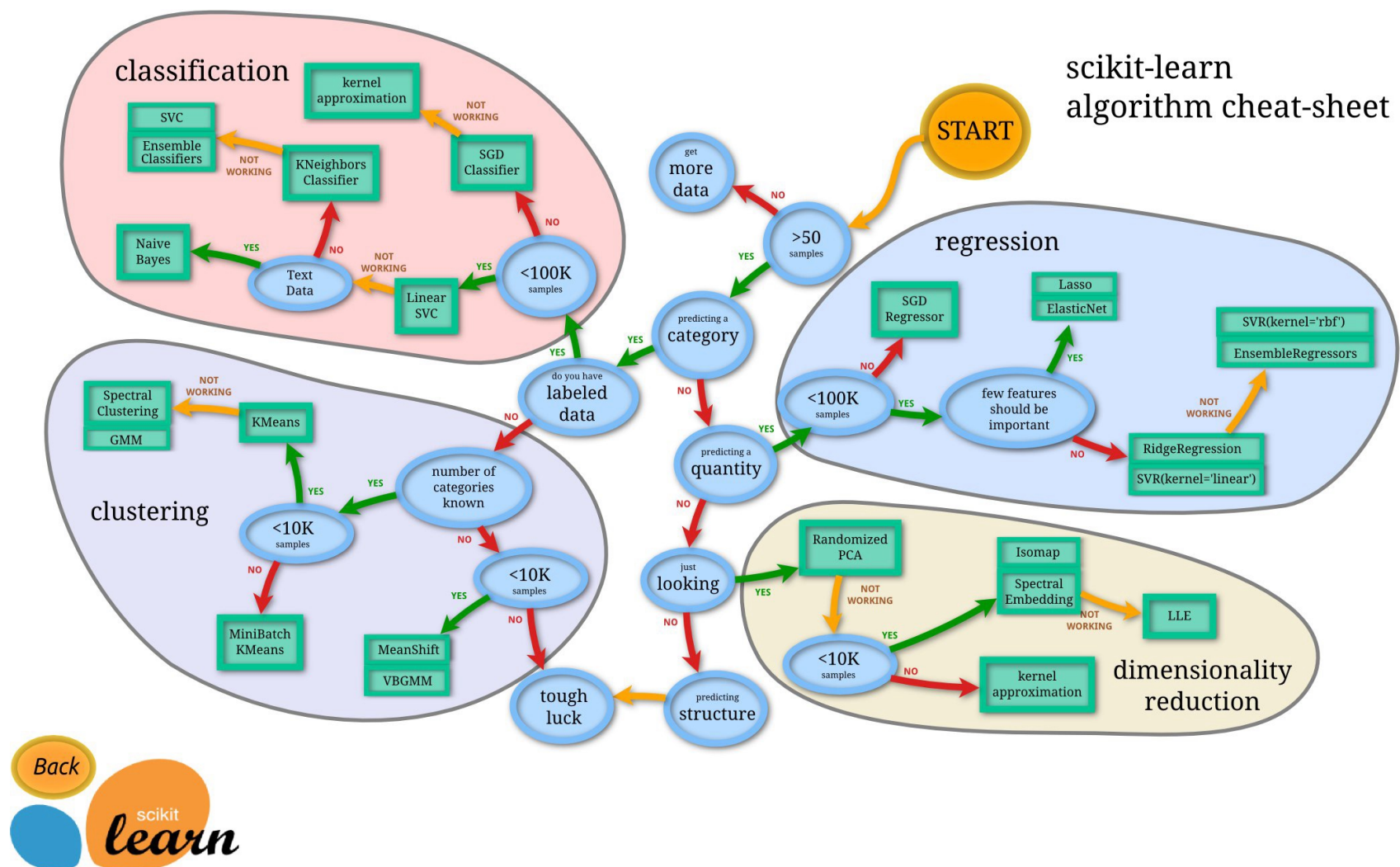
Классификация

Ранжирование

Кластеризация

Понижение размерности

Выбор алгоритма решения



Некоторые задачи

Задача регрессии – прогноз на основе выборки объектов с различными признаками. На выходе - вещественное число (2, 35, 76.454 и др.). Например, цена квартиры, стоимость ценной бумаги через неделю, ожидаемый доход магазина на следующий месяц.

Задача классификации – получение категориального ответа на основе набора признаков. Имеет конечное количество ответов (часто, в формате «да» или «нет»): является ли изображение человеческим лицом, давать ли клиенту кредит, к какой категории отнести товар.

Дано

Множество объектов X .

Множество допустимых ответов Y .

Целевая функция (target function) $y^* : X \rightarrow Y$, значения которой $t_i = y^*(x_i)$ известны только на конечном подмножестве объектов $\{x_1, \dots, x_l\} \subset X$.

Пары «объект– ответ» (x_i, t_i) называются прецедентами.

Совокупность пар $(x_i, t_i)_{i=1}^N$ называется обучающей выборкой (training sample).

Требуется найти

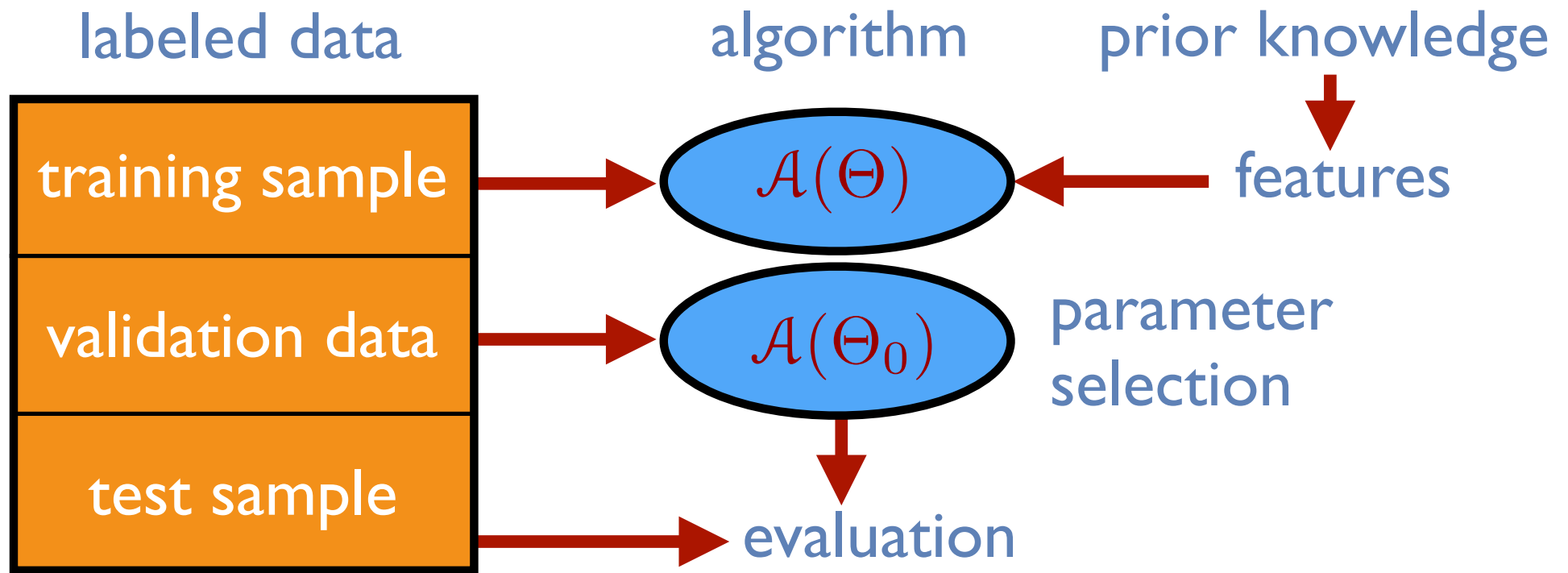
зависимость y^* по выборке $(x_i, t_i)_{i=1}^N$, то есть
построить решающую функцию (decision function)

$$y: X \rightarrow Y,$$

которая приближала бы целевую функцию $y^*(x)$,
причём не только на объектах обучающей выборки,
но и на всём множестве X (или некотором его
подмножестве).

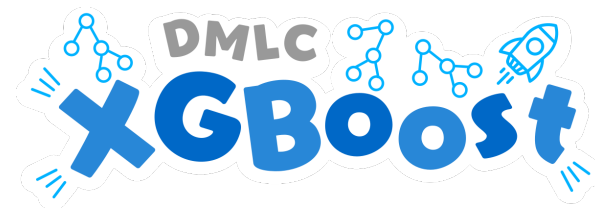
Решающая функция y должна допускать эффективную
компьютерную реализацию.

Этапы обучения



Рекомендуемые системы программирования и библиотеки

- Python
- Jupyter Lab
- NumPy
- Pandas
- Scikit-Learn
- XGBoost

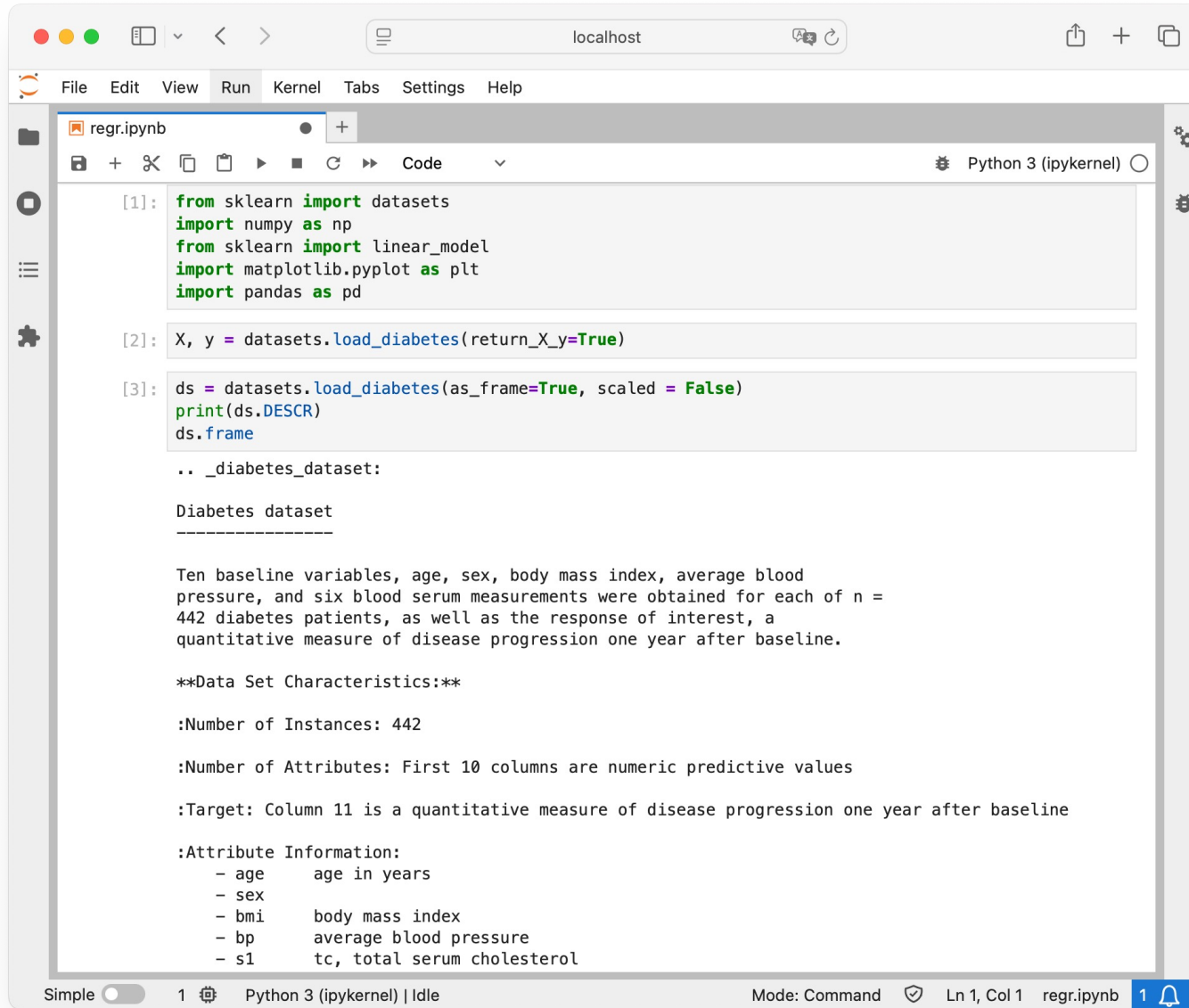


Следующий семестр

- PyTorch, TensorFlow, Transformers



Пример задачи



The screenshot shows a Jupyter Notebook window titled 'regr.ipynb' running on 'localhost'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Tabs, Settings, Help) and a toolbar with icons for file operations and execution. The notebook contains three code cells:

```
[1]: from sklearn import datasets
import numpy as np
from sklearn import linear_model
import matplotlib.pyplot as plt
import pandas as pd

[2]: X, y = datasets.load_diabetes(return_X_y=True)

[3]: ds = datasets.load_diabetes(as_frame=True, scaled = False)
print(ds.DESCR)
ds.frame
```

The output of the third cell shows the description of the Diabetes dataset:

```
.. _diabetes_dataset:

Diabetes dataset
-----

Ten baseline variables, age, sex, body mass index, average blood
pressure, and six blood serum measurements were obtained for each of n =
442 diabetes patients, as well as the response of interest, a
quantitative measure of disease progression one year after baseline.

**Data Set Characteristics:**

:Number of Instances: 442

:Number of Attributes: First 10 columns are numeric predictive values

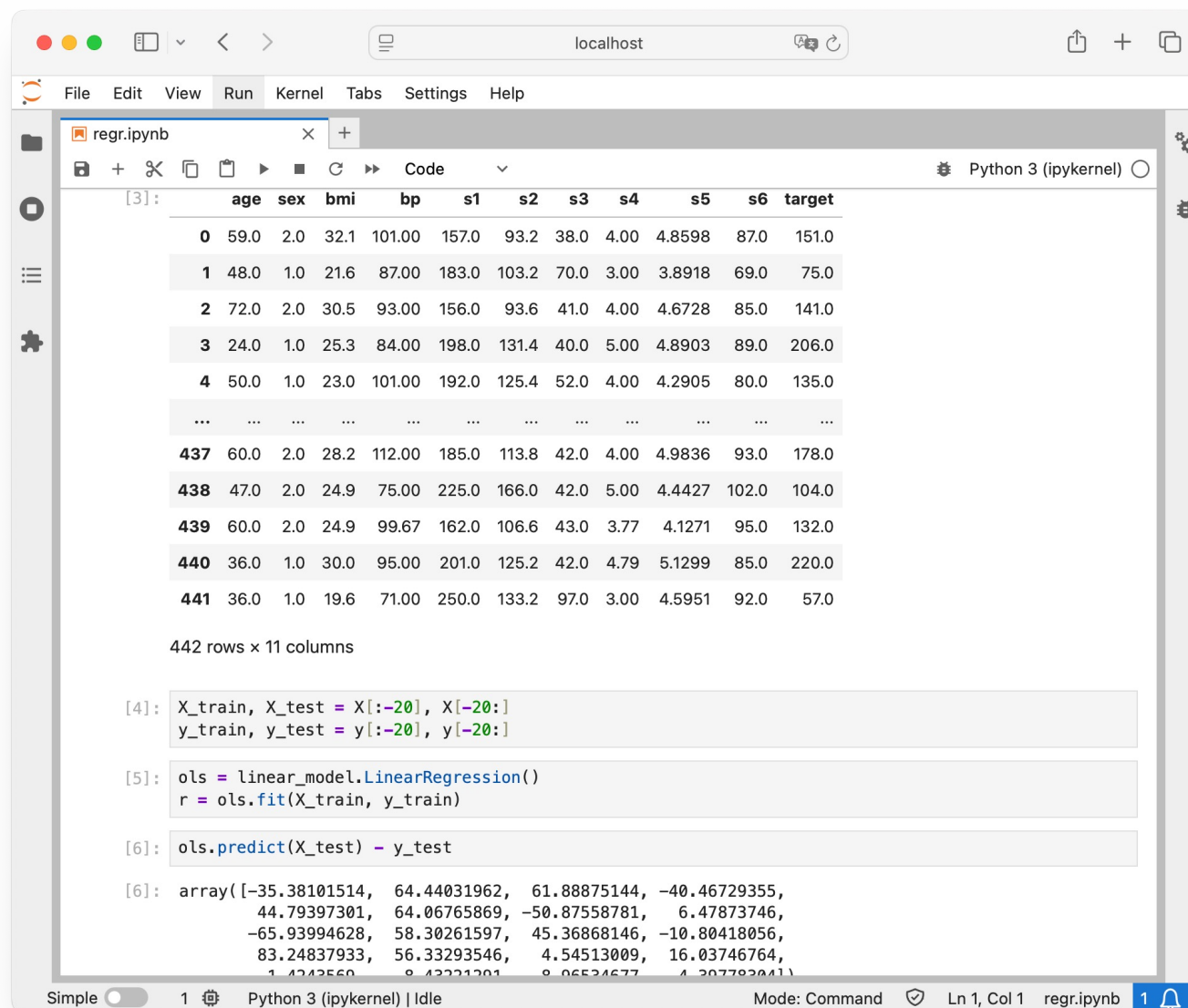
:Target: Column 11 is a quantitative measure of disease progression one year after baseline

:Attribute Information:
- age      age in years
- sex
- bmi      body mass index
- bp       average blood pressure
- s1       tc, total serum cholesterol
```

The status bar at the bottom indicates 'Simple' mode, 'Python 3 (ipykernel) | Idle', 'Mode: Command', and the current position 'Ln 1, Col 1' in the 'regr.ipynb' file.

Пример решения (не идеального)

Что нужно
улучшить?



The screenshot shows a Jupyter Notebook window titled 'regr.ipynb' with a Python 3 (ipykernel) environment. The notebook contains four code cells. The first cell displays a dataset with 442 rows and 11 columns. The second cell splits the data into training and testing sets. The third cell fits a linear regression model. The fourth cell predicts the target values for the test set.

```
[3]:
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	59.0	2.0	32.1	101.00	157.0	93.2	38.0	4.00	4.8598	87.0	151.0
1	48.0	1.0	21.6	87.00	183.0	103.2	70.0	3.00	3.8918	69.0	75.0
2	72.0	2.0	30.5	93.00	156.0	93.6	41.0	4.00	4.6728	85.0	141.0
3	24.0	1.0	25.3	84.00	198.0	131.4	40.0	5.00	4.8903	89.0	206.0
4	50.0	1.0	23.0	101.00	192.0	125.4	52.0	4.00	4.2905	80.0	135.0
...	...	...	...	...	...	...	...	...	...	...	...
437	60.0	2.0	28.2	112.00	185.0	113.8	42.0	4.00	4.9836	93.0	178.0
438	47.0	2.0	24.9	75.00	225.0	166.0	42.0	5.00	4.4427	102.0	104.0
439	60.0	2.0	24.9	99.67	162.0	106.6	43.0	3.77	4.1271	95.0	132.0
440	36.0	1.0	30.0	95.00	201.0	125.2	42.0	4.79	5.1299	85.0	220.0
441	36.0	1.0	19.6	71.00	250.0	133.2	97.0	3.00	4.5951	92.0	57.0

442 rows x 11 columns

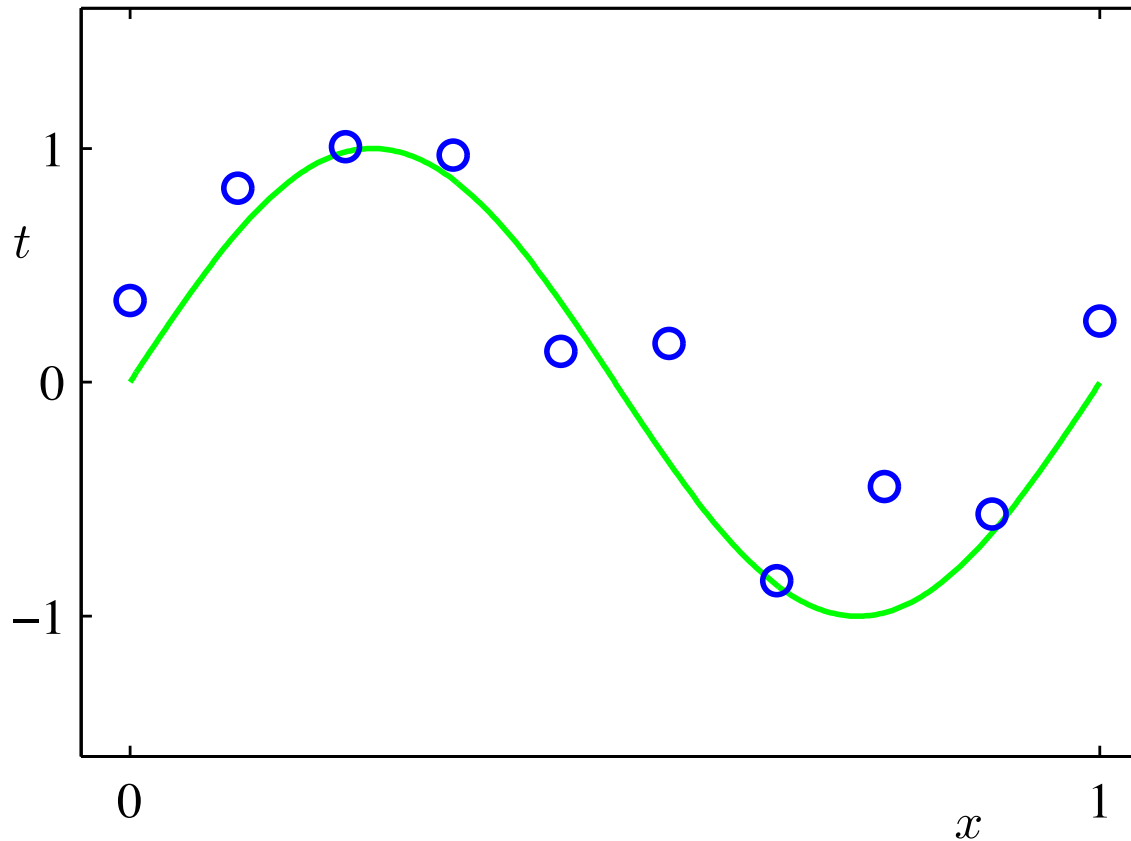
```
[4]: X_train, X_test = X[:-20], X[-20:]  
     y_train, y_test = y[:-20], y[-20:]  
  
[5]: ols = linear_model.LinearRegression()  
     r = ols.fit(X_train, y_train)  
  
[6]: ols.predict(X_test) - y_test  
  
[6]: array([-35.38101514,  64.44031962,  61.88875144, -40.46729355,  
           44.79397301,  64.06765869, -50.87558781,  6.47873746,  
          -65.93994628,  58.30261597,  45.36868146, -10.80418056,  
           83.24837933,  56.33293546,  4.54513009,  16.03746764,  
           1.4242560,  8.42221201,  8.06524677,  4.207782041])
```

Пример

- Рассмотрим синтетически сгенерированный пример набора данных. Данные были получены из функции $\sin(2\pi x)$ путём добавления случайного гауссовского шума со стандартным отклонением $\sigma = 0.3$.
 - Мы сгенерировали $N = 10$ наблюдения, равномерно распределенные в диапазоне $[0,1]$. Эти наблюдения составляют вектор входных данных,
- $x = (x_1, \dots, x_N)^T$
- Для каждого сгенерированного наблюдения x мы получили соответствующее значение функции $\sin(2\pi x)$, а затем добавили случайный шум, чтобы зафиксировать реальную ситуацию с отсутствующей информацией.

$$t = (t_1, \dots, t_N)^T$$

Пример графика



Наша цель — предсказать значение $\hat{t}$ для некоторого **нового** значения $\hat{x}$, при отсутствии каких-либо данных о зелёной кривой.

Пример

Попытаемся подгонять данные с помощью полиномиальной функции вида

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_j x^j$$

где M — *порядок* полинома.

Такие функции, как $y(x, \mathbf{w})$, являющиеся линейными функциями неизвестных коэффициентов $\mathbf{w}$, называются *линейными моделями* .

Функция ошибки

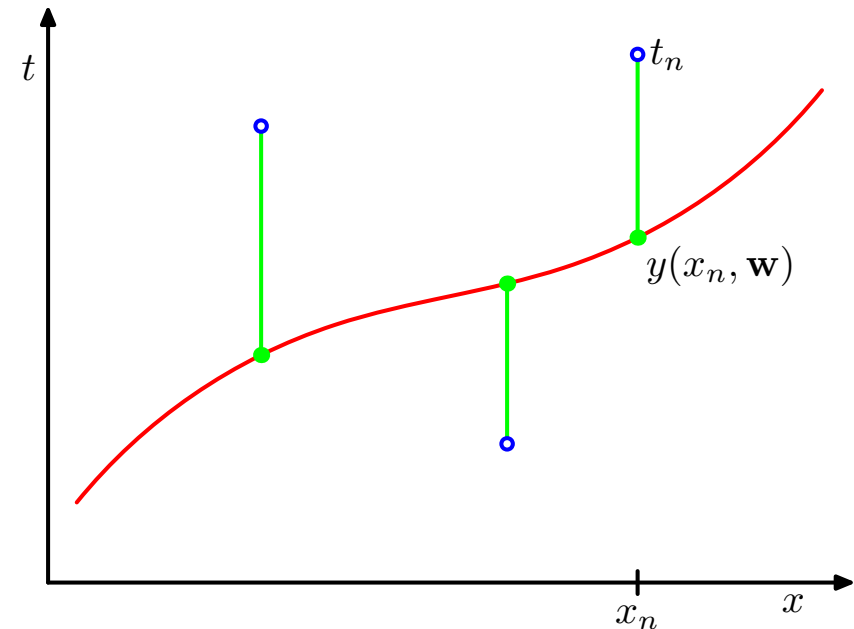
Необходимо определить значения коэффициентов $\mathbf{w}$, подгоняя полином к обучающим данным. Это можно сделать, минимизировав *функцию ошибки*, которая измеряет несоответствие между функцией $y(x, \mathbf{w})$, для заданного значения $\mathbf{w}$, и точками обучающих данных.

Одна простая функция ошибок представляет собой *сумму квадратов ошибок* между $y(x, \mathbf{w})$ и соответствующими целевыми значениями t_n

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y(x_n, \mathbf{w}) - t_n)^2 \geq 0$$

где функция становится равной нулю тогда и только тогда, когда функция $y(x, \mathbf{w})$ проходит точно через каждую точку обучающих данных.

Мы можем решить задачу аппроксимации кривой, выбрав значение $\mathbf{w}$ при котором $E(\mathbf{w})$, как можно меньше. Поскольку функция погрешности квадратичная, её производные линейны, и, следовательно, минимизация функции имеет единственное решение в точке, обозначенное как $\mathbf{w}^*$.



Поиск решения

Для минимизации функции погрешности необходимо вывести вектор градиента, приравнять его к нулю и решить уравнение $\mathbf{w}^*$ следующим образом:

$$\nabla E(\mathbf{w}^*) = \mathbf{0}$$

Сначала нам нужно подставить полином в функцию ошибки

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \sum_{j=0}^M (w_j x_n^j - t_n)^2$$

Обратите внимание, что каждая точка данных из сгенерированного обучающего набора имеет размерность 1, то есть $x \in \mathbb{R}$. Однако полиномиальная функция заполняет M признаков для каждого входного значения x , по сути, преобразуя x в M -мерный вектор. Таким образом, обучающий набор x можно записать в виде $N \times M$ матрицы $\mathbf{X} = \|\mathbf{x}_{nj}\|$, где $\mathbf{x}_{nj}$ представляет x_n^j , то есть n -ое входное значение, возведенное в степень j . Чтобы найти вектор градиента, мы берём частную производную E по произвольному w_k . Дифференцируя сумму почленно, получаем:

$$\begin{aligned} \nabla E(\mathbf{w}^*)_k &= \frac{\partial}{\partial w_k} E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N 2 \left(\sum_{j=0}^M w_j x_n^j - t_n \right) x_n^k = \sum_{n=1}^N \left(\sum_{j=0}^M w_j x_n^j - t_n \right) x_n^k = \sum_{n=1}^N (\mathbf{X}\mathbf{w} - \mathbf{t})_n x_{nk} = \sum_{n=1}^N \mathbf{x}_{kn}^T (\mathbf{X}\mathbf{w} - \mathbf{t})_n \\ &= (\mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{t}))_k \end{aligned}$$

Используя частную производную для одного компонента, мы вычисляем вектор градиента по k индексу. Таким образом, $\mathbf{w}^*$ должно удовлетворять

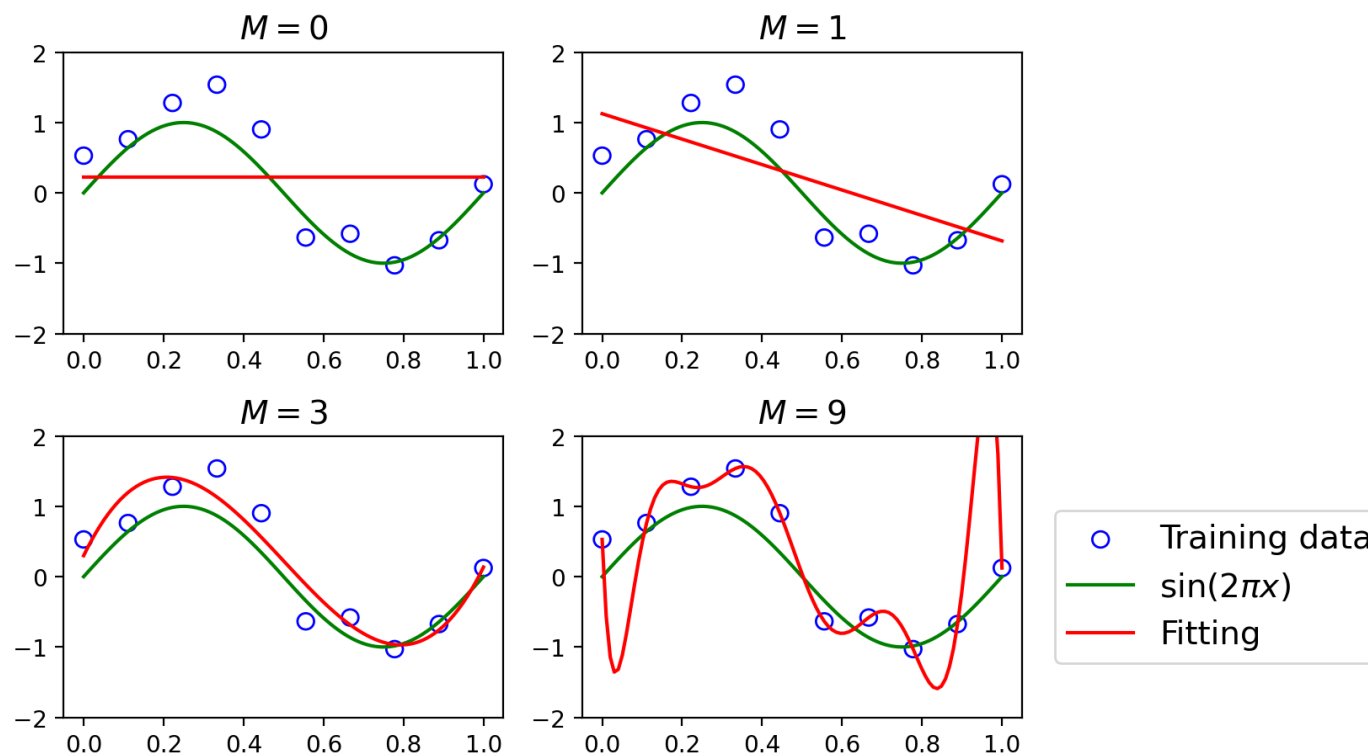
$$\nabla E(\mathbf{w}^*) = \mathbf{X}^T (\mathbf{X}\mathbf{w}^* - \mathbf{t}) = \mathbf{0}$$

Решение $\mathbf{w}^*$ дает единственное решение задачи подгонки кривой

$$\mathbf{X}^T (\mathbf{X}\mathbf{w}^* - \mathbf{t}) = \mathbf{0} \Leftrightarrow \mathbf{X}^T \mathbf{X}\mathbf{w}^* = \mathbf{X}^T \mathbf{t} \Leftrightarrow \mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}$$

Полученный многочлен задается функцией $y(x, \mathbf{w}^*)$.

Выбор модели

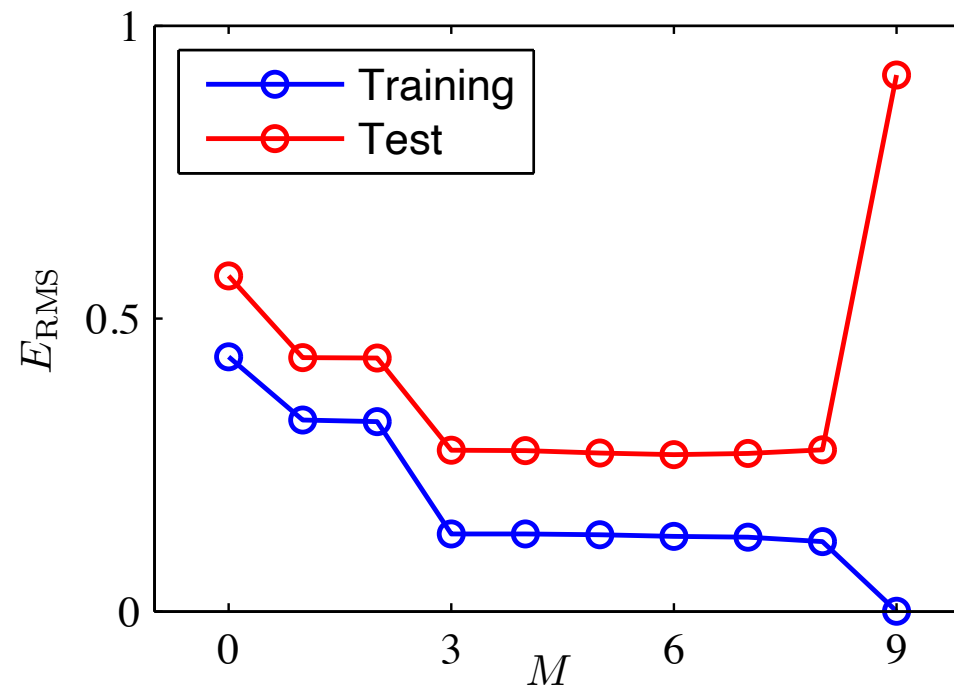


Обратите внимание, что константа ($M = 0$) и полиномы первого порядка ($M = 1$) дают довольно плохое соответствие данным. Полином третьего порядка ($M = 3$) даёт, по-видимому, наилучшее соответствие, в то время как полином более высокого порядка ($M = 9$) даёт отличное соответствие данным, то есть $E(\mathbf{w}^*) = 0$. Однако подобранная кривая плохо отражает базовую функцию $\sin(2\pi x)$. Это явление известно как *переобучение*.

RMS

Более количественное представление об эффективности обобщения M можно получить, используя *среднеквадратичную ошибку* (RMS), определяемую как

$$E_{RMS} = \sqrt{2 \frac{E(\mathbf{w}^*)}{N}}$$



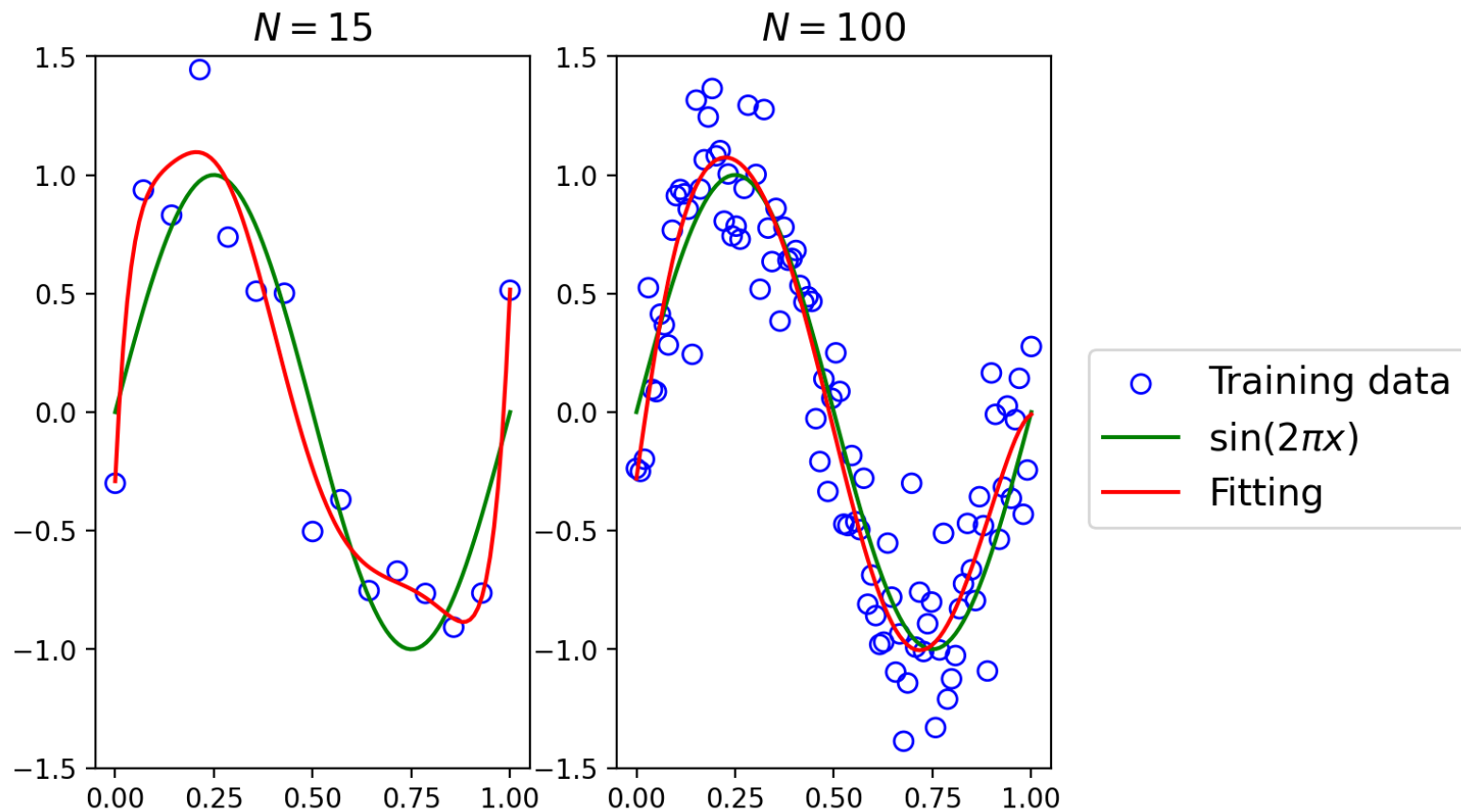
Кажется, что
модель не
должна работать
хуже с
увеличением M

Таблица коэффициентов

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Увеличение объема данных

Полином 9-й степени



Регуляризация

Одним из часто используемых методов контроля явления переобучения является *регуляризация*, которая добавляет штрафной член к функции ошибки, чтобы предотвратить достижение коэффициентами больших значений. Простейшим таким штрафным членом является сумма квадратов всех коэффициентов, что приводит к модифицированной функции ошибки следующего вида:

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y(x_n, \mathbf{w}) - t_n)^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Такие методы известны как методы *сжатия*, поскольку они уменьшают значение коэффициентов. Частный случай квадратичной регуляризации называется *гребневой регрессией* (ridge regression). В нейронных сетях этот подход также известен как *редукция весов* (weight decay).

Часто коэффициент w_0 исключается из регуляризатора, потому что из-за его включения результаты зависят от выбора начала координат для целевой переменной

Минимизация

Функцию ошибки гребня можно точно минимизировать следующим образом:

$$\begin{aligned}\nabla E(\mathbf{w}^*)_k &= \frac{\partial}{\partial w_k} E(\mathbf{w}) \\&= \frac{1}{2} \sum_{n=1}^N 2 \left(\sum_{j=0}^M w_j x_n^j - t_n \right) x_n^k + \frac{1}{2} \lambda 2 w_k \\&= \sum_{n=1}^N \left(\sum_{j=0}^M w_j x_n^j - t_n \right) x_n^k + \lambda w_k \\&= \sum_{n=1}^N (\mathbf{X}\mathbf{w} - \mathbf{t})_n \mathbf{X}_{nk} + \lambda w_k = \sum_{n=1}^N \mathbf{X}_{kn}^T (\mathbf{X}\mathbf{w} - \mathbf{t})_n + \lambda w_k \\&= (\mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{t}))_k + \lambda w_k\end{aligned}$$

Используя частную производную для одного компонента, мы вычисляем вектор градиента, отбрасывая k нижний индекс. Тогда $\mathbf{w}^*$ должен удовлетворять

$$\nabla E(\mathbf{w}^*) = \mathbf{X}^T (\mathbf{X}\mathbf{w}^* - \mathbf{t}) + \lambda \mathbf{w}^* \mathbf{I} = \mathbf{0}$$

Решение $\mathbf{w}^*$ дает единственное решение, которое минимизирует ошибку гребня

$$\begin{aligned}\mathbf{X}^T (\mathbf{X}\mathbf{w}^* - \mathbf{t}) + \lambda \mathbf{w}^* \mathbf{I} &= \mathbf{0} \Leftrightarrow \\ \mathbf{X}^T \mathbf{X}\mathbf{w}^* - \mathbf{X}^T \mathbf{t} + \lambda \mathbf{w}^* \mathbf{I} &= \mathbf{0} \Leftrightarrow \\ \mathbf{X}^T \mathbf{X}\mathbf{w}^* + \lambda \mathbf{w}^* \mathbf{I} &= \mathbf{X}^T \mathbf{t} \Leftrightarrow \\ \mathbf{w}^* (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) &= \mathbf{X}^T \mathbf{t} \Leftrightarrow \\ \mathbf{w}^* &= (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{t}\end{aligned}$$

Результаты для полинома $M=9$

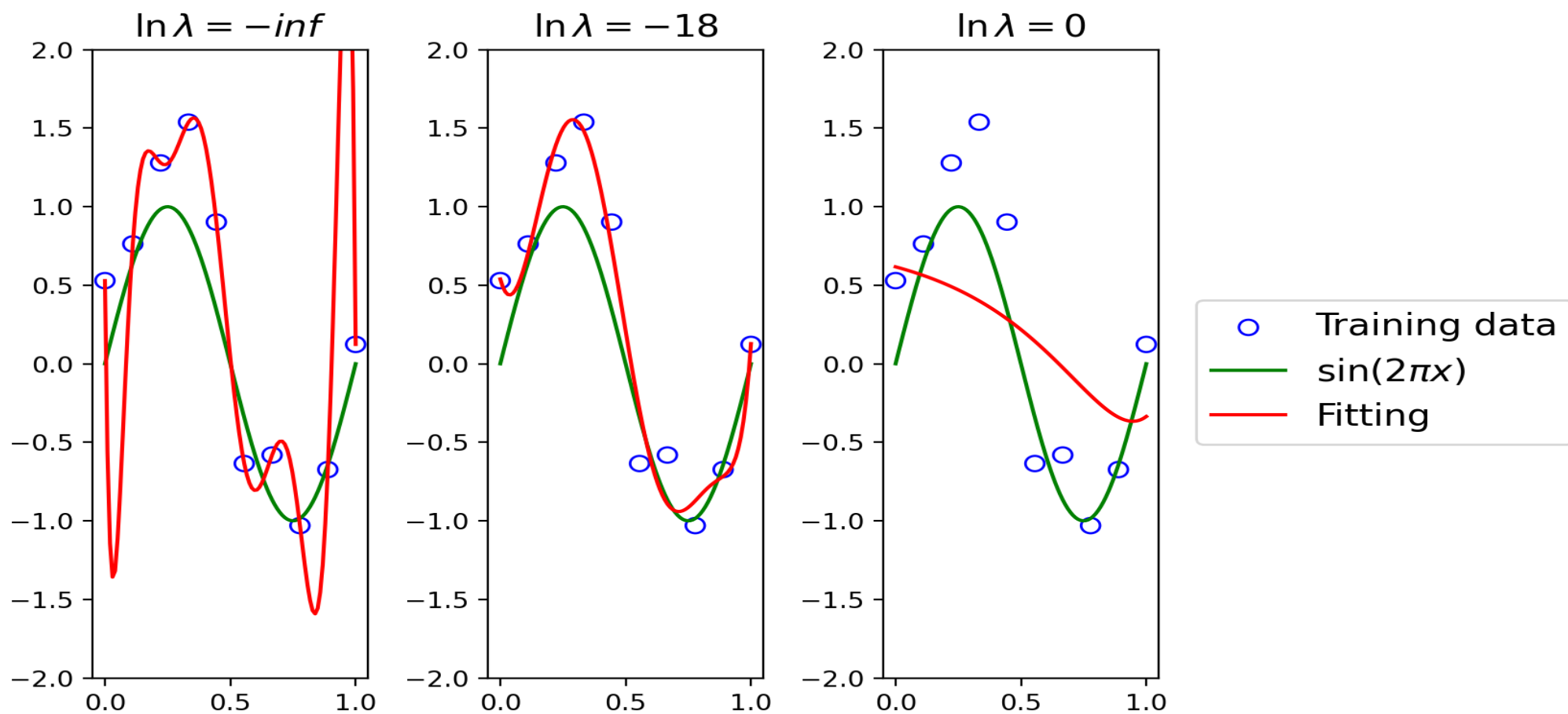
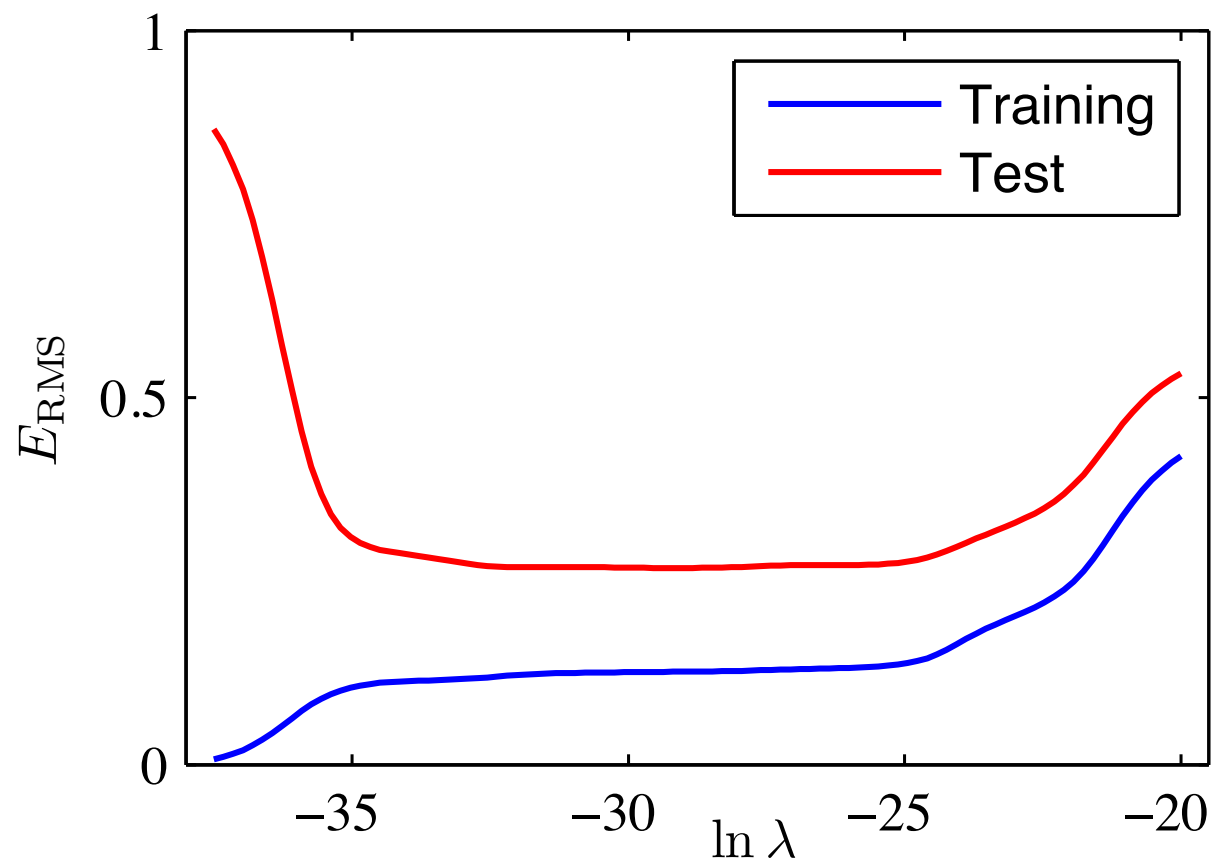


Таблица коэффициентов

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w0	0.39	0.38	0.36
w1	-135.04	-2.14	-0.46
w2	3206.76	81,88	-0.39
w3	-29215.92	-390.51	-0.22
w4	139594.34	578.12	-0.05
w5	-388863.80	-31.48	0.07
w6	652373.24	-49.12	0.17
w7	-648124.69	-28.57	0.25
w8	350721.94	540.17	0.31
w9	-79556.29	-255.65	0.35

График среднеквадратической ошибки в зависимости от $\ln \lambda$ для полинома степени $M = 9$



Теорема Байеса в ML

Пусть $\mathbf{w}$ — параметры и $\mathcal{D}$ — данные. Теорема Байеса задаётся формулой:

$$p(\mathbf{w}|\mathcal{D}) = \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w})}{p(\mathcal{D})} \Leftrightarrow \text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$$

Частотная парадигма обычно количественно оценивает свойства величин, определяемых данными, в свете фиксированных параметров модели, тогда как байесовская парадигма обычно количественно оценивает свойства неизвестных параметров модели в свете наблюдаемых данных.

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

$p(\mathcal{D})$ можно найти из условия: $\int p(\mathbf{w}|\mathcal{D})d\mathbf{w} = 1 \Rightarrow p(\mathcal{D}) = \int p(\mathcal{D}|\mathbf{w})p(\mathbf{w})d\mathbf{w}$

Оценка правдоподобия

- Широко используемой частотной оценкой $p(\mathcal{D}|\mathbf{w})$ является оценка максимального правдоподобия, при котором значение $\mathbf{w}$ максимизирует функцию правдоподобия $p(\mathcal{D}|\mathbf{w})$. Это соответствует выбору значения $\mathbf{w}$, для которого вероятность наблюдаемого набора данных максимизируется. В литературе по машинному обучению отрицательная логарифмическая функция правдоподобия называется функцией ошибок. Поскольку отрицательный логарифм является монотонно убывающей функцией, максимизация правдоподобия эквивалентна минимизации ошибки.
- Одним из подходов к определению частотной величины ошибки является бутстрэп, в котором несколько наборов данных создаются следующим образом. Предположим, наш исходный набор данных состоит из N точек данных $X = \{x_1, \dots, x_N\}$. Мы можем создать новый набор данных X_b , извлекая N точек случайным образом из набора X с возвращением, так что некоторые точки из X могут повторяться в X_b , тогда как другие точки в X могут отсутствовать в X_b . Этот процесс может быть повторен L раз для генерации L наборов данных, содержащих по N точек, и каждый из них получается путем выбора из исходного набора данных
- Статистическую точность оценок параметров можно определить, посмотрев на изменчивость предсказаний между различными наборами данными.

Задача +1 балл

Монета бросается три раза и каждый раз выпадает орел. Какова вероятность получить орла в четвертом броске?

Не стесняйтесь задавать уточняющие вопросы от них многое зависит.

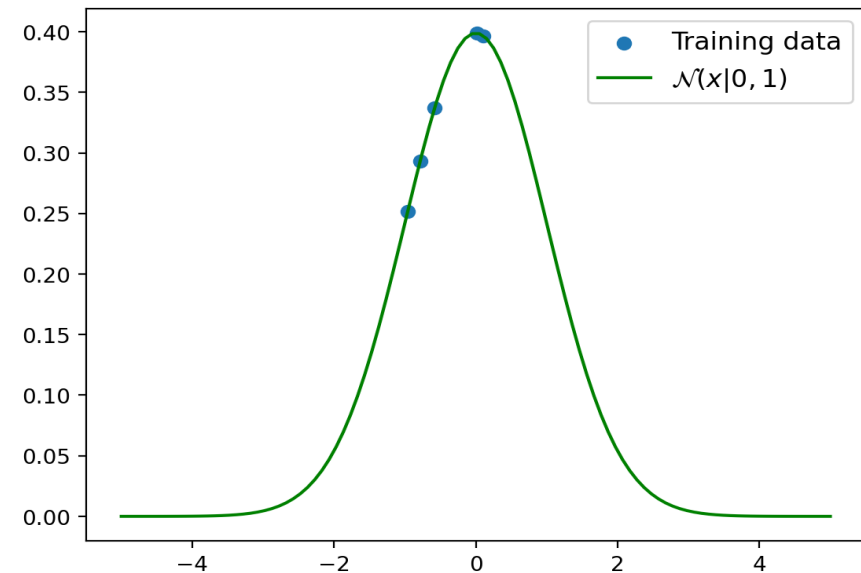
Нормальное распределение

$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp\left\{-\frac{1}{2\sigma^2} (x - \mu)^2\right\}$$

Предположим, у нас есть набор данных наблюдений, $x = (x_1, \dots, x_N)^T$ полученных независимо из гауссовского распределения. Точки данных, полученные независимо из одного и того же распределения, называются независимыми и одинаково распределёнными.

Поскольку данные независимы, функция правдоподобия одномерного гауссовского распределения выглядит следующим образом:

$$p(x|\mu, \sigma^2) = \prod_{n=1}^N \mathcal{N}(x_n|\mu, \sigma^2)$$



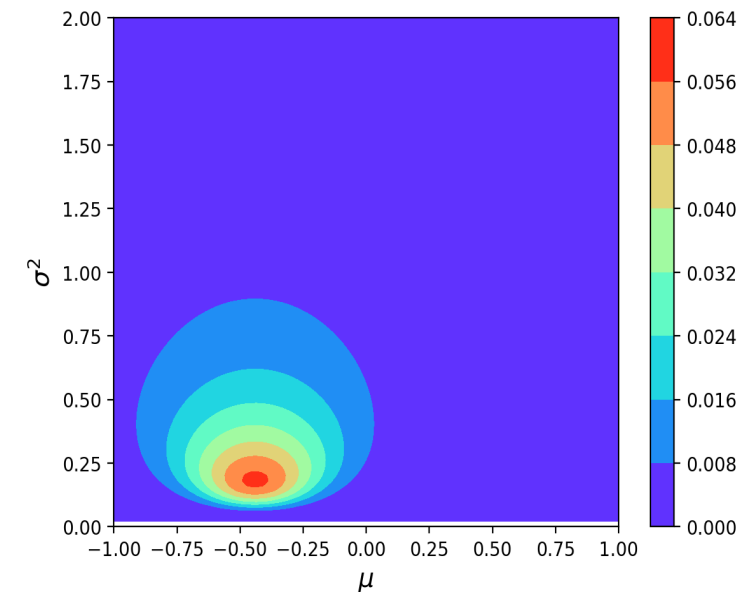
Нахождение параметров μ, σ^2

Для нахождения неизвестных параметров μ и σ^2 необходимо использовать наблюдаемый набор данных и найти значения параметров, максимизирующие функцию правдоподобия. Логарифмическую функцию правдоподобия можно записать следующим образом:

$$\begin{aligned}\ln p(x|\mu, \sigma^2) &= \ln[\prod_{n=1}^N \mathcal{N}(x_n|\mu, \sigma^2)] \\ &= \sum_{n=1}^N \ln \mathcal{N}(x_n|\mu, \sigma^2) \\ &= \sum_{n=1}^N \ln \left(\frac{1}{(2\pi\sigma^2)^{1/2}} \exp \left\{ -\frac{1}{2\sigma^2} (x_n - \mu)^2 \right\} \right) \\ &= \sum_{n=1}^N \ln \left(\frac{1}{(2\pi\sigma^2)^{1/2}} \right) + \sum_{n=1}^N \ln \left(\exp \left\{ -\frac{1}{2\sigma^2} (x_n - \mu)^2 \right\} \right) \\ &= N \ln \left(\frac{1}{(2\pi\sigma^2)^{1/2}} \right) - \sum_{n=1}^N \frac{1}{2\sigma^2} (x_n - \mu)^2 \\ &= N \ln 1 - N \ln(2\pi\sigma^2)^{1/2} - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \\ &\stackrel{\ln 1=0}{=} -N \ln(2\pi\sigma^2)^{1/2} - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \\ &\stackrel{\ln x^y = y \ln x}{=} -\frac{N}{2} \ln 2\pi\sigma^2 - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \\ &= -\frac{N}{2} \ln 2\pi - \frac{N}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2\end{aligned}$$

Отрицательный логарифм функции правдоподобия называется функцией ошибки.

Поскольку отрицательный логарифм — монотонно убывающая функция, максимизация правдоподобия эквивалентна минимизации ошибки.



Нахождение параметров μ, σ^2

$$\begin{aligned}
 \frac{\partial \ln p(x|\mu, \sigma^2)}{\partial \mu} &= 0 \Leftrightarrow \\
 \frac{\partial}{\partial \mu} \left(\frac{N}{2} \ln 2\pi - \frac{N}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \right) &= 0 \Leftrightarrow \\
 \frac{\partial}{\partial \mu} \left(\frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \right) &= 0 \Leftrightarrow \\
 \frac{\partial}{\partial \mu} \sum_{n=1}^N (x_n - \mu)^2 &= 0 \Leftrightarrow \\
 \sum_{n=1}^N (2\mu - 2x_n) &= 0 \Leftrightarrow 2 \sum_{n=1}^N (\mu - x_n) = 0 \Leftrightarrow \\
 N\mu - \sum_{n=1}^N x_n &= 0 \Leftrightarrow N\mu = \sum_{n=1}^N x_n \Leftrightarrow \\
 \mu_{ML} &= \frac{1}{N} \sum_{n=1}^N x_n
 \end{aligned}$$

$$\begin{aligned}
 \frac{\partial \ln p(x|\mu, \sigma^2)}{\partial \sigma^2} &= 0 \Leftrightarrow \\
 \frac{\partial}{\partial \sigma^2} \left(\frac{N}{2} \ln 2\pi - \frac{N}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \right) &= 0 \Leftrightarrow \\
 \frac{\partial}{\partial \sigma^2} \left(-\frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 - \frac{N}{2} \ln \sigma^2 \right) &= 0 \Leftrightarrow \\
 \frac{\partial}{\partial \sigma^2} \left(-\frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \right) + \frac{\partial}{\partial \sigma^2} \left(-\frac{N}{2} \ln \sigma^2 \right) &= 0 \Leftrightarrow \\
 \frac{\partial}{\partial \sigma^2} \left(-(2\sigma^2)^{-1} \sum_{n=1}^N (x_n - \mu)^2 \right) + \frac{N}{2\sigma^2} &= 0 \Leftrightarrow \\
 \frac{1}{4\sigma^4} \sum_{n=1}^N (x_n - \mu)^2 - \frac{N}{2\sigma^2} &= 0 \Leftrightarrow \\
 \frac{1}{4\sigma^4} \sum_{n=1}^N (x_n - \mu)^2 &= \frac{N}{2\sigma^2} \Leftrightarrow \\
 \sigma_{ML}^2 &= \frac{1}{N} \sum_{n=1}^N (x_n - \mu_{ML})^2
 \end{aligned}$$

Проблема, возникающая в контексте наших решений для метода максимального правдоподобия, заключается в том, что он систематически недооценивает дисперсию гауссовского распределения. Это пример явления, называемого смещением, и связано с проблемой переобучения, возникающей в контексте полиномиальной

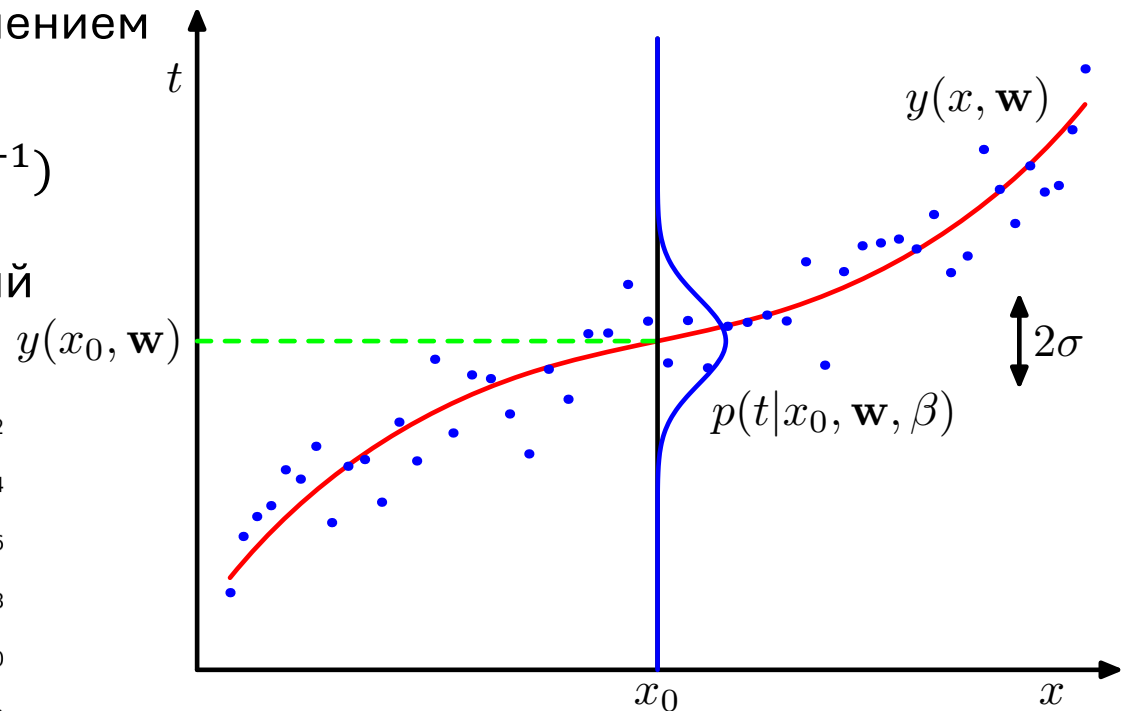
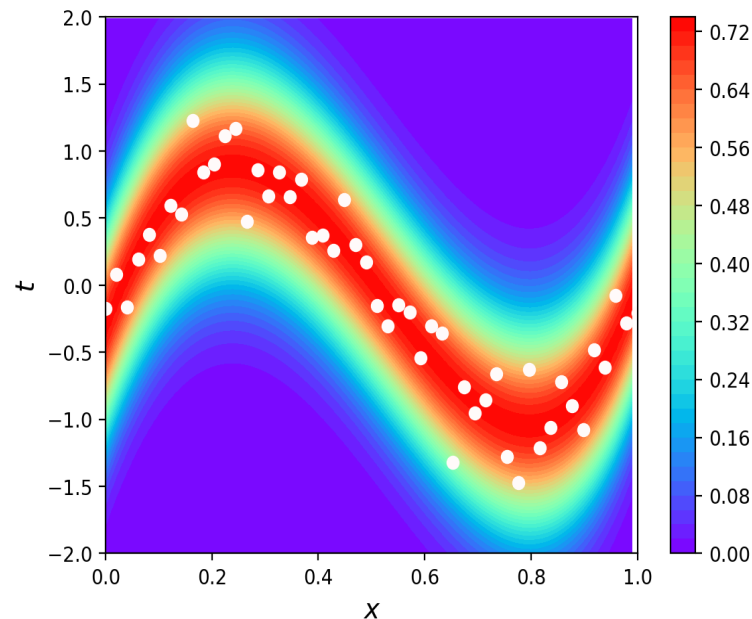
подгонки кривой. $\tilde{\sigma}^2 = \frac{N}{N-1} \sigma_{ML}^2 = \frac{1}{N-1} \sum_{n=1}^N (x_n - \mu_{ML})^2$

Аппроксимация кривой

Предположим, что целевая переменная t имеет гауссовское распределение со средним значением $y(x, \mathbf{w})$:

$$p(t|x, \mathbf{w}, \beta) = \mathcal{N}(t|y(x, \mathbf{w}), \beta^{-1})$$

где β параметр, соответствующий обратной дисперсии.



Аппроксимация кривой

Используем обучающие данные x, t для определения значений неизвестных параметров $\mathbf{w}, \beta$ методом максимального правдоподобия. Если данные являются независимыми, функция правдоподобия задаётся следующим образом:

$$p(t|x, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n | y(x, \mathbf{w}), \beta^{-1})$$

Подобно гауссовскому распределению ранее, мы максимизируем логарифм функции правдоподобия в форме

$$\ln p(t|x, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{n=1}^N (y(x, \mathbf{w}) - t_n)^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln 2\pi$$

Для коэффициентов полинома $\mathbf{w}_{ML}$ можно опустить члены, не зависящие от $\mathbf{w}$. Поскольку β не меняет положения максимума функции, её можно заменить на 1. Следовательно, максимизация эквивалентна минимизации *функции ошибки суммы квадратов*, как показано в примере аппроксимации кривой полиномом. Функция ошибки суммы квадратов возникла как следствие максимизации правдоподобия в предположении гауссовского распределения шума.

Определение β

$$\begin{aligned}\frac{\partial}{\partial \beta} \ln p(t|x, \mathbf{w}, \beta) &= 0 \Leftrightarrow \\ \frac{\partial}{\partial \beta} \left[-\frac{\beta}{2} \sum_{n=1}^N (y(x, \mathbf{w}) - t_n)^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln 2\pi \right] &= 0 \Leftrightarrow \\ \frac{\partial}{\partial \beta} \left[-\frac{\beta}{2} \sum_{n=1}^N (y(x, \mathbf{w}) - t_n)^2 \right] + \frac{\partial}{\partial \beta} \left[\frac{N}{2} \ln \beta \right] &= 0 \Leftrightarrow \\ -\frac{1}{2} \sum_{n=1}^N (y(x, \mathbf{w}) - t_n)^2 + \frac{N}{2} \frac{1}{\beta} &= 0 \Leftrightarrow \\ \sum_{n=1}^N (y(x, \mathbf{w}) - t_n)^2 &= \frac{N}{\beta} \Leftrightarrow \\ \frac{1}{\beta_{ML}} &= \frac{1}{N} \sum_{n=1}^N (y(x, \mathbf{w}) - t_n)^2\end{aligned}$$

Поскольку у нас есть вероятностная модель, мы можем использовать предсказательное распределение, которое дает распределение вероятностей по t , а не простую точечную оценку.

$$p(t|x, \mathbf{w}_{ML}, \beta_{ML}) = \mathcal{N}(t|y(x, \mathbf{w}_{ML}), \beta_{ML}^{-1})$$

Можно дополнительно ввести априорное распределение для коэффициентов полинома $\mathbf{w}$, чтобы сделать шаг к более байесовскому подходу.

Априорное распределение для коэффициентов полинома $\mathbf{w}$

Рассмотрим, например, гауссовское априорное распределение вида

$$p(\mathbf{w}|\alpha) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \alpha^{-1}\mathbf{I}) = \left(\frac{\alpha}{2\pi}\right)^{(M+1)/2} \exp\left\{-\frac{\alpha}{2} \mathbf{w}^T \mathbf{w}\right\}$$

где α — точность многомерного гауссовского распределения, а M — порядок полинома, то есть размерность вектора параметров $\mathbf{w}$. Тогда из теоремы Байеса следует, что

$$p(\mathbf{w}|x, t, \alpha, \beta) \propto p(t|x, \mathbf{w}, \beta)p(\mathbf{w}|\alpha)$$

Следовательно, можно определить $\mathbf{w}$, максимизируя апостериорное распределение. Этот метод известен как *метод максимального апостериорного распределения*(MAP) или MAP-вывод. Максимум апостериорного распределения определяется минимумом отрицательного логарифма от $p(\mathbf{w}|x, t, \alpha, \beta)$:

$$\frac{\beta}{2} \sum_{n=1}^N (y(x, \mathbf{w}) - t_n)^2 + \frac{\alpha}{2} \mathbf{w}^T \mathbf{w}$$

Таким образом, мы видим, что максимизация апостериорного распределения эквивалентна минимизации регуляризованной суммы квадратов функции ошибок, встречающейся при регуляризации, где $\lambda = \alpha/\beta$.

Байесовская аппроксимация кривой

$$p(t|x, \mathbf{x}, \mathbf{t}) = \int p(t|x, \mathbf{w})p(\mathbf{w}|\mathbf{x}, \mathbf{t})d\mathbf{w}$$

Интегрирование в данной формуле можно выполнить аналитически, в результате чего прогностическое распределение принимает вид нормального распределения:

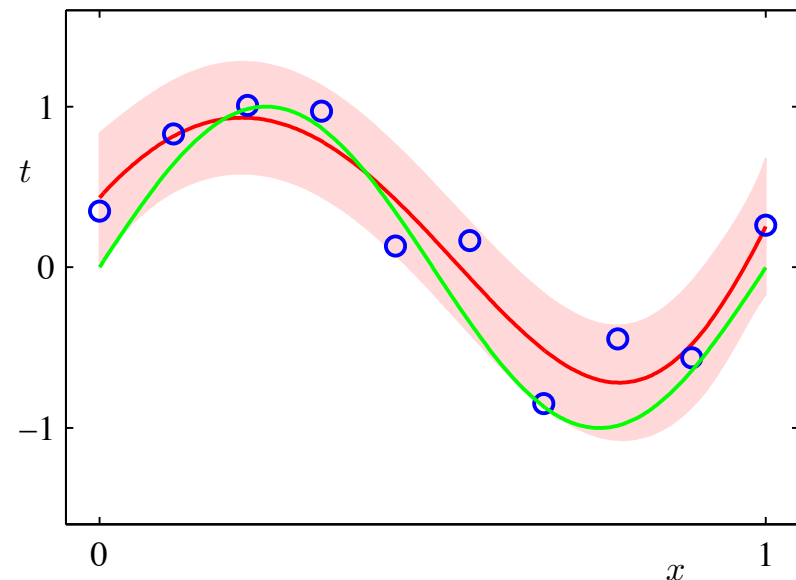
$$p(t|x, \mathbf{x}, \mathbf{t}) = N(t|m(x), s^2(x))$$

$$m(x) = \beta \phi(x)^T S \sum_{n=1}^N \phi(x_n) t_n$$

$$s^2(x) = \beta^{-1} + \phi(x)^T S \phi(x)$$

$$S^{-1} = \alpha \mathbf{I} + \beta \sum_{n=1}^N \phi(x_n) \phi(x_n)^T$$

$$\phi(x) = (x^i: i = \overline{0, M})$$



Выбор модели

В примере, посвященном аппроксимации полиномиальных кривых с использованием метода наименьших квадратов, существует оптимальный порядок полинома, который дает наилучшее обобщение.

Порядок полинома определяет количество свободных параметров в модели и тем самым определяет ее сложность.

В регуляризованном методе наименьших квадратов на реальную сложность модели влияет коэффициент регуляризации в то время как для более сложных моделей, таких как смеси распределений или нейронные сети, может существовать множество параметров, определяющих их сложность.

Для практического применения нам необходимо определить значения таких параметров, и главная цель при этом - достижение наилучших прогностических характеристик параметров сложности в данной модели.

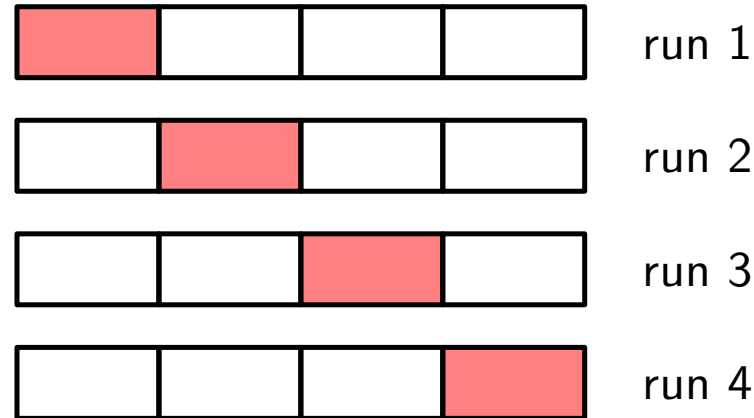
Следует рассмотреть ряд различных видов моделей, попытаюсь найти лучшую для конкретного приложения.

No free lunch theorem

Все модели неверны, но некоторые из них полезны. — Джордж Бокс

В литературе описано множество самых разных моделей, поэтому естественно спросить, какая из них наилучшая. К сожалению, не существует одной модели, которая была бы оптимальна для всех задач, — иногда этот факт называют **теоремой об отсутствии бесплатных завтраков**. Причина в том, что набор допущений (иногда называемый **индуктивным смещением**), который хорошо работает в одной предметной области, может давать никуда не годные результаты в другой. При выборе подходящей модели лучше всего опираться на знание предметной области и (или) на метод проб и ошибок (т. е. применять такие способы выбора модели, как перекрестная проверка или байесовские методы. Поэтому так важно иметь в своем арсенале много моделей и алгоритмических методов.

Перекрёстная проверка



- Метод S -групповой перекрестной проверки, проиллюстрированный здесь для случая $S = 4$, включает в себя получение доступных данных и разбиение их на S групп (в простейшем случае они имеют одинаковый размер). Затем $S - 1$ групп используются для обучения набора моделей, а оценка точности осуществляется на оставшейся группе. После этого описанная процедура повторяется для всех возможных вариантов выбора S групп, обозначенных здесь красным цветом, а показатели точности, полученные в результате S сеансов обучения, усредняются
- Если данных мало то можно взять $S=N$
- Если параметров много, то изучение комбинаций таких параметров могло бы в худшем случае потребовать нескольких сеансов обучения, количество которых экспоненциально растет при увеличении количества параметров.

Штраф за сложность

Информационный критерий Акаике, или (Akaike, AIC):

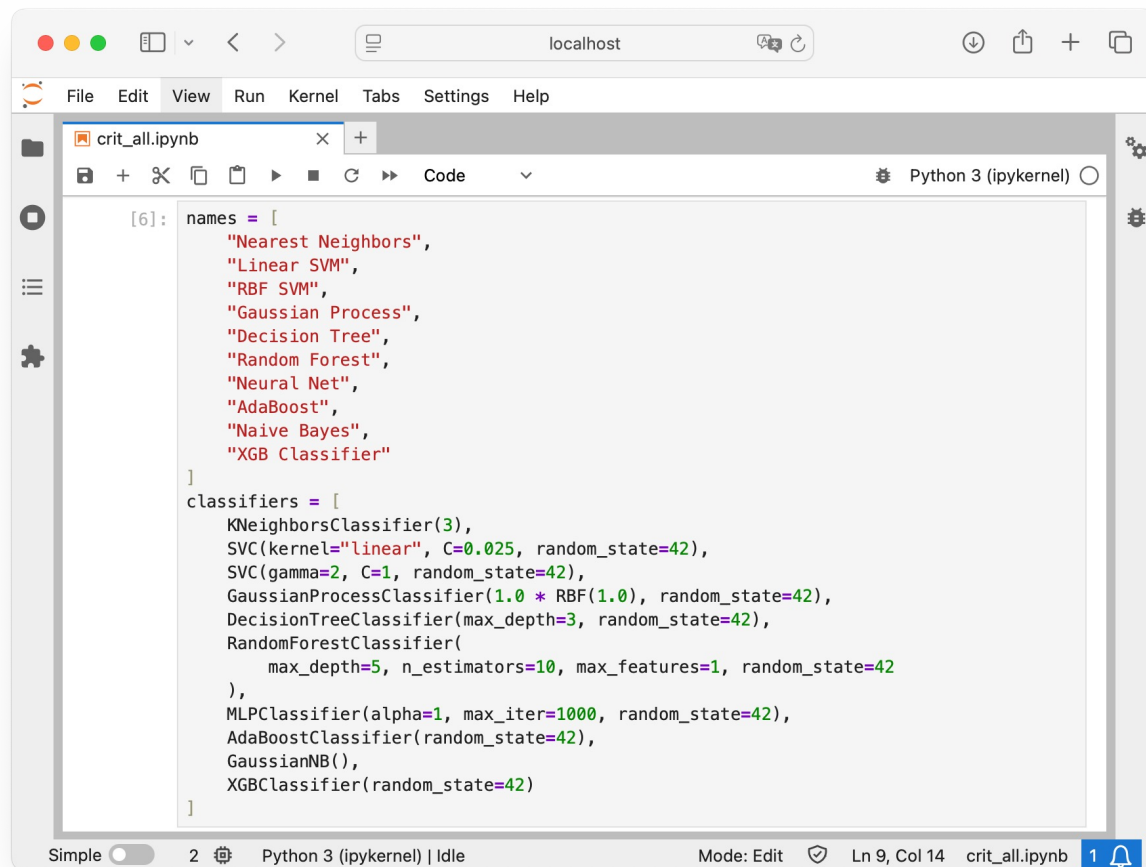
$$\ln p(D|w_{ML}) - M$$

$\ln p(D|w_{ML})$ - максимум логарифмической функции правдоподобия.

M – число параметров модели.

Однако такие критерии не учитывают неопределенность в параметрах модели, и на практике они, как правило, предпочитают слишком простые модели.

Пример выбора модели



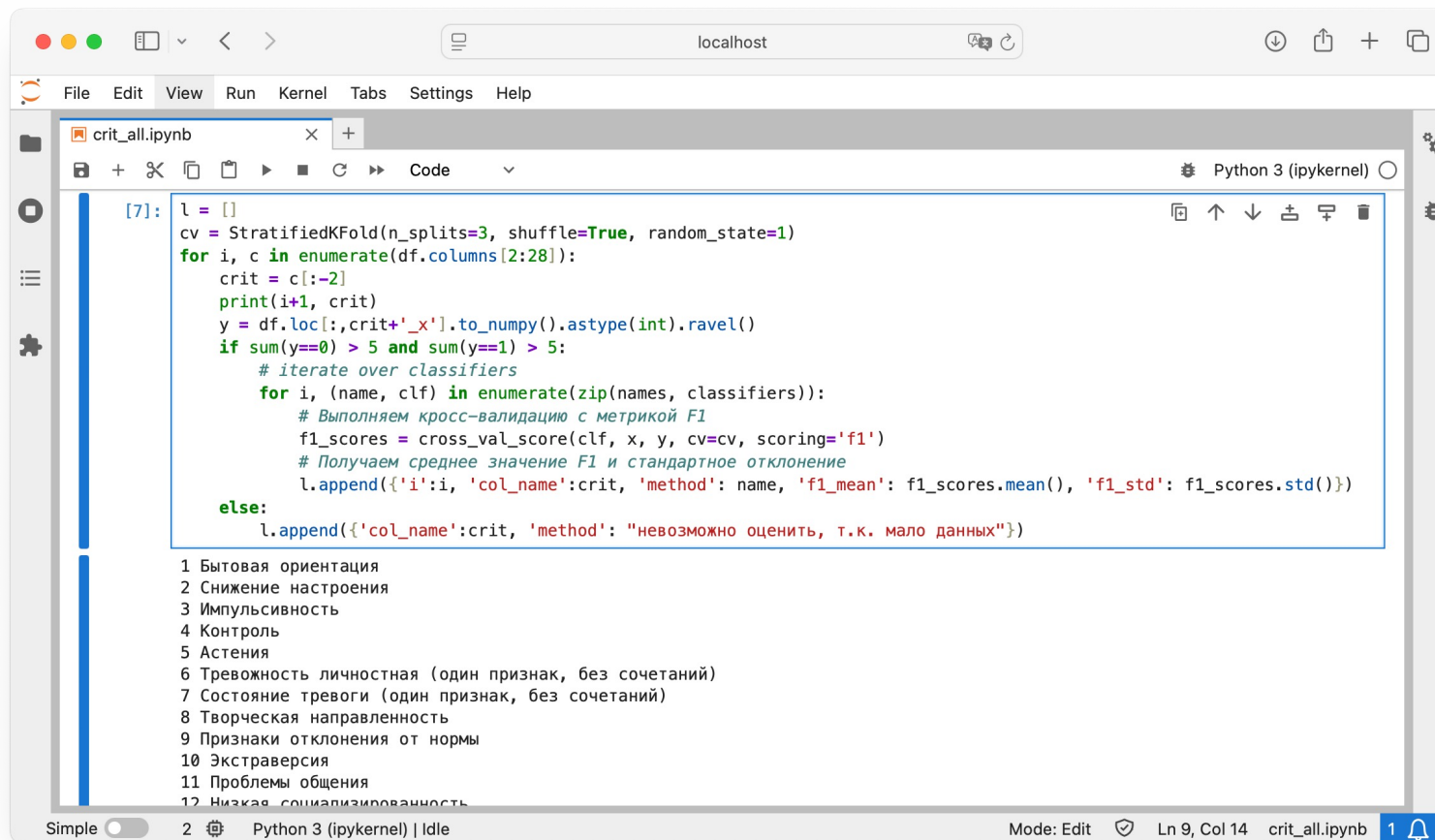
The screenshot shows a Jupyter Notebook window titled 'crit_all.ipynb' running on 'localhost'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Tabs, Settings, Help) and a toolbar with icons for file operations and execution. The code cell [6] contains the following Python code:

```
[6]: names = [
    "Nearest Neighbors",
    "Linear SVM",
    "RBF SVM",
    "Gaussian Process",
    "Decision Tree",
    "Random Forest",
    "Neural Net",
    "AdaBoost",
    "Naive Bayes",
    "XGB Classifier"
]

classifiers = [
    KNeighborsClassifier(3),
    SVC(kernel="linear", C=0.025, random_state=42),
    SVC(gamma=2, C=1, random_state=42),
    GaussianProcessClassifier(1.0 * RBF(1.0), random_state=42),
    DecisionTreeClassifier(max_depth=3, random_state=42),
    RandomForestClassifier(
        max_depth=5, n_estimators=10, max_features=1, random_state=42
    ),
    MLPClassifier(alpha=1, max_iter=1000, random_state=42),
    AdaBoostClassifier(random_state=42),
    GaussianNB(),
    XGBClassifier(random_state=42)
]
```

The status bar at the bottom indicates 'Simple' mode, 'Python 3 (ipykernel) | Idle', 'Mode: Edit', 'Ln 9, Col 14', and the file name 'crit_all.ipynb'.

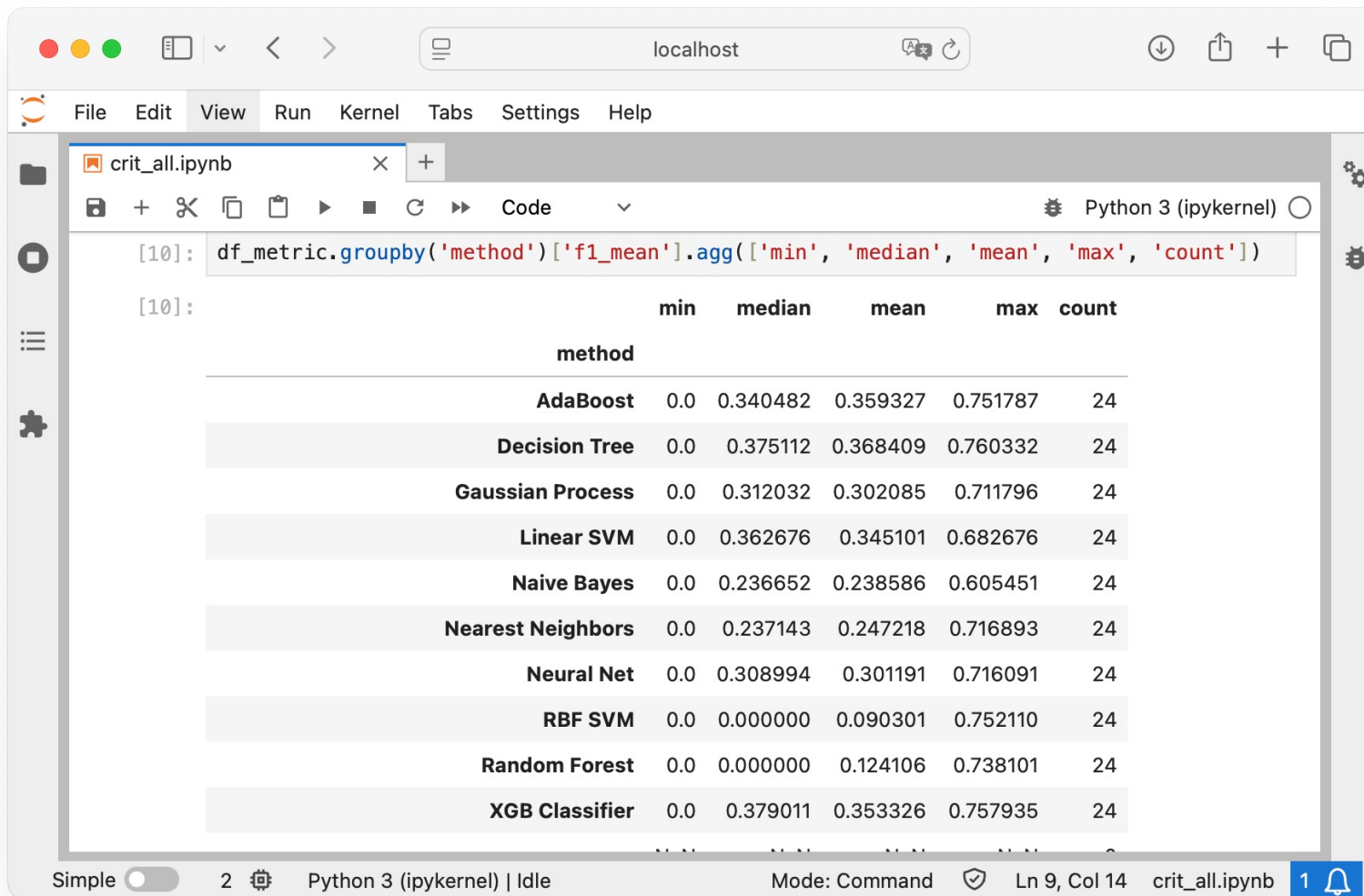
Пример выбора модели



```
[7]: l = []
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=1)
for i, c in enumerate(df.columns[2:28]):
    crit = c[:-2]
    print(i+1, crit)
    y = df.loc[:, crit+'_X'].to_numpy().astype(int).ravel()
    if sum(y==0) > 5 and sum(y==1) > 5:
        # iterate over classifiers
        for i, (name, clf) in enumerate(zip(names, classifiers)):
            # Выполняем кросс-валидацию с метрикой F1
            f1_scores = cross_val_score(clf, x, y, cv=cv, scoring='f1')
            # Получаем среднее значение F1 и стандартное отклонение
            l.append({'i':i, 'col_name':crit, 'method': name, 'f1_mean': f1_scores.mean(), 'f1_std': f1_scores.std()})
    else:
        l.append({'col_name':crit, 'method': "невозможно оценить, т.к. мало данных"})
```

- 1 Бытовая ориентация
- 2 Снижение настроения
- 3 Импульсивность
- 4 Контроль
- 5 Астенция
- 6 Тревожность личностная (один признак, без сочетаний)
- 7 Состояние тревоги (один признак, без сочетаний)
- 8 Творческая направленность
- 9 Признаки отклонения от нормы
- 10 Экстраверсия
- 11 Проблемы общения
- 12 Низкая социализированность

Пример выбора модели



The screenshot shows a Jupyter Notebook window titled 'crit_all.ipynb' running on 'localhost'. The code cell [10] contains the following Python code:

```
df_metric.groupby('method')['f1_mean'].agg(['min', 'median', 'mean', 'max', 'count'])
```

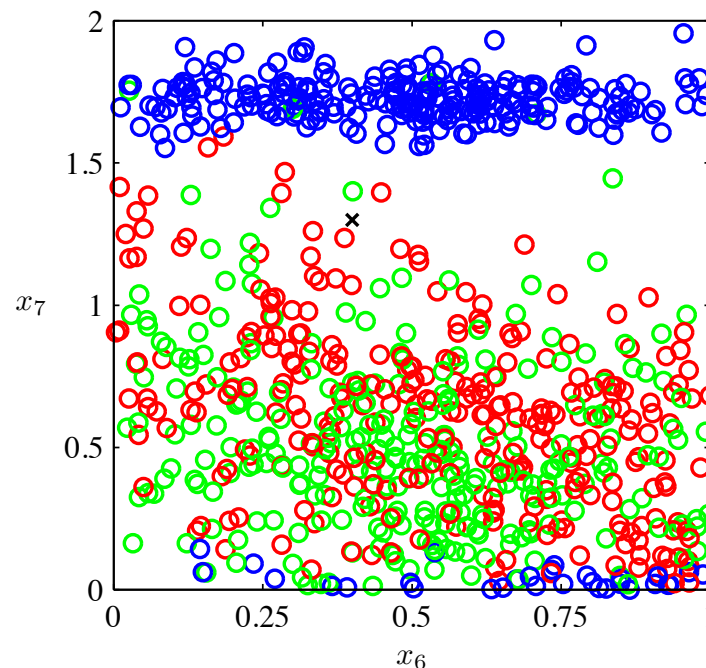
The output cell [10] displays a DataFrame with the following data:

	min	median	mean	max	count
method					
AdaBoost	0.0	0.340482	0.359327	0.751787	24
Decision Tree	0.0	0.375112	0.368409	0.760332	24
Gaussian Process	0.0	0.312032	0.302085	0.711796	24
Linear SVM	0.0	0.362676	0.345101	0.682676	24
Naive Bayes	0.0	0.236652	0.238586	0.605451	24
Nearest Neighbors	0.0	0.237143	0.247218	0.716893	24
Neural Net	0.0	0.308994	0.301191	0.716091	24
RBF SVM	0.0	0.000000	0.090301	0.752110	24
Random Forest	0.0	0.000000	0.124106	0.738101	24
XGB Classifier	0.0	0.379011	0.353326	0.757935	24

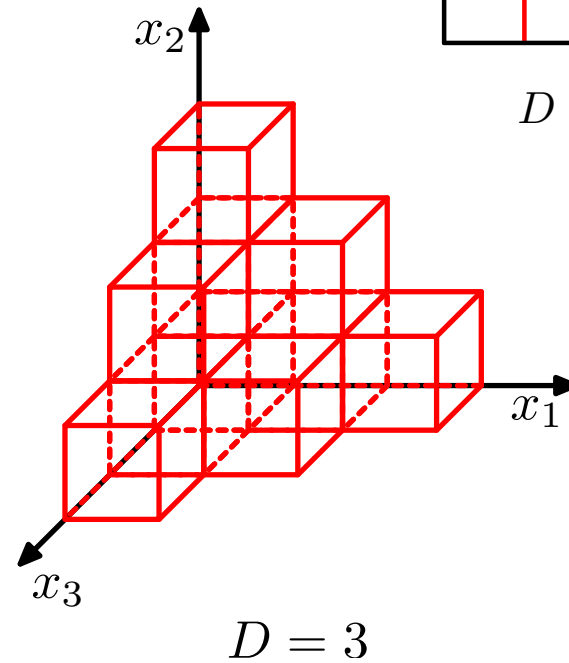
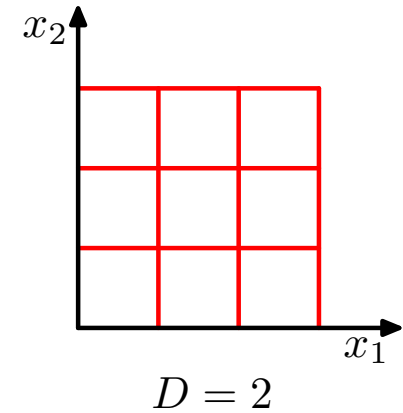
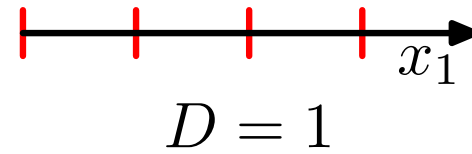
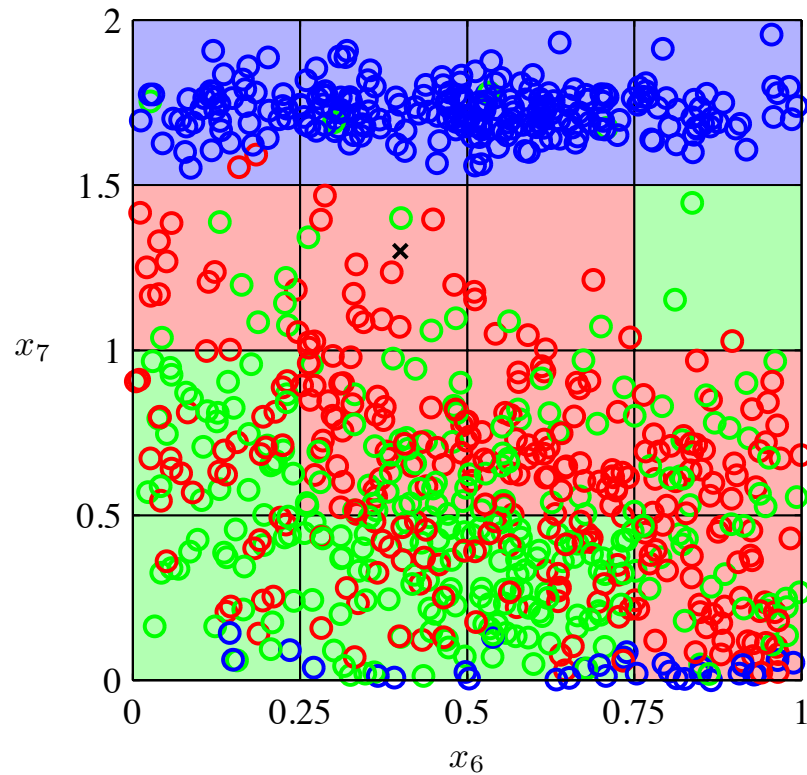
The Jupyter Notebook interface includes a menu bar (File, Edit, View, Run, Kernel, Tabs, Settings, Help), a toolbar with icons for file operations and execution, and a status bar at the bottom showing 'Simple' mode, 'Python 3 (ipykernel) | Idle', 'Mode: Command', and the current cell location 'Ln 9, Col 14'.

Проклятие размерности

На рисунке показаны 100 точек из набора данных в виде диаграммы рассеяния, показывающей 2 из 12 измерений, x_6 и x_7 (остальные десять входных значений не указаны, чтобы не усложнять рисунок). Каждая точка данных помечена в соответствии с тем, какому из трех геометрических классов она принадлежит. Наша цель состоит в том, чтобы использовать эти данные в качестве обучающего множества и получить возможность классифицировать новое наблюдение (x_6 , x_7), обозначен крестиком.



Проклятие размерности



Теория принятия решений

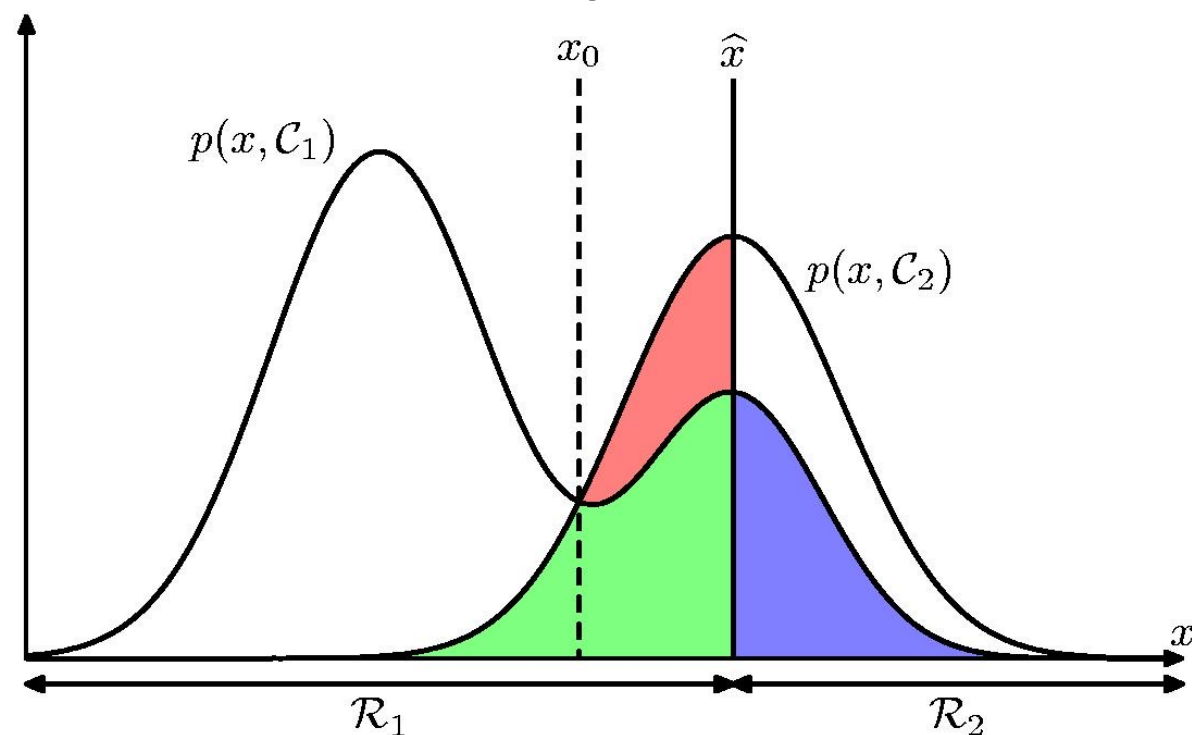
Шаг вывода

- Определите $p(t|\mathbf{x})$ или $p(\mathbf{x}, t)$.

Шаг принятия решений

- Для заданного $\mathbf{x}$, определите оптимальное t .

Минимальная частота ошибочной классификации



$$\begin{aligned} p(\text{mistake}) &= p(\mathbf{x} \in \mathcal{R}_1, C_2) + p(\mathbf{x} \in \mathcal{R}_2, C_1) \\ &= \int_{\mathcal{R}_1} p(\mathbf{x}, C_2) d\mathbf{x} + \int_{\mathcal{R}_2} p(\mathbf{x}, C_1) d\mathbf{x}. \end{aligned}$$

Минимальные ожидаемые потери

Пример: классифицировать медицинские изображения как «рак»(cancer) или «норма».

		Decision	
		cancer	normal
Truth	cancer	0	1000
	normal	1	0

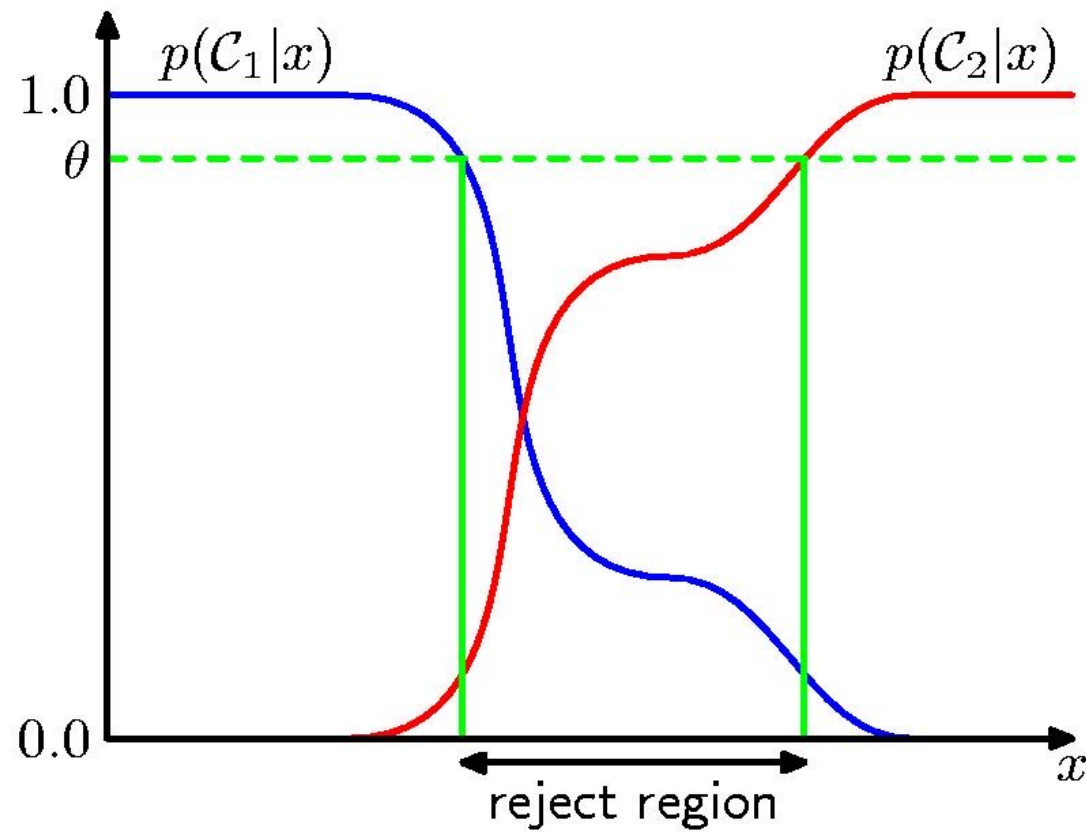
Минимальные ожидаемые потери

$$\mathbb{E}[L] = \sum_k \sum_j \int_{\mathcal{R}_j} L_{kj} p(\mathbf{x}, \mathcal{C}_k) d\mathbf{x}$$

Регионы $\mathcal{R}_j$ выбираются так,
чтобы минимизировать

$$\mathbb{E}[L] = \sum_k L_{kj} p(\mathcal{C}_k | \mathbf{x})$$

Опция «не принимать решение»



3 подхода к принятию решений

Генеративные (порождающие) модели

- Определение условных вероятностей классов $p(x/C_k)$
- Вывод априорных вероятностей классов $p(C_k)$
- Использование теоремы Байеса для нахождения апостериорных вероятностей классов $p(C_k/x) = p(x/C_k)p(C_k)/p(x)$
- Или сразу определять $p(x, C_k)$ и далее определять условные вероятности через нормировку
- Использование теории принятия решений для отнесения каждого нового x к одному из классов

Дискриминантные модели

- Определение апостериорных вероятностей классов $p(C_k/x)$
- Использование теории принятия решений для отнесения каждого нового x к одному из классов

Нахождение дискриминантной функции

- $f(x)$ отображает x на метку класса
- В этом случае вероятности не играют никакой роли. Единая задача обучения.

Преимущества и недостатки

- Первый подход является наиболее требовательным, поскольку предполагает поиск совместного распределения по x и C_k . Во многих приложениях вектор x будет иметь большую размерность, и, следовательно, нам может понадобиться большой набор обучающих данных, чтобы иметь возможность определять условные по классу плотности (class-conditional densities) с разумной точностью.
- Априорные вероятности классов $p(C_k)$ часто можно вычислить просто как долю точек из множества обучающих данных, принадлежащих каждому из классов.
- Одним из преимуществ первого подхода является то, что он также позволяет определить маргинальную плотность данных $p(x)$ для обнаружения по модели новых точек с низкой вероятностью и точек, прогнозы которых могут иметь низкую точность. Это явление называется обнаружением выбросов или обнаружением новизны.

Зачем разделять вывод и решение?

Минимизация риска (матрица потерь может меняться со временем)

Опция «Не принимать решение»

Несбалансированные априорные классы.

- Рассмотрим нашу медицинскую рентгенологическую задачу еще раз и предположим, что мы собрали большое количество рентгеновских снимков населения в целом для использования в качестве обучающих данных с целью создания автоматизированной системы скрининга. Поскольку рак встречается редко среди населения в целом, то если бы мы использовали такой набор данных для обучения адаптивной модели, мы могли бы столкнуться с серьезными трудностями из-за небольшой доли класса рака. Например, классификатор, который назначает каждую точку нормальному классу, уже достиг бы 99,9% точности, и было бы трудно избежать этого тривиального решения. Кроме того, даже большой набор данных будет содержать очень мало примеров рентгеновских снимков, соответствующих раку, и поэтому алгоритм обучения не будет подвергаться широкому диапазону примеров таких изображений и, следовательно, вряд ли будет хорошо обобщать. Сбалансированный набор данных, в котором мы выбрали равное количество примеров из каждого из классов, позволил бы нам найти более точную модель. Однако затем нам необходимо компенсировать эффекты наших модификаций обучающих данных. Из теоремы Байеса мы видим, что апостериорные вероятности пропорциональны априорным вероятностям, которые мы можем интерпретировать как доли точек в каждом классе. Поэтому мы можем просто взять апостериорные вероятности, полученные из нашего искусственно сбалансированного набора данных, и сначала разделить их на доли классов в этом наборе данных, а затем умножить на доли классов в генеральной совокупности, к которой мы хотим применить модель. Наконец, нам необходимо выполнить нормализацию, чтобы гарантировать, что сумма новых апостериорных вероятностей равна единице. Обратите внимание, что эту процедуру нельзя применить, если мы обучили дискриминантную функцию напрямую, а не определяли апостериорные вероятности.

Объединение моделей.

- Для сложных приложений мы, возможно, захотим разбить задачу на ряд меньших подзадач, каждая из которых может решаться отдельным модулем. Например, в гипотетической проблеме медицинского диагноза у нас может быть информация, полученная из анализов крови, а также по рентгеновским снимкам. Вместо того чтобы объединять всю эту неоднородную информацию в одном огромном пространстве входных данных, может быть более эффективным создание одной системы для интерпретации рентгеновских изображений и другой для интерпретации показателей крови. Поскольку каждая из двух моделей дает апостериорные вероятности для классов, мы можем систематически комбинировать выходы с использованием правил теории вероятностей.

Теория принятия решений для регрессии

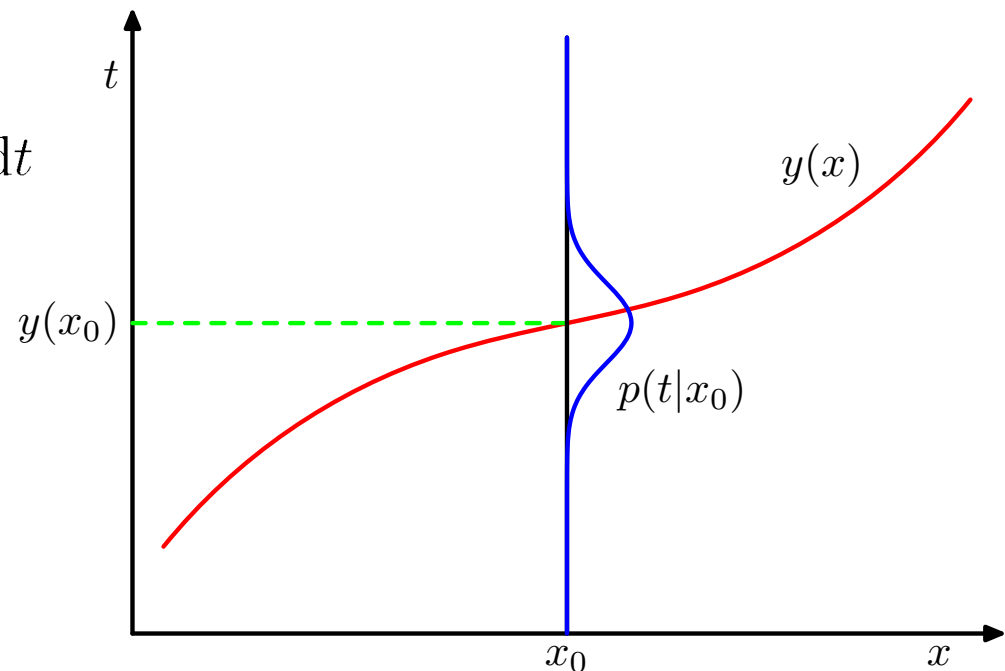
Шаг вывода

- Определить $p(\mathbf{x}, t)$.

Шаг решения

- Для заданного x , построить оптимальный прогноз $y(x)$ для t .
- Функция потерь:

$$\mathbb{E}[L] = \iint L(t, y(\mathbf{x})) p(\mathbf{x}, t) d\mathbf{x} dt$$



Квадратичная функция потерь

$$\mathbb{E}[L] = \iint \{y(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) \, d\mathbf{x} \, dt$$

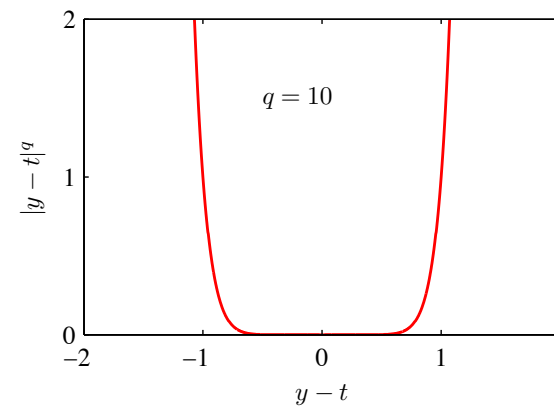
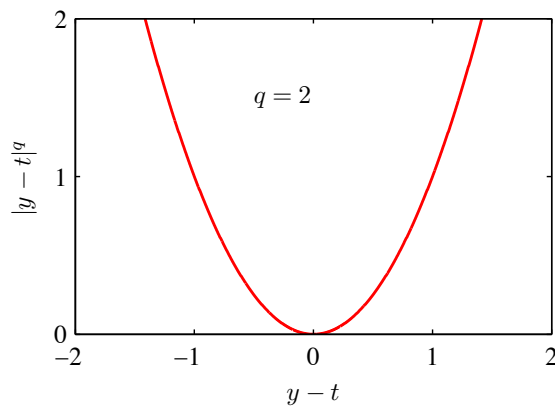
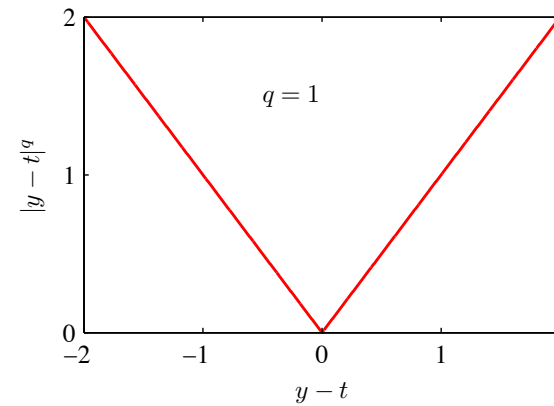
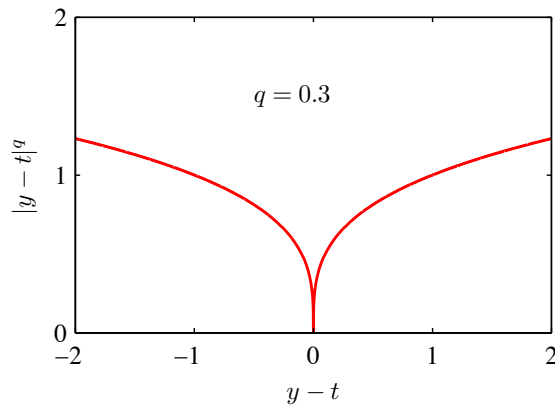
$$\begin{aligned} \{y(\mathbf{x}) - t\}^2 &= \{y(\mathbf{x}) - \mathbb{E}[t|\mathbf{x}] + \mathbb{E}[t|\mathbf{x}] - t\}^2 \\ &= \{y(\mathbf{x}) - \mathbb{E}[t|\mathbf{x}]\}^2 + 2\{y(\mathbf{x}) - \mathbb{E}[t|\mathbf{x}]\}\{\mathbb{E}[t|\mathbf{x}] - t\} + \{\mathbb{E}[t|\mathbf{x}] - t\}^2 \end{aligned}$$

$$\mathbb{E}[L] = \int \{y(\mathbf{x}) - \mathbb{E}[t|\mathbf{x}]\}^2 p(\mathbf{x}) \, d\mathbf{x} + \int \text{var} [t|\mathbf{x}] p(\mathbf{x}) \, d\mathbf{x}$$

$$y(\mathbf{x}) = \mathbb{E}[t|\mathbf{x}]$$

Функция потерь Минковского

$$\mathbb{E}[L_q] = \iint |y(x) - t|^q p(x, t) dx dt$$



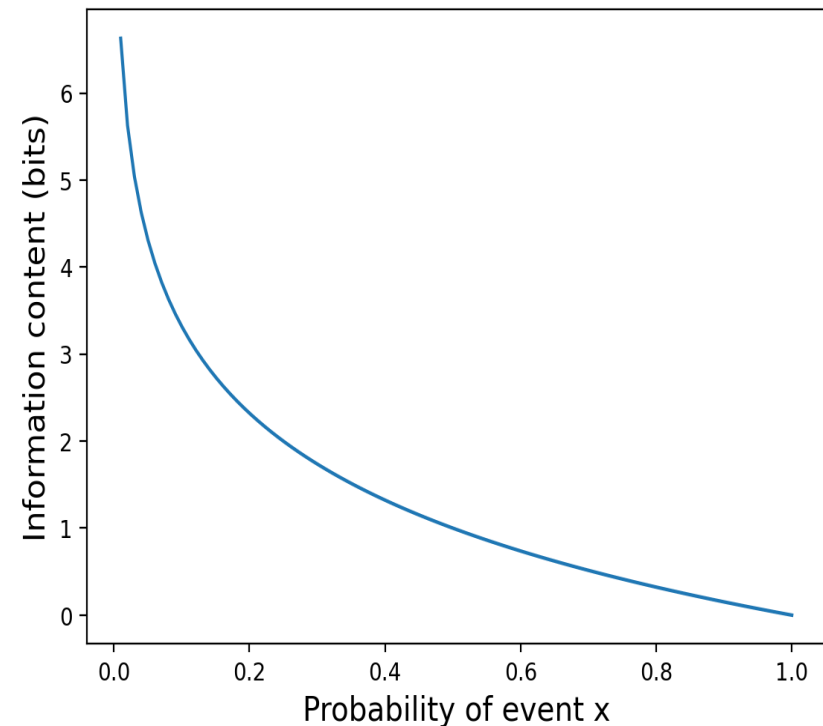
Теория информации

Сколько информации мы получаем, когда наблюдаем определенную переменную?

Рассмотрим, например, событие появления в небе инопланетного космического корабля. Мы никогда его не видели и были бы крайне удивлены, если бы такое событие когда-нибудь произошло, поскольку это неожиданно с учетом наших текущих знаний. Следовательно, количество информации можно рассматривать как степень *удивления* при получении значения x . Мера информационного содержания, зависит от распределения вероятностей $p(x)$ и, в частности, определяется логарифмом $p(x)$:

$$h(x) = -\log_2 p(x)$$

где знак «минус» гарантирует, что $h(x) \geq 0$. Если основание логарифма равно 2, то единицы измерения $h(x)$ — **биты**.

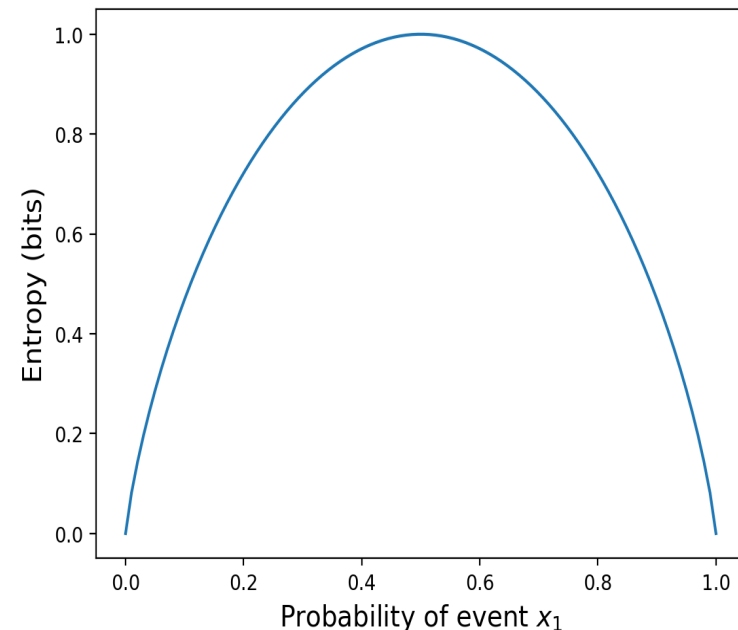


Энтропия

Средний объем информации получается путем умножения ожидаемого значения на содержание информации, заданное формулой:

$$H[x] = - \sum_x p(x) h(x) = - \sum_x p(x) \log_2 p(x)$$

Эта важная величина называется *энтропией* случайной величины X . Рассмотрим дискретную случайную величину, имеющую два возможных значения x_1 с x_2 вероятностями p и $1 - p$:



Энтропия

- Теория кодирования: x дискретен с 8 возможными состояниями; сколько бит нужно для передачи состояния x ?
- Все состояния одинаково вероятны

$$H[x] = -8 \times \frac{1}{8} \log_2 \frac{1}{8} = 3 \text{ bits.}$$

Энтропия

x	a	b	c	d	e	f	g	h
$p(x)$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{64}$	$\frac{1}{64}$	$\frac{1}{64}$	$\frac{1}{64}$
code	0	10	110	1110	111100	111101	111110	111111

$$\begin{aligned} H[x] &= -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{4} \log_2 \frac{1}{4} - \frac{1}{8} \log_2 \frac{1}{8} - \frac{1}{16} \log_2 \frac{1}{16} - \frac{4}{64} \log_2 \frac{1}{64} \\ &= 2 \text{ bits} \end{aligned}$$

$$\begin{aligned} \text{average code length} &= \frac{1}{2} \times 1 + \frac{1}{4} \times 2 + \frac{1}{8} \times 3 + \frac{1}{16} \times 4 + 4 \times \frac{1}{64} \times 6 \\ &= 2 \text{ bits} \end{aligned}$$

Энтропия

- Сколькими способами можно разместить N одинаковых объектов в M ячейках?

$$W = \frac{N!}{\prod_i n_i!}$$

- W – кратность.

$$H = \frac{1}{N} \ln W \simeq - \lim_{N \rightarrow \infty} \sum_i \left(\frac{n_i}{N} \right) \ln \left(\frac{n_i}{N} \right) = - \sum_i p_i \ln p_i$$

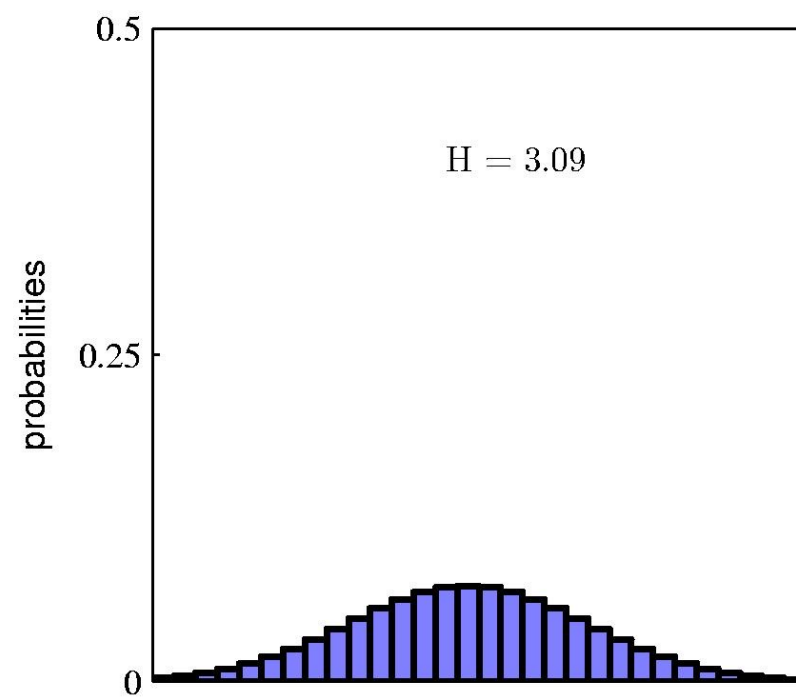
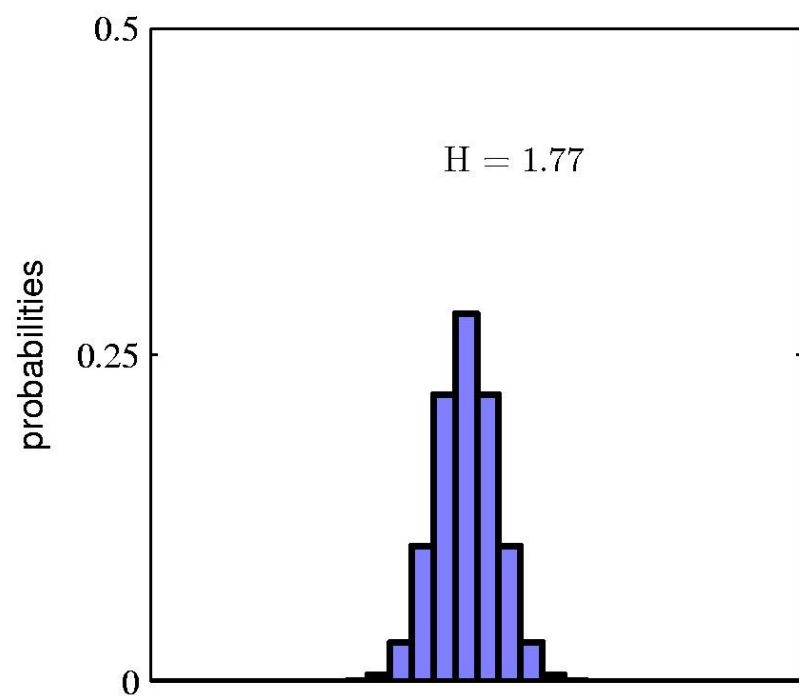
- В выводе использовано приближение Стирлинга:

$$\ln N! = N \ln N - N$$

- Энтропия максимизируется, когда

$$\forall i : p_i = \frac{1}{M}$$

Энтропия



Дифференциальная энтропия

- Разбейте на отрезки длиной Δ вещественную ось:

$$\lim_{\Delta \rightarrow 0} \left\{ - \sum_i p(x_i) \Delta \ln p(x_i) \right\} = - \int p(x) \ln p(x) dx$$

- Максимум дифференциальной энтропии на неограниченной вещественной оси (для фикс. σ^2)

$$p(x) = \mathcal{N}(x|\mu, \sigma^2)$$

- равен

$$H[x] = \frac{1}{2} \{ 1 + \ln(2\pi\sigma^2) \} .$$

Условная энтропия

В случае совместного распределения $p(\mathbf{x}, \mathbf{y})$ средняя дополнительная информация, необходимая для указания $\mathbf{y}$ при том что значение $\mathbf{x}$ уже известно, называется *условной энтропией* и определяется как

$$H[\mathbf{y}|\mathbf{x}] = -\int p(\mathbf{x}, \mathbf{y}) \ln p(\mathbf{y}|\mathbf{x}) d\mathbf{x}d\mathbf{y}$$

Более того, используя правило произведения, легко видеть, что условная энтропия удовлетворяет соотношению:

$$\begin{aligned} H[\mathbf{x}, \mathbf{y}] &= -\int \int p(\mathbf{x}, \mathbf{y}) \ln p(\mathbf{x}, \mathbf{y}) d\mathbf{x}d\mathbf{y} \\ &= -\int \int p(\mathbf{x}, \mathbf{y}) \ln(p(\mathbf{y}|\mathbf{x})p(\mathbf{x})) d\mathbf{x}d\mathbf{y} \\ &= -\int \int p(\mathbf{x}, \mathbf{y}) (\ln p(\mathbf{y}|\mathbf{x}) + \ln p(\mathbf{x})) d\mathbf{x}d\mathbf{y} \\ &= -\int \int p(\mathbf{x}, \mathbf{y}) \ln p(\mathbf{y}|\mathbf{x}) d\mathbf{x}d\mathbf{y} - \int \int p(\mathbf{x}, \mathbf{y}) \ln p(\mathbf{x}) d\mathbf{x}d\mathbf{y} \\ &= -\int \int p(\mathbf{x}, \mathbf{y}) \ln p(\mathbf{y}|\mathbf{x}) d\mathbf{x}d\mathbf{y} - \int \int p(\mathbf{x}, \mathbf{y}) d\mathbf{y} \ln p(\mathbf{x}) d\mathbf{x} \\ &= -\int \int p(\mathbf{x}, \mathbf{y}) \ln p(\mathbf{y}|\mathbf{x}) d\mathbf{x}d\mathbf{y} - \int p(\mathbf{x}) \ln p(\mathbf{x}) d\mathbf{x} \\ &= H[\mathbf{y}|\mathbf{x}] + H[\mathbf{x}] \end{aligned}$$

Информация, необходимая для описания $\mathbf{x}, \mathbf{y}$ представлена $\mathbf{x}$ только информацией, необходимой для его описания, и дополнительной информацией, необходимой для уточнения $\mathbf{y}$.

Дивергенция Кульбака — Лейблера

Рассмотрим некоторое неизвестное распределение $p(\mathbf{x})$ и предположим, что смоделировали его, используя аппроксимирующее распределение $q(\mathbf{x})$. Если мы используем распределение $q(\mathbf{x})$ для создания схемы кодирования с целью передачи значений $\mathbf{x}$ получателю, тогда среднее дополнительное количество информации (измеренное в натах), необходимое для указания значения $\mathbf{x}$ (при условии, что мы выбираем эффективную схему кодирования), в результате использования $q(\mathbf{x})$ вместо истинного распределения $p(\mathbf{x})$ задается формулой

$$\text{KL}(p\|q) = - \int p(\mathbf{x}) \ln q(\mathbf{x}) d\mathbf{x} - \left(- \int p(\mathbf{x}) \ln p(\mathbf{x}) d\mathbf{x} \right)$$

$$= - \int p(\mathbf{x}) \ln \left\{ \frac{q(\mathbf{x})}{p(\mathbf{x})} \right\} d\mathbf{x}$$

$$\text{KL}(p\|q) \simeq \frac{1}{N} \sum_{n=1}^N \{ -\ln q(\mathbf{x}_n|\boldsymbol{\theta}) + \ln p(\mathbf{x}_n) \}$$

$$\text{KL}(p\|q) \geq 0$$

$$\text{KL}(p\|q) \neq \text{KL}(q\|p)$$

Взаимная информация

$$\begin{aligned} I[\mathbf{x}, \mathbf{y}] &\equiv \text{KL}(p(\mathbf{x}, \mathbf{y}) \| p(\mathbf{x})p(\mathbf{y})) \\ &= - \iint p(\mathbf{x}, \mathbf{y}) \ln \left(\frac{p(\mathbf{x})p(\mathbf{y})}{p(\mathbf{x}, \mathbf{y})} \right) d\mathbf{x} d\mathbf{y} \end{aligned}$$

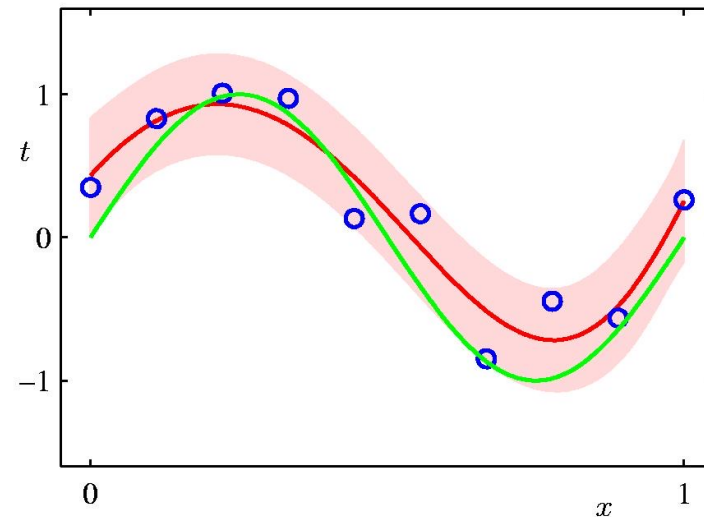
$$I[\mathbf{x}, \mathbf{y}] = H[\mathbf{x}] - H[\mathbf{x}|\mathbf{y}] = H[\mathbf{y}] - H[\mathbf{y}|\mathbf{x}]$$

Параметрические распределения

- Основные блоки: $p(\mathbf{x}|\boldsymbol{\theta})$
- Необходимо определить $\boldsymbol{\theta}$ по $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$
- Представление: $\boldsymbol{\theta}^*$ или $p(\boldsymbol{\theta})$?

- Подгонка кривой

$$p(t|x, \mathbf{x}, \mathbf{t}) = \int p(t|x, \mathbf{w})p(\mathbf{w}|\mathbf{x}, \mathbf{t}) d\mathbf{w}$$



Бинарные переменные (1)

- Подбрасывание монеты: орел=1, решка=0

$$p(x = 1|\mu) = \mu$$

- Распределение Бернулли

$$\text{Bern}(x|\mu) = \mu^x(1 - \mu)^{1-x}$$

$$\mathbb{E}[x] = \mu$$

$$\text{var}[x] = \mu(1 - \mu)$$

Бинарные переменные (2)

- N подбрасываний монеты:

$$p(m \text{ heads} | N, \mu)$$

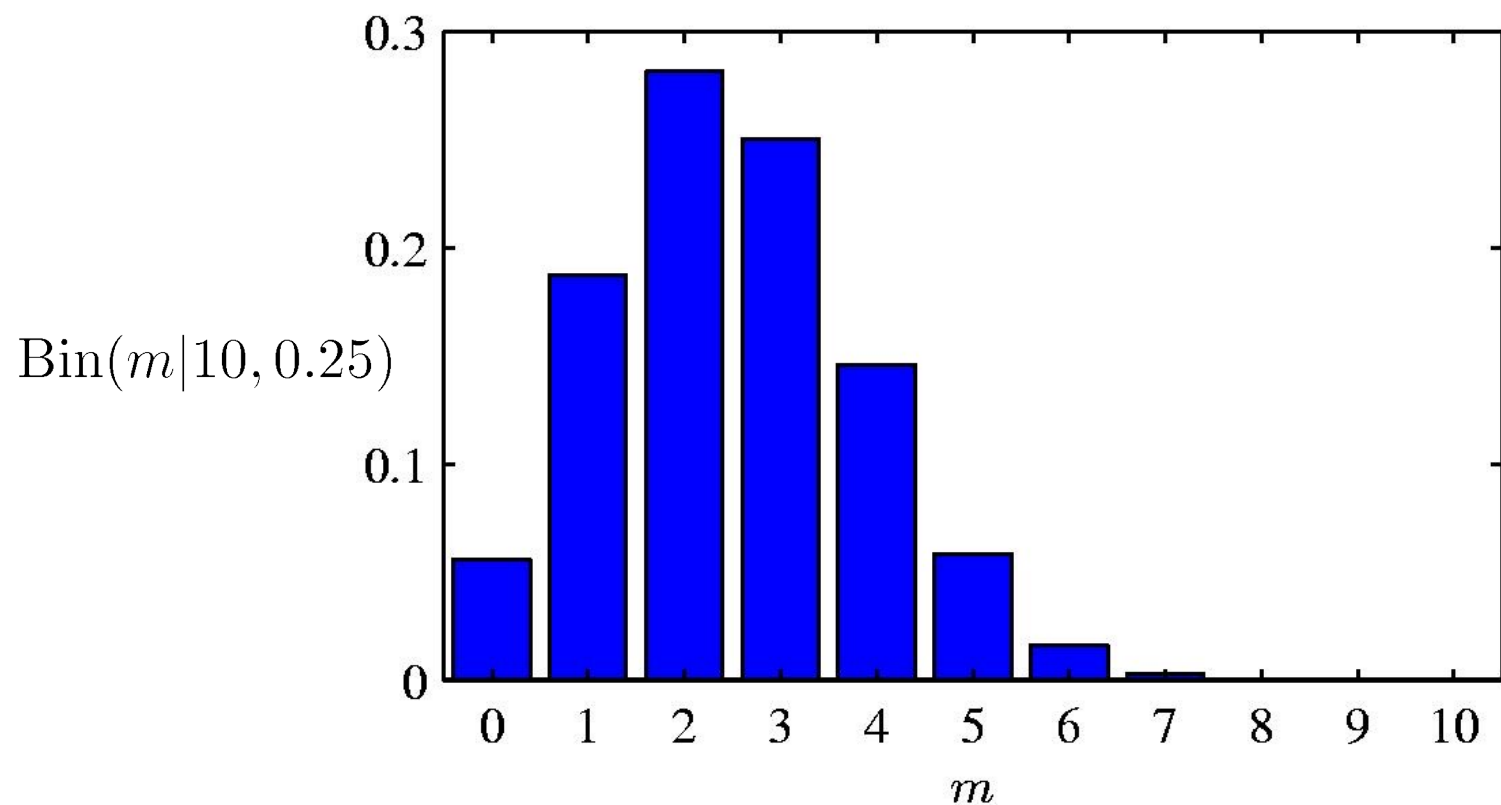
- Биномиальное распределение

$$\text{Bin}(m | N, \mu) = \binom{N}{m} \mu^m (1 - \mu)^{N-m}$$

$$\mathbb{E}[m] \equiv \sum_{m=0}^N m \text{Bin}(m | N, \mu) = N\mu$$

$$\text{var}[m] \equiv \sum_{m=0}^N (m - \mathbb{E}[m])^2 \text{Bin}(m | N, \mu) = N\mu(1 - \mu)$$

Биномиальное распределение



Оценка параметров (1)

- ML для Бернулли

- Дано:

$$\mathcal{D} = \{x_1, \dots, x_N\}, \text{ } m \text{ heads (1), } N - m \text{ tails (0)}$$

$$p(\mathcal{D}|\mu) = \prod_{n=1}^N p(x_n|\mu) = \prod_{n=1}^N \mu^{x_n} (1 - \mu)^{1-x_n}$$

$$\ln p(\mathcal{D}|\mu) = \sum_{n=1}^N \ln p(x_n|\mu) = \sum_{n=1}^N \{x_n \ln \mu + (1 - x_n) \ln(1 - \mu)\}$$

$$\mu_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n = \frac{m}{N}$$

Бета-распределение

Распределение для $\mu \in [0, 1]$.

$$\text{Beta}(\mu|a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \mu^{a-1} (1-\mu)^{b-1}$$

$$\mathbb{E}[\mu] = \frac{a}{a+b}$$

$$\text{var}[\mu] = \frac{ab}{(a+b)^2(a+b+1)}$$

Сопряженное распределение

Распределения $p(\theta) \sim A(\alpha_0)$ и $p(x|\theta) \sim B(\beta)$ являются сопряженными, если $p(\theta|x) \sim A(\alpha_1)$

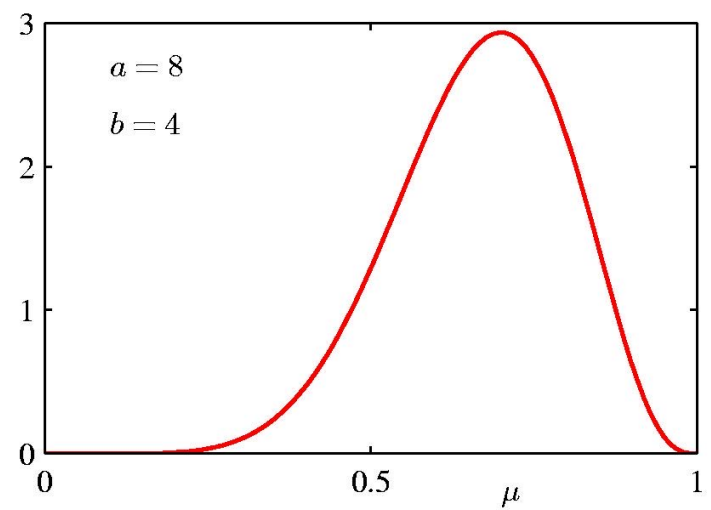
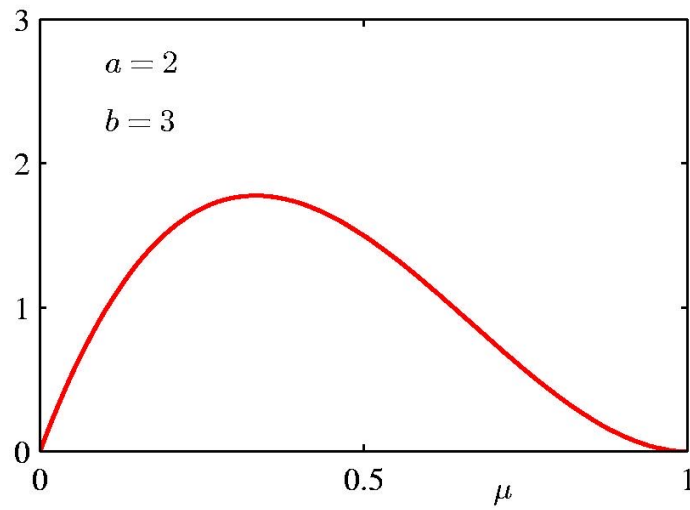
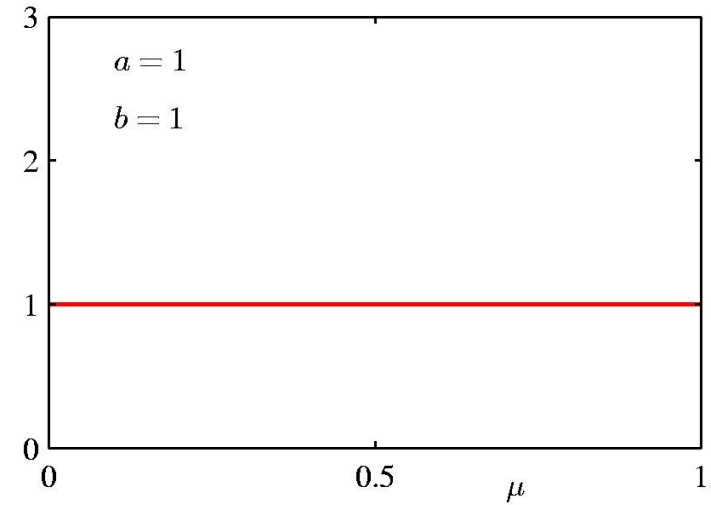
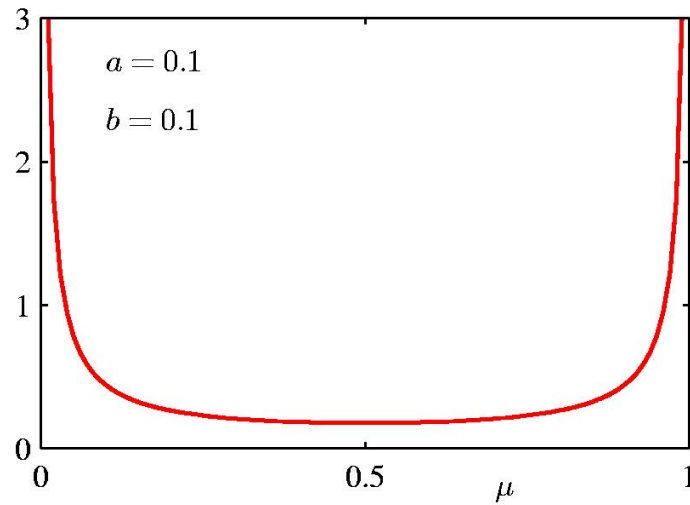
Байесовский Бернулли

$$\begin{aligned} p(\mu|a_0, b_0, \mathcal{D}) &\propto p(\mathcal{D}|\mu)p(\mu|a_0, b_0) \\ &= \left(\prod_{n=1}^N \mu^{x_n} (1 - \mu)^{1-x_n} \right) \text{Beta}(\mu|a_0, b_0) \\ &\propto \mu^{m+a_0-1} (1 - \mu)^{(N-m)+b_0-1} \\ &\propto \text{Beta}(\mu|a_N, b_N) \end{aligned}$$

$$a_N = a_0 + m \quad b_N = b_0 + (N - m)$$

Бета-распределение является *сопряженным* априорным распределением для распределения Бернулли.

Бета-распределение



Мультиномиальные случайные переменные

1-в-K схема кодирования: $\mathbf{x} = (0, 0, 1, 0, 0, 0)^T$

$$p(\mathbf{x}|\boldsymbol{\mu}) = \prod_{k=1}^K \mu_k^{x_k}$$

$$\forall k : \mu_k \geq 0 \quad \text{and} \quad \sum_{k=1}^K \mu_k = 1$$

$$\mathbb{E}[\mathbf{x}|\boldsymbol{\mu}] = \sum_{\mathbf{x}} p(\mathbf{x}|\boldsymbol{\mu}) \mathbf{x} = (\mu_1, \dots, \mu_K)^T = \boldsymbol{\mu}$$

$$\sum_{\mathbf{x}} p(\mathbf{x}|\boldsymbol{\mu}) = \sum_{k=1}^K \mu_k = 1$$

ML Оценка параметров

- Дано: $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$

$$p(\mathcal{D}|\boldsymbol{\mu}) = \prod_{n=1}^N \prod_{k=1}^K \mu_k^{x_{nk}} = \prod_{k=1}^K \mu_k^{(\sum_n x_{nk})} = \prod_{k=1}^K \mu_k^{m_k}$$

- Чтобы гарантировать $\sum_k \mu_k = 1$, используем множитель Лагранжа

$$\sum_{k=1}^K m_k \ln \mu_k + \lambda \left(\sum_{k=1}^K \mu_k - 1 \right)$$

$$\mu_k = -m_k / \lambda \quad \mu_k^{\text{ML}} = \frac{m_k}{N}$$

Мультиномиальное распределение

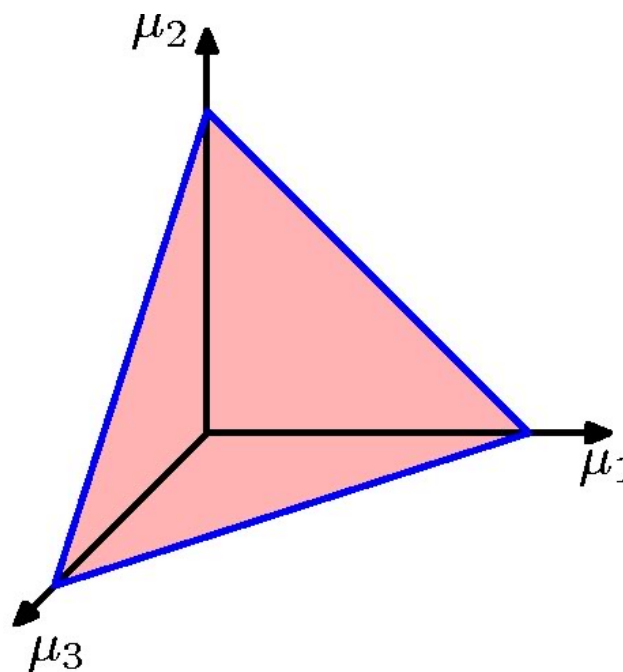
$$\begin{aligned}\text{Mult}(m_1, m_2, \dots, m_K | \boldsymbol{\mu}, N) &= \binom{N}{m_1 m_2 \dots m_K} \prod_{k=1}^K \mu_k^{m_k} \\ \mathbb{E}[m_k] &= N\mu_k \\ \text{var}[m_k] &= N\mu_k(1 - \mu_k) \\ \text{cov}[m_j m_k] &= -N\mu_j\mu_k\end{aligned}$$

Распределение Дирихле

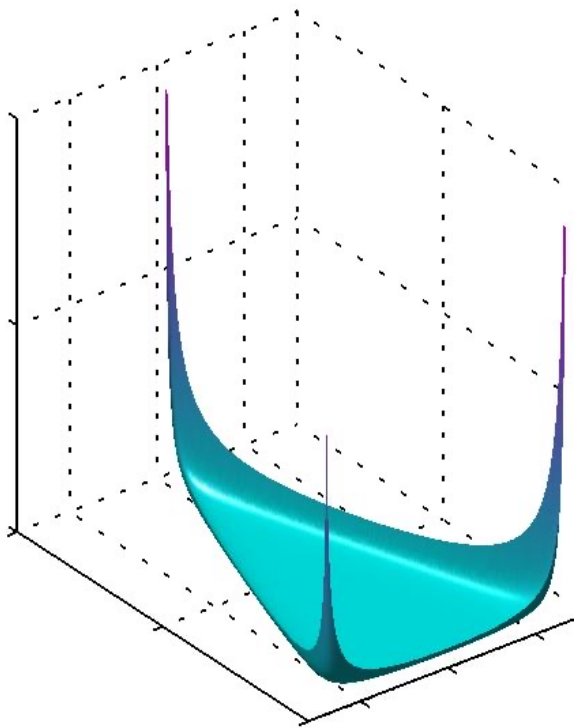
$$\text{Dir}(\boldsymbol{\mu}|\boldsymbol{\alpha}) = \frac{\Gamma(\alpha_0)}{\Gamma(\alpha_1) \cdots \Gamma(\alpha_K)} \prod_{k=1}^K \mu_k^{\alpha_k - 1}$$

$$\alpha_0 = \sum_{k=1}^K \alpha_k$$

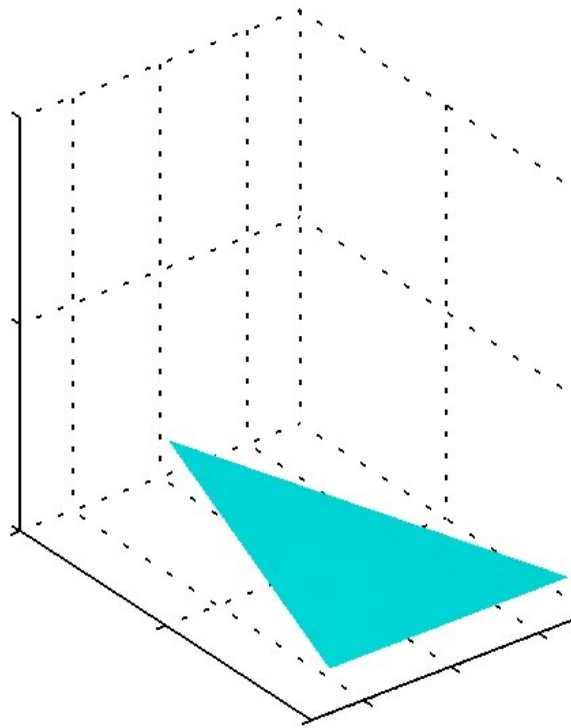
Сопряженное
априорное
распределение для
мультиномиального
распределения.



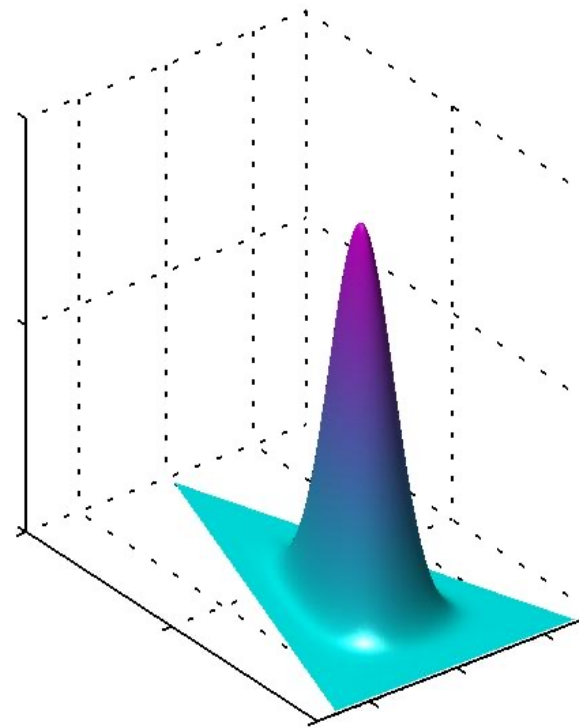
Распределение Дирихле (2)



$$\alpha_k = 10^{-1}$$

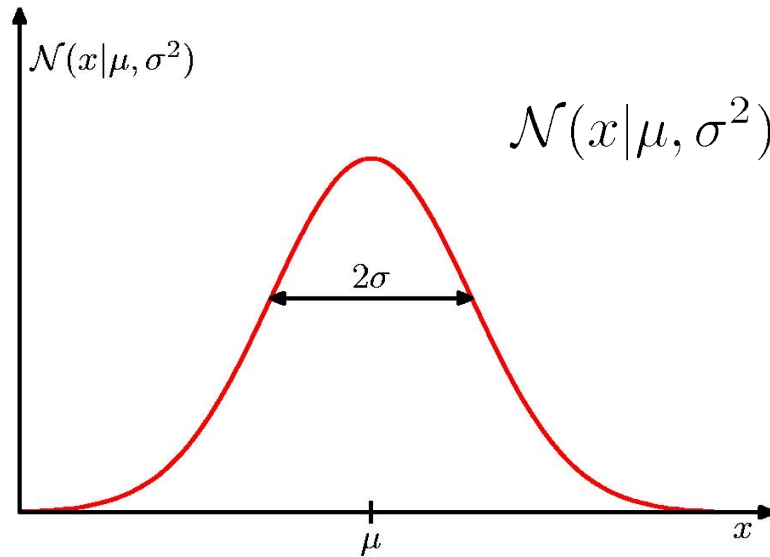


$$\alpha_k = 10^0$$

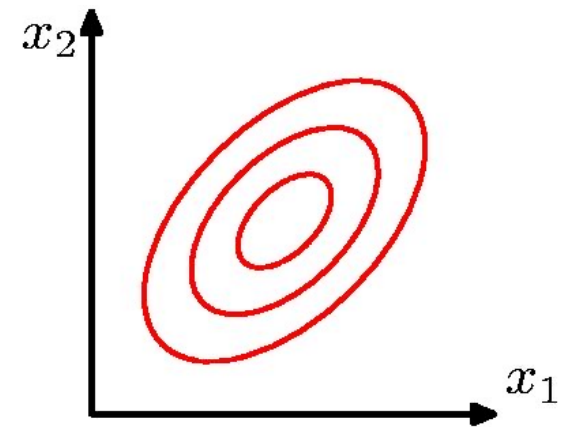


$$\alpha_k = 10^1$$

Нормальное распределение



$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp \left\{ -\frac{1}{2\sigma^2} (x - \mu)^2 \right\}$$

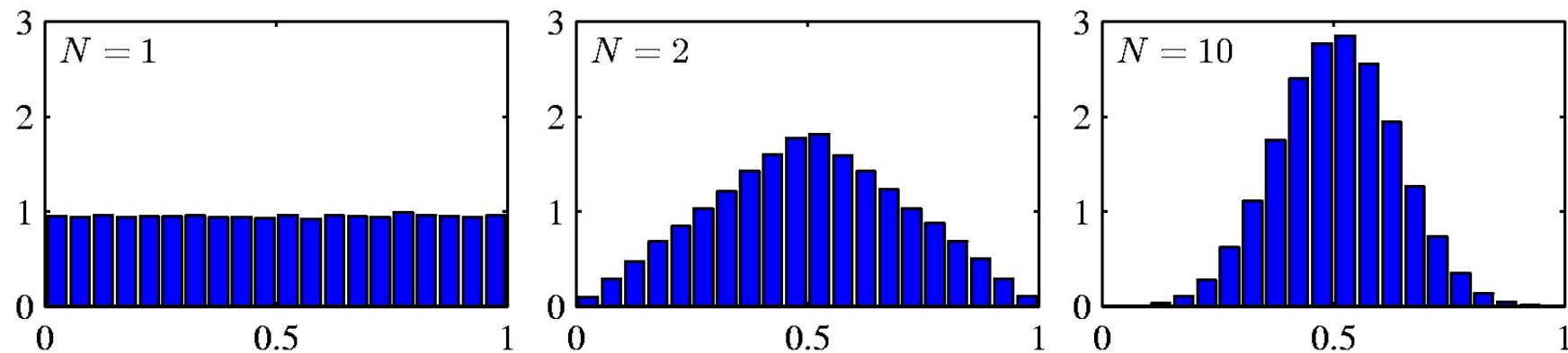


$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\boldsymbol{\Sigma}|^{1/2}} \exp \left\{ -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right\}$$

Центральная предельная теорема

Распределение суммы N н.о.р. случайных переменных приближается к нормальному по мере роста N .

Пример: N равномерных $[0,1]$ случайных переменных.



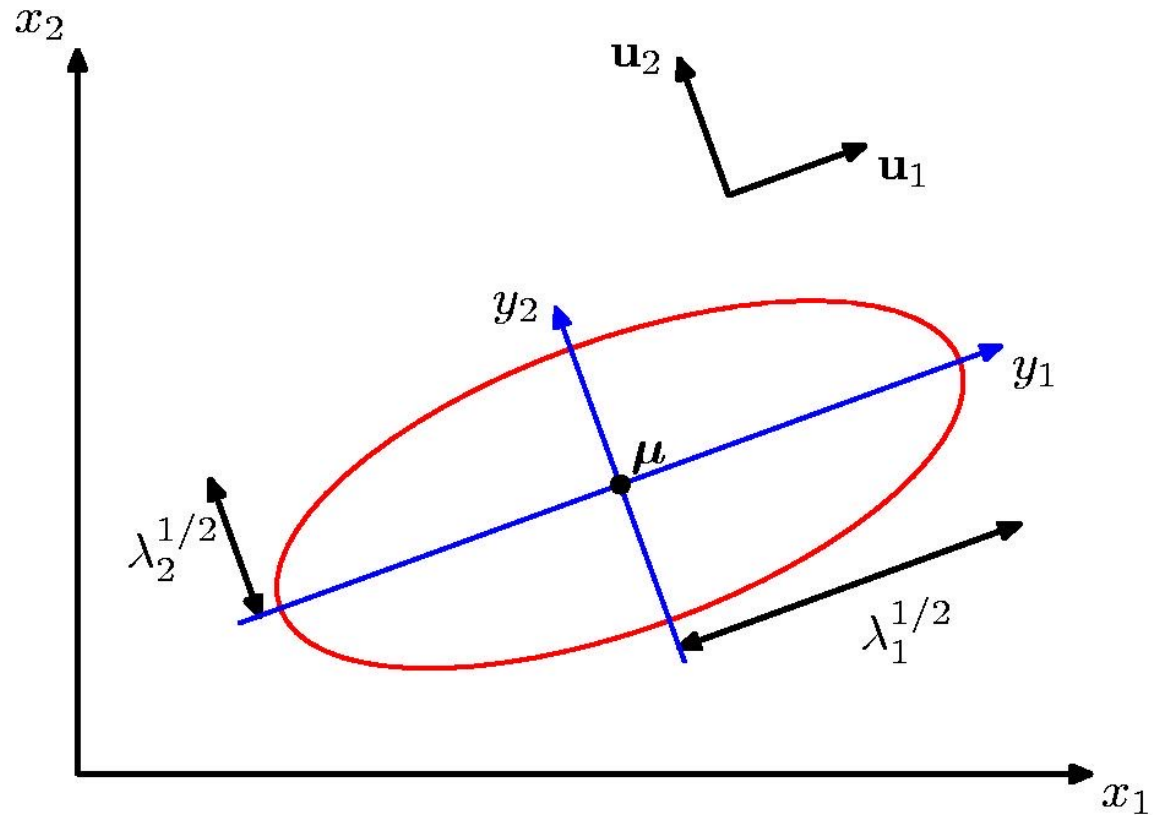
Геометрическое представление многомерного нормального распределения

$$\Delta^2 = (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$$

$$\boldsymbol{\Sigma}^{-1} = \sum_{i=1}^D \frac{1}{\lambda_i} \mathbf{u}_i \mathbf{u}_i^T$$

$$\Delta^2 = \sum_{i=1}^D \frac{y_i^2}{\lambda_i}$$

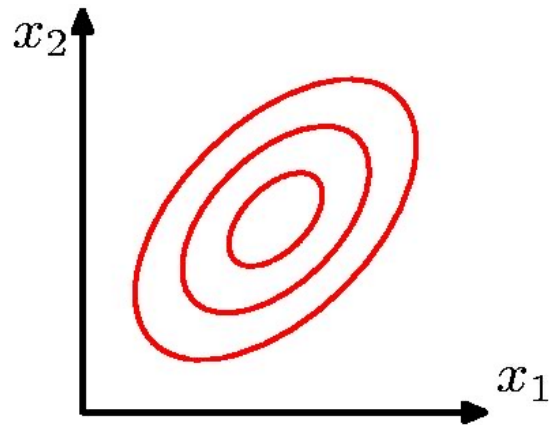
$$y_i = \mathbf{u}_i^T (\mathbf{x} - \boldsymbol{\mu})$$



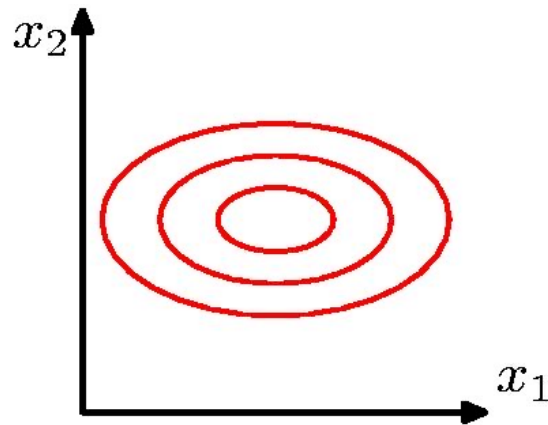
Моменты

$$\mathbb{E}[\mathbf{x}\mathbf{x}^T] = \boldsymbol{\mu}\boldsymbol{\mu}^T + \boldsymbol{\Sigma}$$

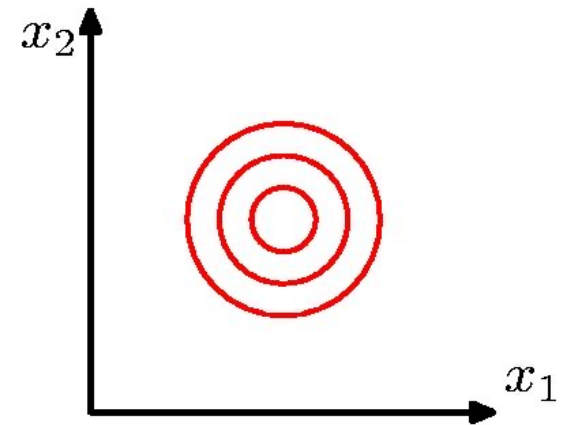
$$\text{cov}[\mathbf{x}] = \mathbb{E}[(\mathbf{x} - \mathbb{E}[\mathbf{x}])(\mathbf{x} - \mathbb{E}[\mathbf{x}])^T] = \boldsymbol{\Sigma}$$



(a)



(b)



(c)

Разделение переменных в нормальном распределении

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

$$\mathbf{x} = \begin{pmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{pmatrix} \quad \boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix} \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}$$

$$\boldsymbol{\Lambda} \equiv \boldsymbol{\Sigma}^{-1} \quad \boldsymbol{\Lambda} = \begin{pmatrix} \boldsymbol{\Lambda}_{aa} & \boldsymbol{\Lambda}_{ab} \\ \boldsymbol{\Lambda}_{ba} & \boldsymbol{\Lambda}_{bb} \end{pmatrix}$$

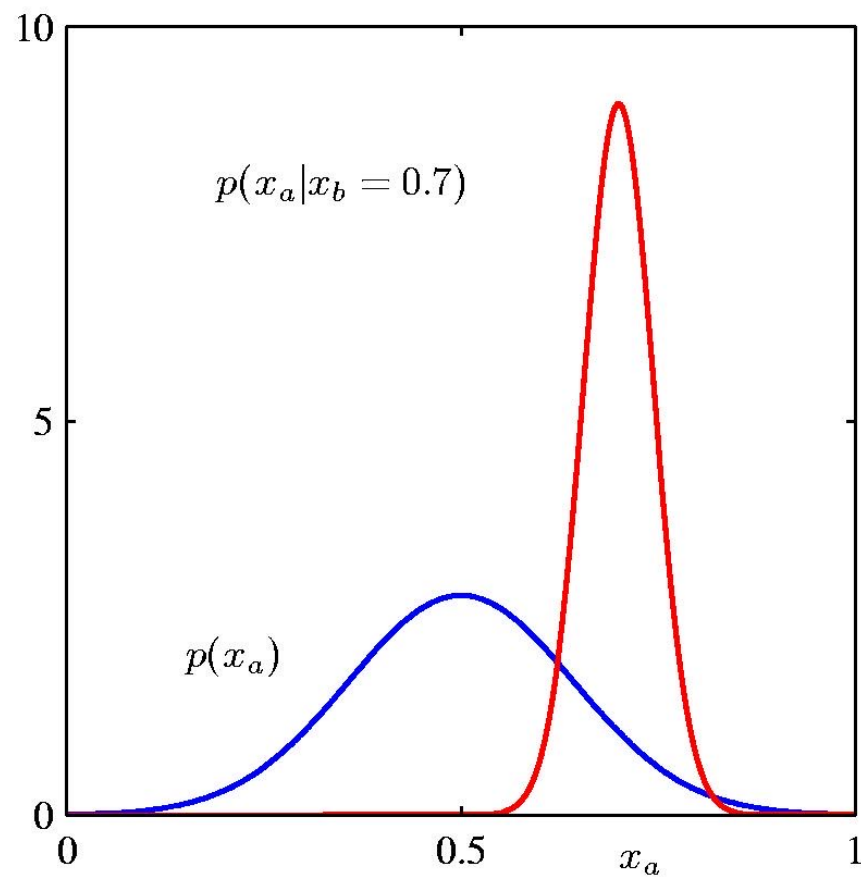
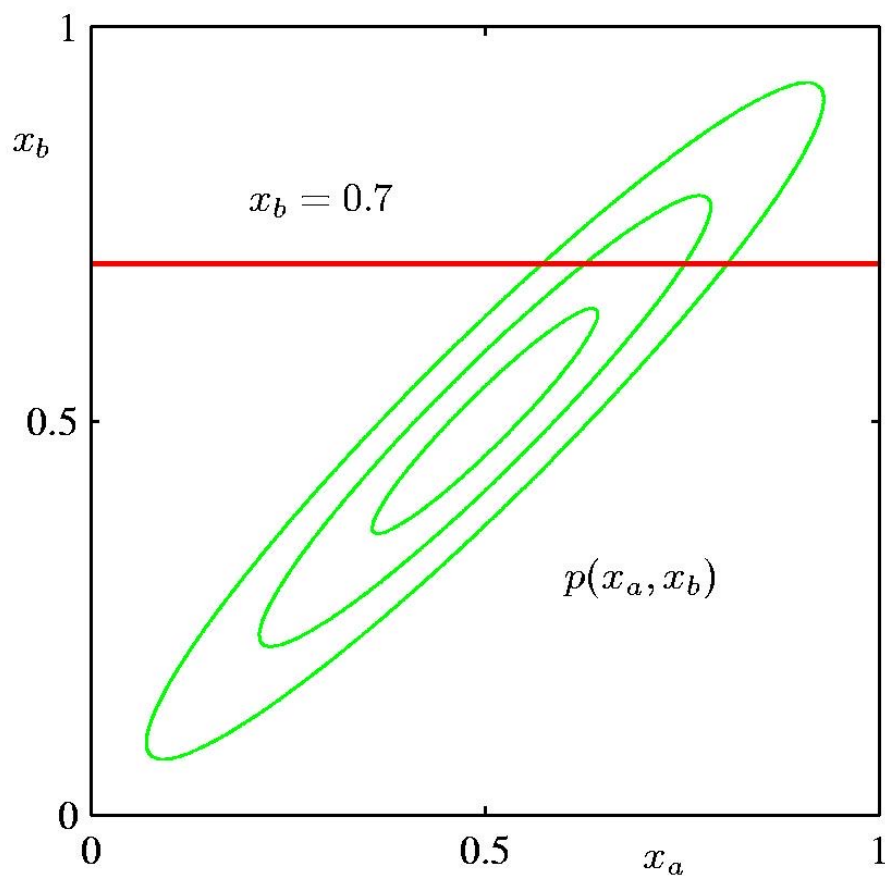
Условные вероятности и частное распределение

$$p(\mathbf{x}_a|\mathbf{x}_b) = \mathcal{N}(\mathbf{x}_a|\boldsymbol{\mu}_{a|b}, \boldsymbol{\Sigma}_{a|b})$$

$$\begin{aligned}\boldsymbol{\Sigma}_{a|b} &= \boldsymbol{\Lambda}_{aa}^{-1} = \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba} \\ \boldsymbol{\mu}_{a|b} &= \boldsymbol{\Sigma}_{a|b} \{ \boldsymbol{\Lambda}_{aa}\boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{ab}(\mathbf{x}_b - \boldsymbol{\mu}_b) \} \\ &= \boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{aa}^{-1}\boldsymbol{\Lambda}_{ab}(\mathbf{x}_b - \boldsymbol{\mu}_b) \\ &= \boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\mathbf{x}_b - \boldsymbol{\mu}_b)\end{aligned}$$

$$\begin{aligned}p(\mathbf{x}_a) &= \int p(\mathbf{x}_a, \mathbf{x}_b) d\mathbf{x}_b \\ &= \mathcal{N}(\mathbf{x}_a|\boldsymbol{\mu}_a, \boldsymbol{\Sigma}_{aa})\end{aligned}$$

Условные вероятности



Теорема Байеса для нормального закона

- Дано

$$\begin{aligned}p(\mathbf{x}) &= \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Lambda}^{-1}) \\p(\mathbf{y}|\mathbf{x}) &= \mathcal{N}(\mathbf{y}|\mathbf{A}\mathbf{x} + \mathbf{b}, \mathbf{L}^{-1})\end{aligned}$$

- Получено

$$\begin{aligned}p(\mathbf{y}) &= \mathcal{N}(\mathbf{y}|\mathbf{A}\boldsymbol{\mu} + \mathbf{b}, \mathbf{L}^{-1} + \mathbf{A}\boldsymbol{\Lambda}^{-1}\mathbf{A}^T) \\p(\mathbf{x}|\mathbf{y}) &= \mathcal{N}(\mathbf{x}|\boldsymbol{\Sigma}\{\mathbf{A}^T\mathbf{L}(\mathbf{y} - \mathbf{b}) + \boldsymbol{\Lambda}\boldsymbol{\mu}\}, \boldsymbol{\Sigma})\end{aligned}$$

- где

$$\boldsymbol{\Sigma} = (\boldsymbol{\Lambda} + \mathbf{A}^T\mathbf{L}\mathbf{A})^{-1}$$

Последовательная оценка

Вклад N -й точки данных, $\mathbf{x}_N$

$$\begin{aligned}\mu_{\text{ML}}^{(N)} &= \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n \\&= \frac{1}{N} \mathbf{x}_N + \frac{1}{N} \sum_{n=1}^{N-1} \mathbf{x}_n \\&= \frac{1}{N} \mathbf{x}_N + \frac{N-1}{N} \mu_{\text{ML}}^{(N-1)} \\&= \underbrace{\mu_{\text{ML}}^{(N-1)}}_{\text{старая оценка}} + \underbrace{\frac{1}{N} (\mathbf{x}_N - \mu_{\text{ML}}^{(N-1)})}_{\text{коррекция веса}}\end{aligned}$$

→ коррекция по $\mathbf{x}_N$

→ Коррекция веса

→ старая оценка

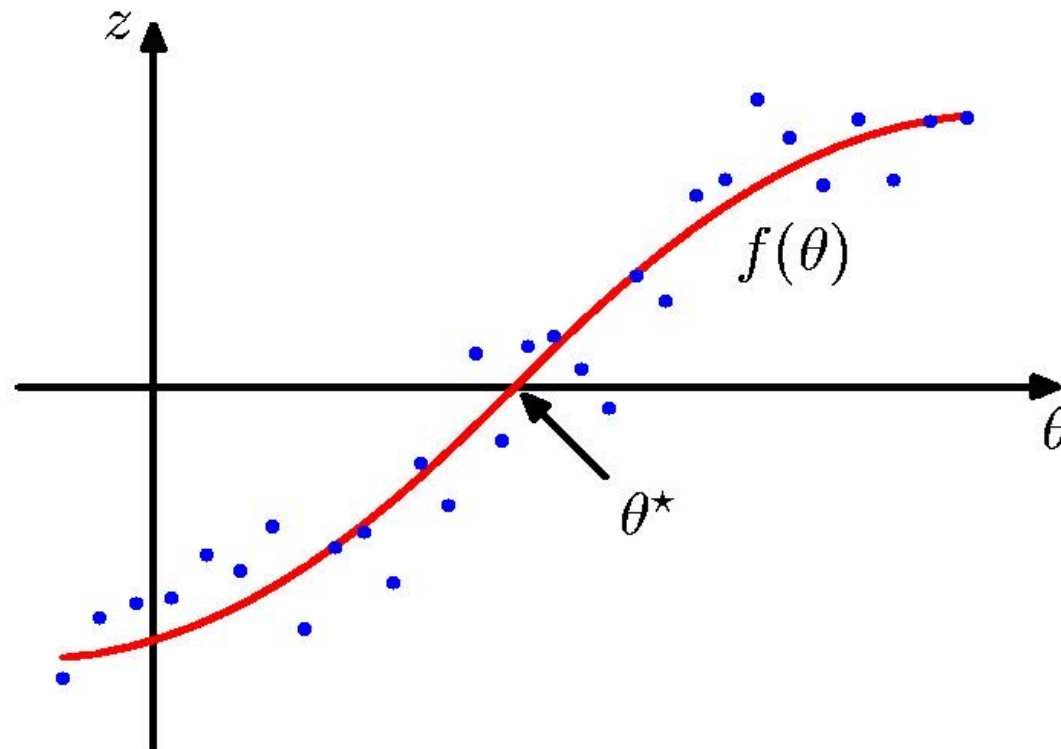
Алгоритм Роббинса-Монро (1)

Рассмотрим θ и z определяемые через $p(z, \theta)$ и определим функцию регрессии

$$f(\theta) \equiv \mathbb{E}[z|\theta] = \int z p(z|\theta) dz$$

Найдем θ^* такую что $f(\theta^*) = 0$.

Алгоритм Роббинса-Монро (2)



Предполагаем, наблюдения из $p(z, \theta)$
поступают по одному.

Алгоритм Роббинса-Монро(3)

Успешная оценка вычисляется по формуле

$$\theta^{(N)} = \theta^{(N-1)} - a_{N-1} z(\theta^{(N-1)}).$$

Условия для a_N :

$$\lim_{N \rightarrow \infty} a_N = 0 \qquad \sum_{N=1}^{\infty} a_N = \infty \qquad \sum_{N=1}^{\infty} a_N^2 < \infty$$

Распределение Стьюдента

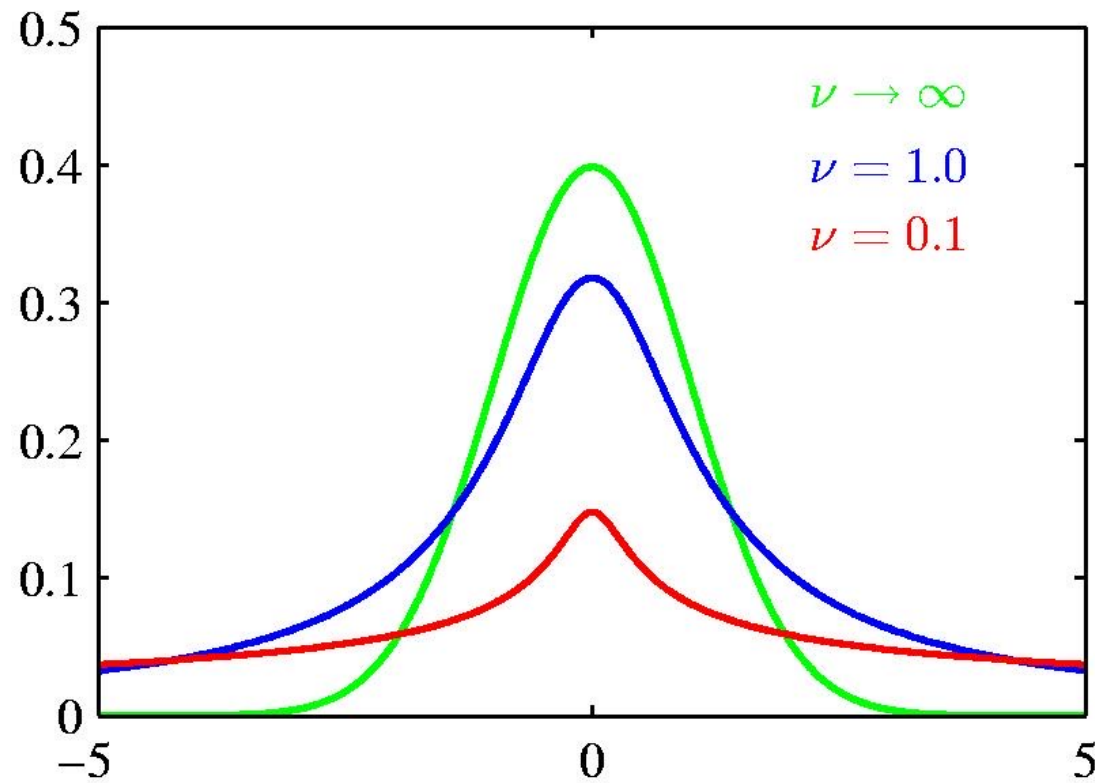
$$\begin{aligned} p(x|\mu, a, b) &= \int_0^\infty \mathcal{N}(x|\mu, \tau^{-1}) \text{Gam}(\tau|a, b) d\tau \\ &= \int_0^\infty \mathcal{N}(x|\mu, (\eta\lambda)^{-1}) \text{Gam}(\eta|\nu/2, \nu/2) d\eta \quad \leftarrow \\ &= \frac{\Gamma(\nu/2 + 1/2)}{\Gamma(\nu/2)} \left(\frac{\lambda}{\pi\nu} \right)^{1/2} \left[1 + \frac{\lambda(x - \mu)^2}{\nu} \right]^{-\nu/2 - 1/2} \\ &= \text{St}(x|\mu, \lambda, \nu) \end{aligned}$$

- где

$$\lambda = a/b \quad \eta = \tau b/a \quad \nu = 2a.$$

- Бесконечная смесь гауссианов.

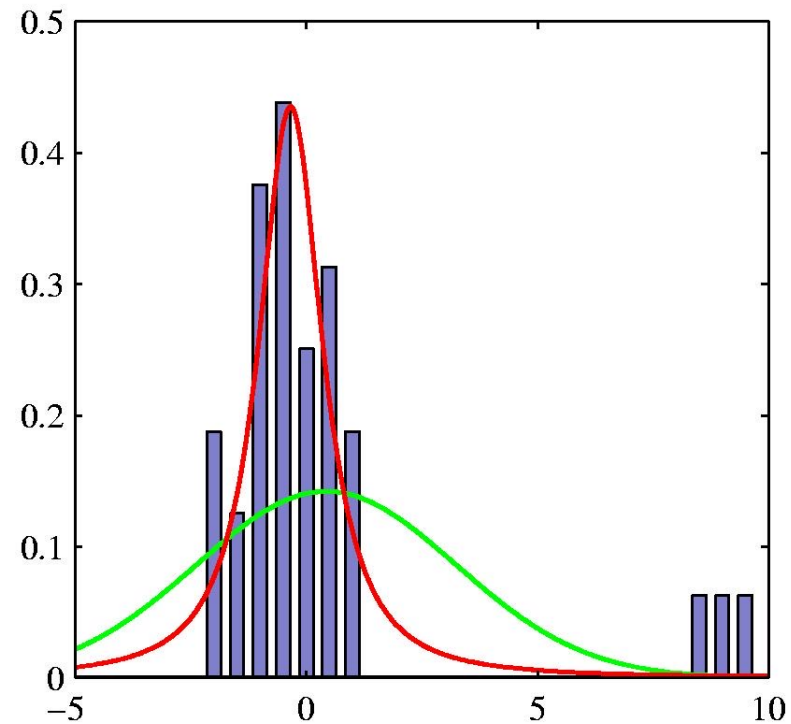
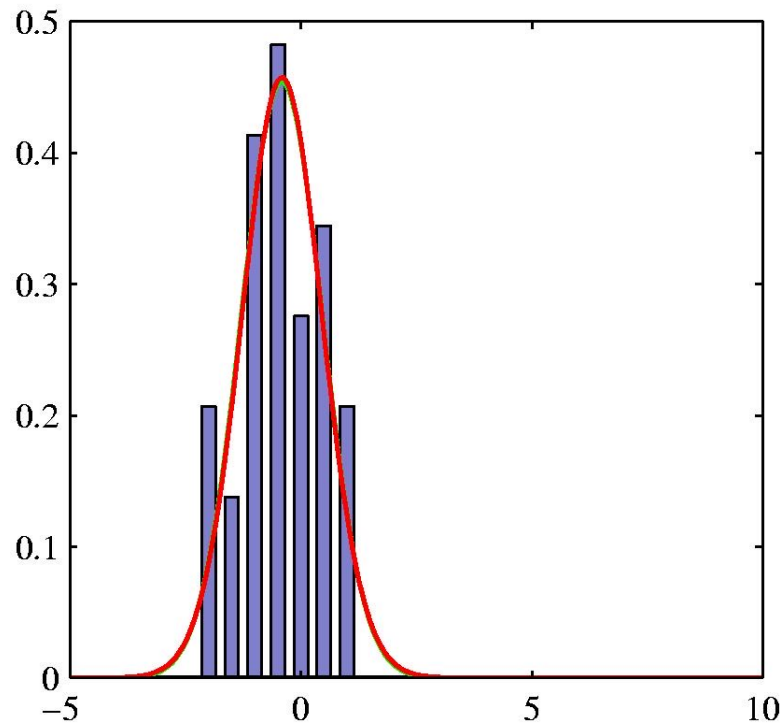
Распределение Стьюдента



	$\nu = 1$	$\nu \rightarrow \infty$
$\text{St}(x \mu, \lambda, \nu)$	Cauchy	$\mathcal{N}(x \mu, \lambda^{-1})$

Распределение Стьюдента

- Робастность: Нормальное vs t-распределение.



Периодические случайные величины

- Примеры: календарное время, направление
- Необходимо чтобы

$$\begin{aligned}p(\theta) &\geq 0 \\ \int_0^{2\pi} p(\theta) d\theta &= 1 \\ p(\theta + 2\pi) &= p(\theta).\end{aligned}$$

Распределение Мизеса (1)

- Требованиям удовлетворяет

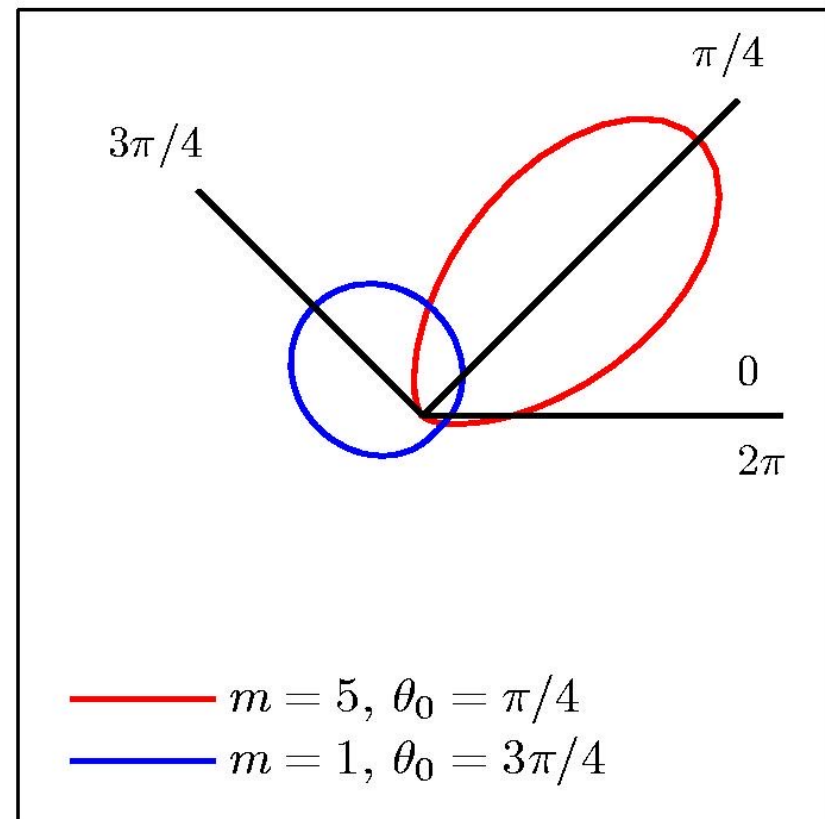
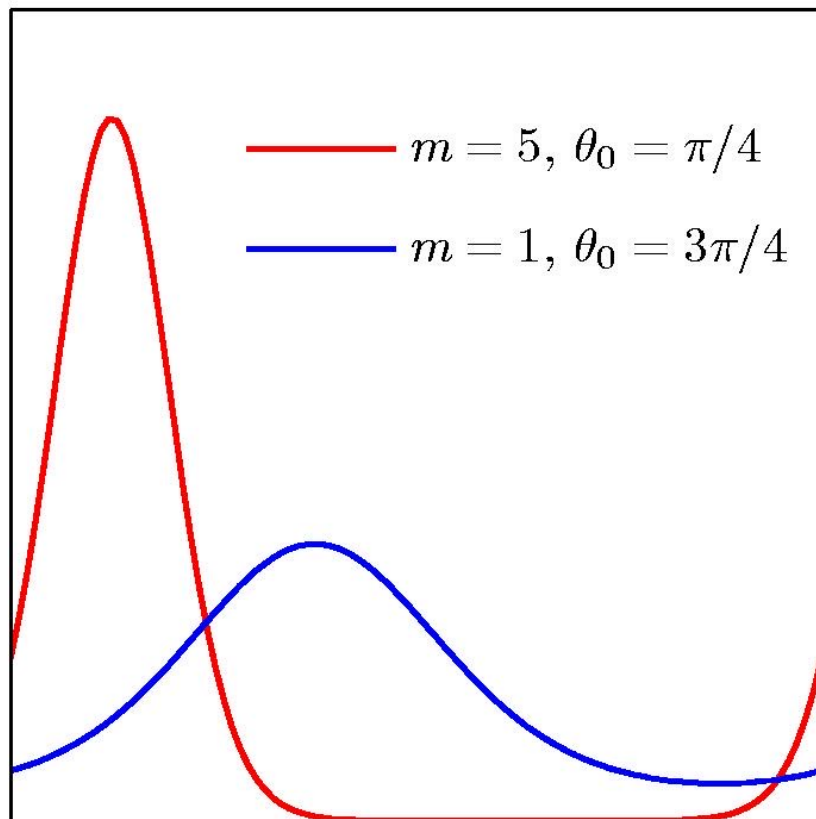
$$p(\theta|\theta_0, m) = \frac{1}{2\pi I_0(m)} \exp \{m \cos(\theta - \theta_0)\}$$

- где

$$I_0(m) = \frac{1}{2\pi} \int_0^{2\pi} \exp \{m \cos \theta\} d\theta$$

- модифицированная функция Бесселя 0-го порядка 1-го рода.

Распределение Мизеса (2)



Максимальное правдоподобие

При наличии набора данных $\mathcal{D} = \{\theta_1, \dots, \theta_N\}$ логарифмическая функция правдоподобия задается как

$$\ln p(\mathcal{D}|\theta_0, m) = -N \ln(2\pi) - N \ln I_0(m) + m \sum_{n=1}^N \cos(\theta_n - \theta_0).$$

Максимизируя по θ_0 , мы сразу получаем

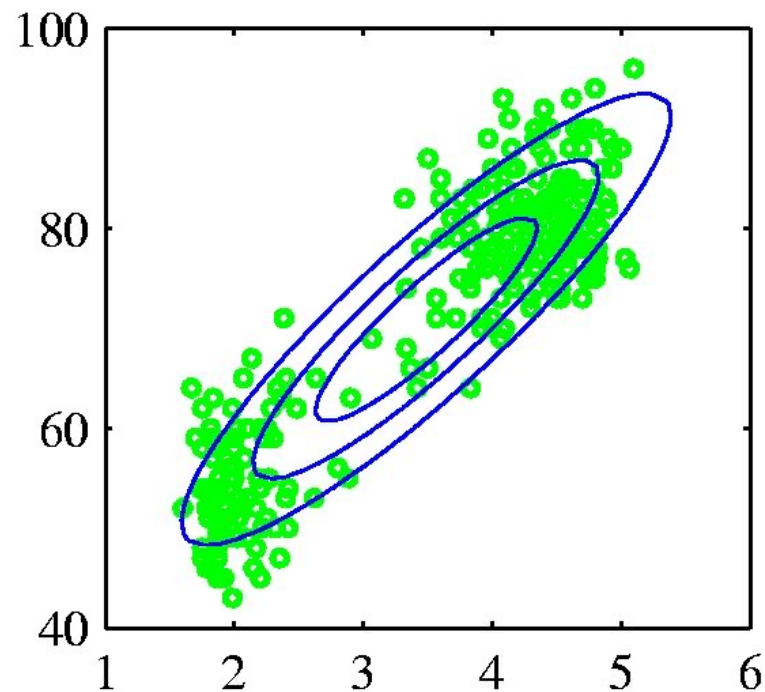
$$\theta_0^{\text{ML}} = \tan^{-1} \left\{ \frac{\sum_n \sin \theta_n}{\sum_n \cos \theta_n} \right\}.$$

Аналогично, максимизируя по m , получаем

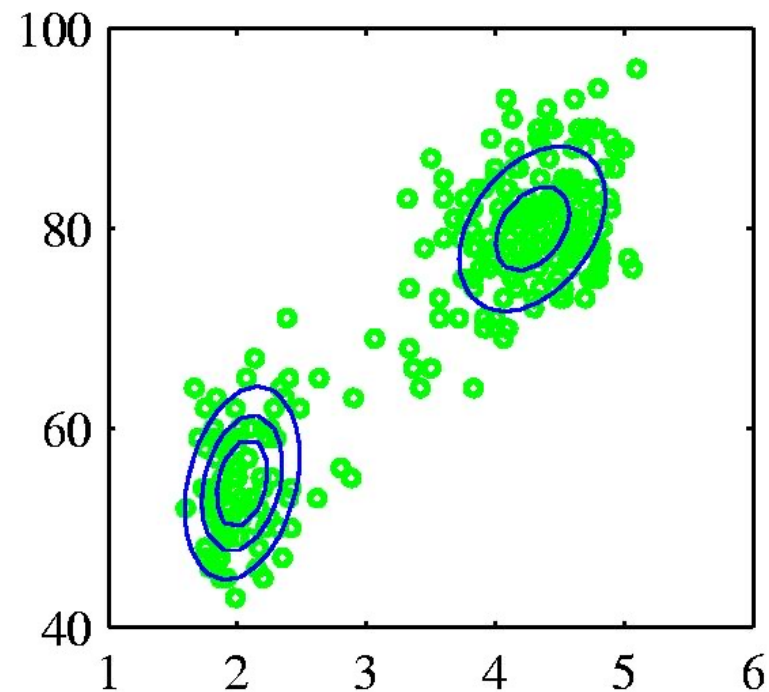
$$\frac{I_1(m_{\text{ML}})}{I_0(m_{\text{ML}})} = \frac{1}{N} \sum_{n=1}^N \cos(\theta_n - \theta_0^{\text{ML}})$$

которое можно решить численно относительно m_{ML} .

Смесь нормальных распределений (1)



Single Gaussian



Mixture of two Gaussians

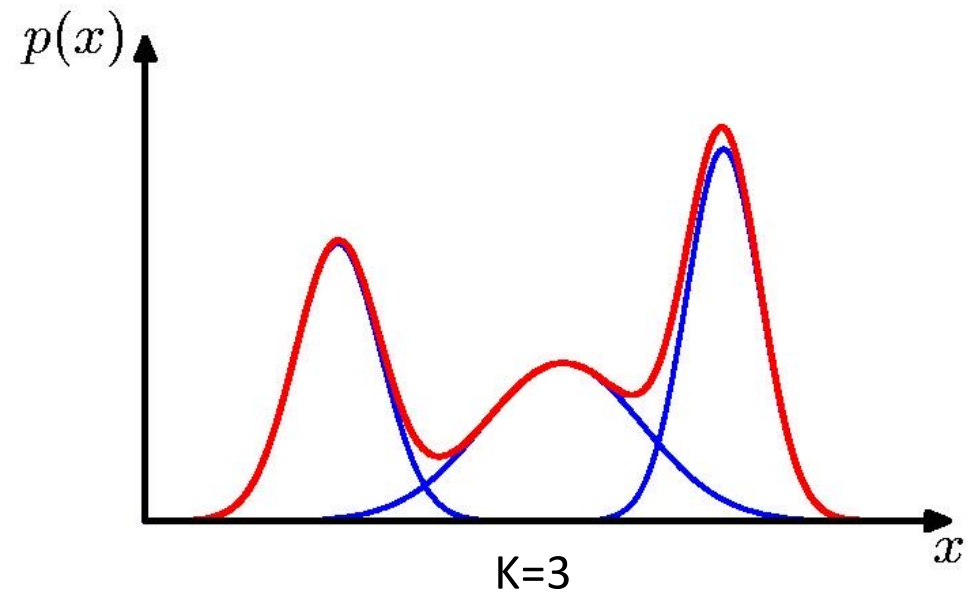
Смесь нормальных распределений (2)

Комбинация простых моделей в сложную:

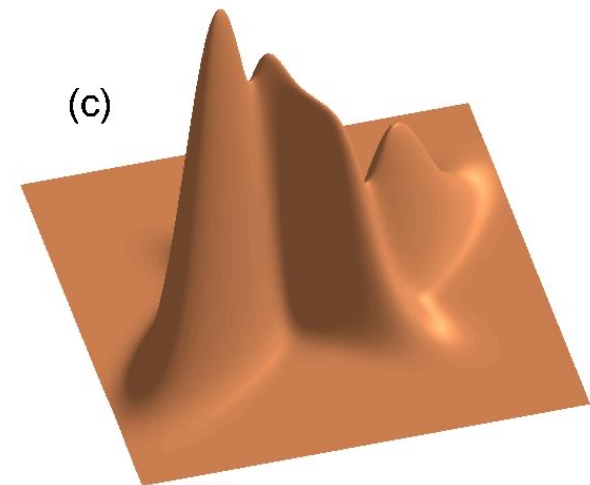
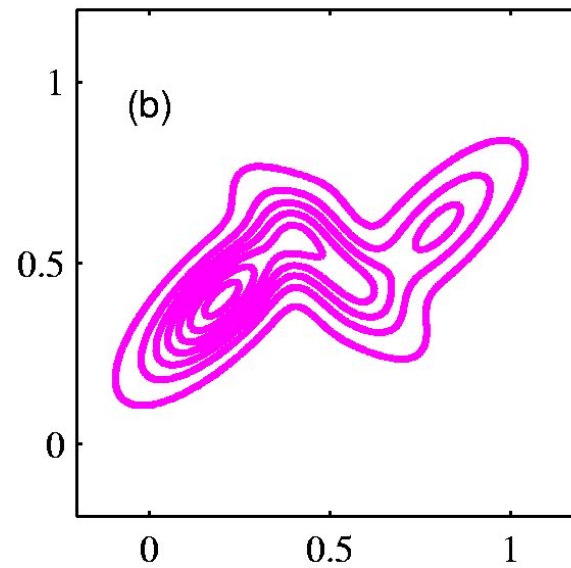
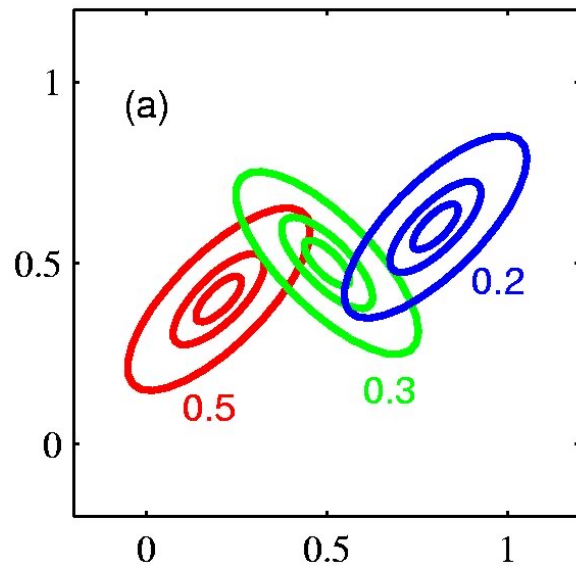
$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \underbrace{\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}_{\text{компонент смеси}}$$

↑
коэфф. смешивания

$$\forall k : \pi_k \geq 0 \quad \sum_{k=1}^K \pi_k = 1$$



Смесь нормальных распределений (3)



Семейство экспоненциальных распределений (1)

$$p(\mathbf{x}|\boldsymbol{\eta}) = h(\mathbf{x})g(\boldsymbol{\eta}) \exp \{ \boldsymbol{\eta}^T \mathbf{u}(\mathbf{x}) \}$$

где $\boldsymbol{\eta}$ – естественные параметры

$$g(\boldsymbol{\eta}) \int h(\mathbf{x}) \exp \{ \boldsymbol{\eta}^T \mathbf{u}(\mathbf{x}) \} d\mathbf{x} = 1$$

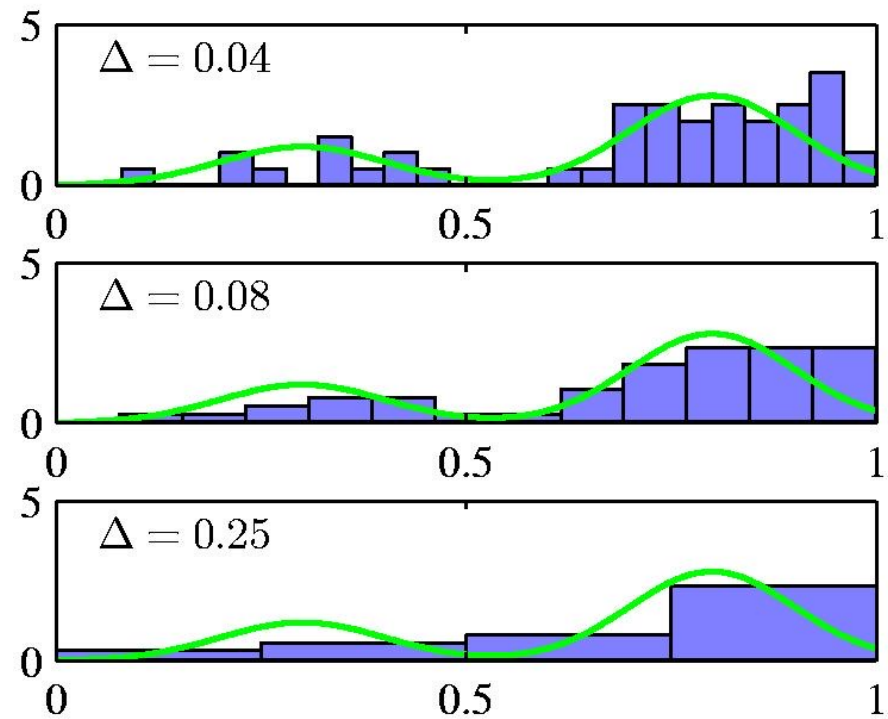
$g(\boldsymbol{\eta})$ можно интерпретировать как коэффициент нормализации.

Непараметрические методы

Методы гистограмм разбивают пространство данных на отдельные ячейки шириной Δ_i и подсчитывают количество наблюдений n_i в каждой ячейке.

$$p_i = \frac{n_i}{N\Delta_i}$$

- Часто для всех ячеек используется одинаковая ширина, $\Delta_i = \Delta$.
- Δ действует как параметр сглаживания.



использование M ячеек в каждом измерении потребует M^D ячеек!

Непараметрические методы (2)

Предположим, что наблюдения получены из плотности $p(\mathbf{x})$ и рассмотрим небольшую область R , содержащую $\mathbf{x}$, такую, что

$$P = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}.$$

Вероятность того, что K из N наблюдений лежат внутри R , равна $\text{Bin}(K | N, P)$ и если N велико

$$K \simeq NP.$$

Если объем R, V , достаточно мал, $p(\mathbf{x})$ приблизительно постоянен по R и

$$P \simeq p(\mathbf{x})V$$

Таким образом

$$p(\mathbf{x}) = \frac{K}{NV}.$$

V мало, но $K > 0$, поэтому N большое?

Непараметрические методы (3)

Оценка плотности ядра: фиксируем V , оцениваем K по данным. Пусть R — гиперкуб с центром в точке $\mathbf{x}$, и определяем функцию ядра (окно Парзена).

$$k((\mathbf{x} - \mathbf{x}_n)/h) = \begin{cases} 1, & |(x_i - x_{ni})/h| \leq 1/2, \\ 0, & \text{otherwise.} \end{cases} \quad i = 1, \dots, D,$$

Из этого следует, что

$$K = \sum_{n=1}^N k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right) \text{ и, следовательно, } p(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h^D} k\left(\frac{\mathbf{x} - \mathbf{x}_n}{h}\right).$$

Непараметрические методы (4)

Чтобы избежать разрывов в $p(x)$, используйте гладкое ядро, например, гауссово.

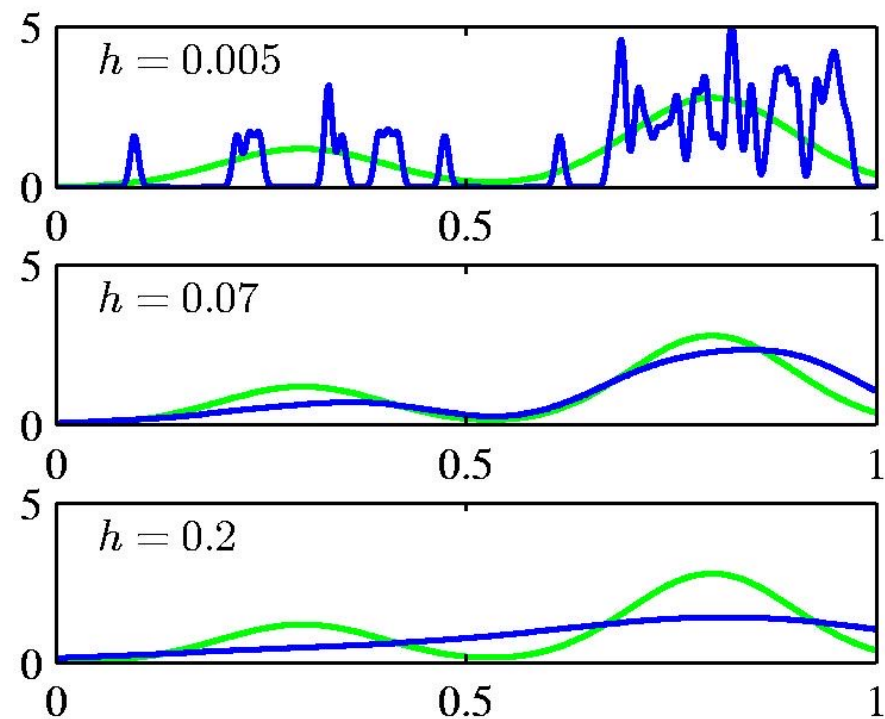
$$p(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{(2\pi h^2)^{D/2}} \exp \left\{ -\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{2h^2} \right\}$$

Любое ядро такое, что

$$k(\mathbf{u}) \geq 0,$$

$$\int k(\mathbf{u}) d\mathbf{u} = 1$$

будет работать.

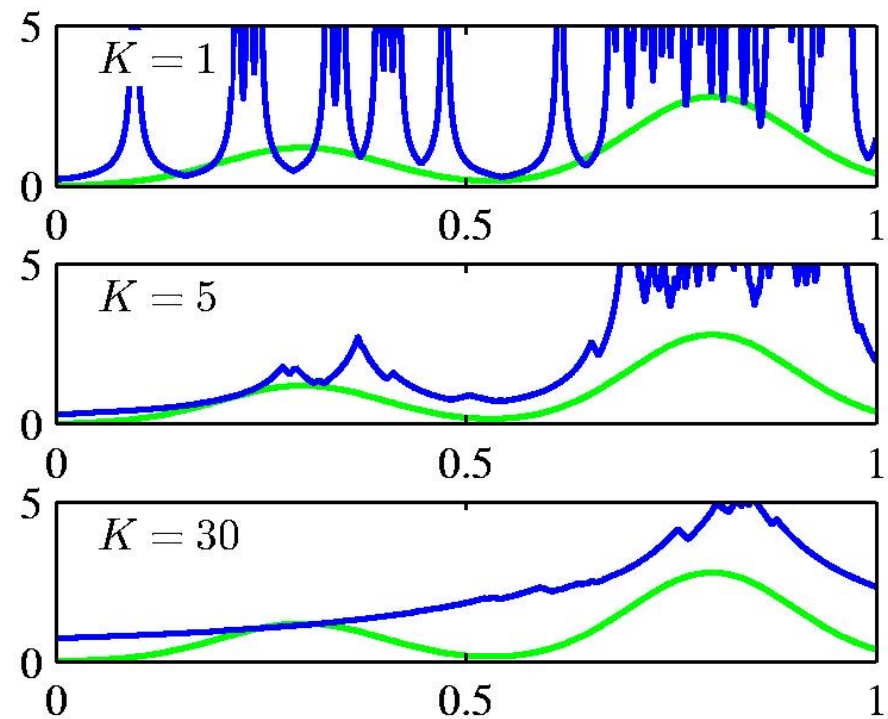


h действует как сглаживатель.

Непараметрические методы (5)

Оценка плотности ближайших соседей:
фиксируем K , оценим V по данным. Рассмотрим гиперсферу с центром в точке x и увеличим её объём до V . Она включает K из заданных N точек данных. Тогда

$$p(\mathbf{x}) \simeq \frac{K}{NV^*}.$$



K действует как сглаживающее средство.

Непараметрические методы (6)

Непараметрические модели (не гистограммы) требуют хранения и вычислений со всем набором данных.

После настройки параметрические модели становятся гораздо более эффективными с точки зрения хранения и вычислений.

K -ближайших соседей для классификации (1)

Учитывая набор данных с N_k точками данных из класса C_k и $\sum_k N_k = N$, мы имеем

$$p(\mathbf{x}) = \frac{K}{NV}$$

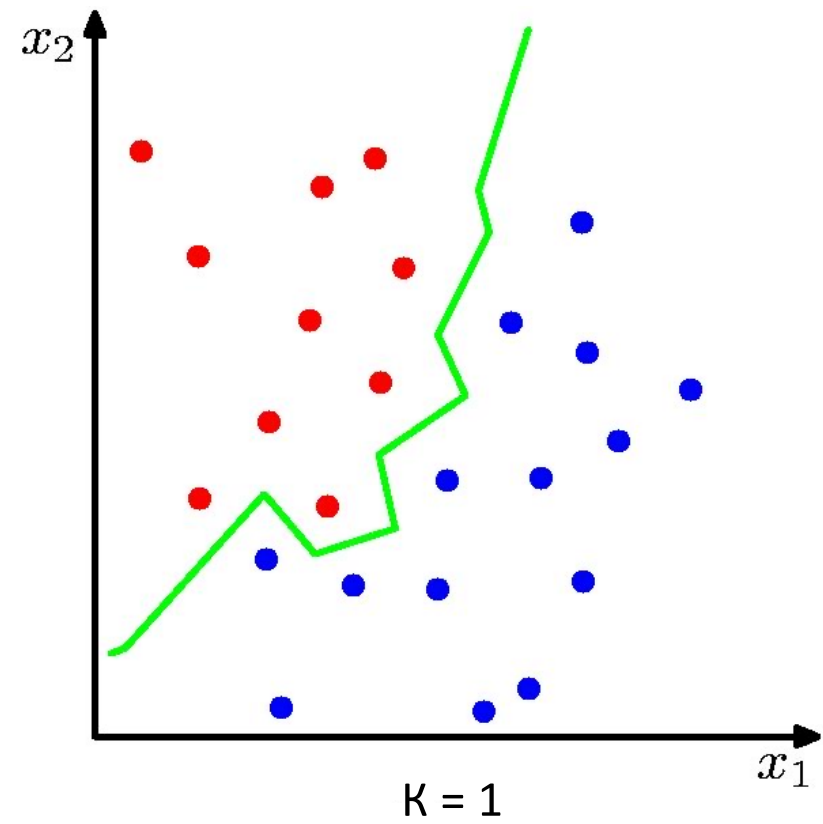
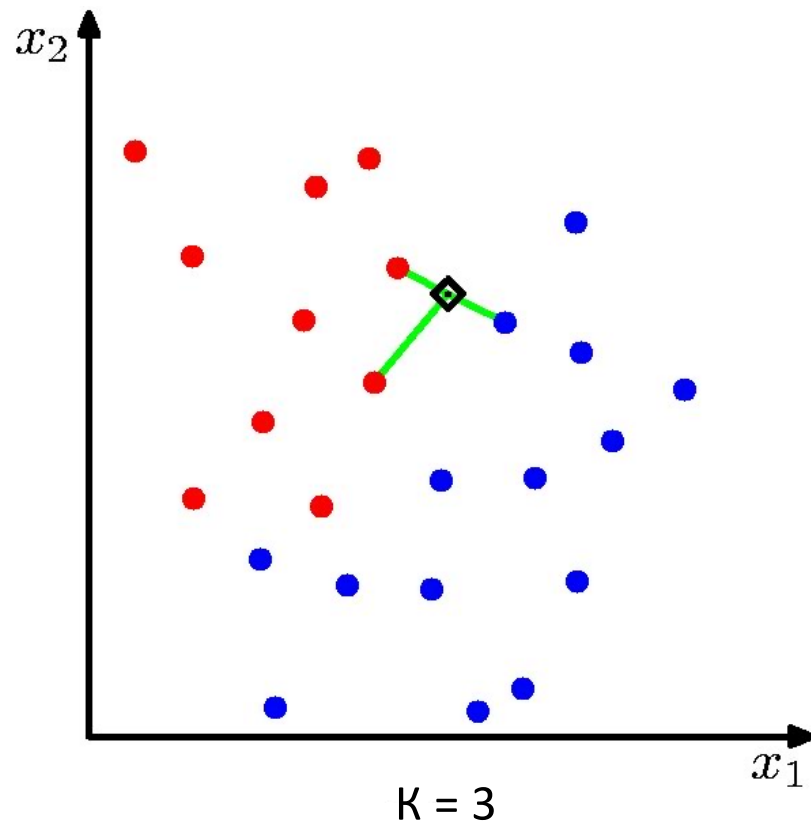
и соответственно

$$p(\mathbf{x}|C_k) = \frac{K_k}{N_k V}.$$

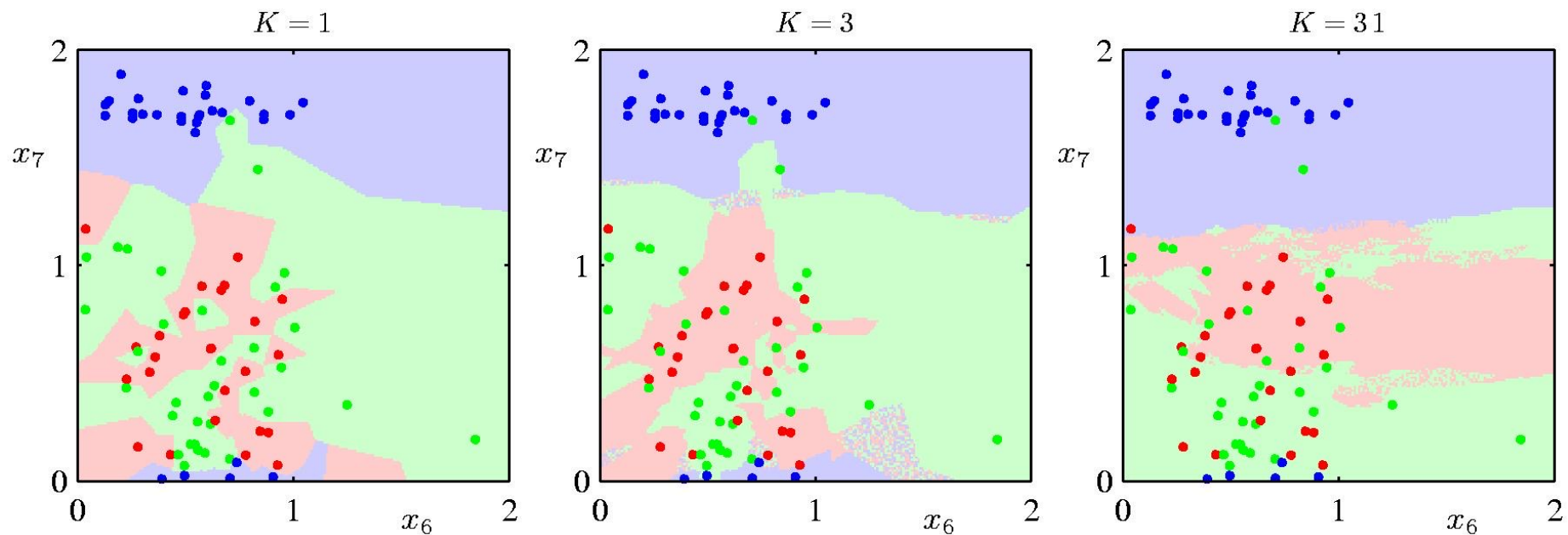
Поскольку $p(C_k) = N_k/N$ теорема Байеса дает

$$p(C_k|\mathbf{x}) = \frac{p(\mathbf{x}|C_k)p(C_k)}{p(\mathbf{x})} = \frac{K_k}{K}.$$

К-ближайших соседей для классификации (2)



К-ближайших соседей для классификации (3)



К действует как «сглаживатель»

Для $N \rightarrow \infty$, частота ошибок классификатора с одним ближайшим соседом никогда не превышает более чем в два раза оптимальную ошибку (полученную из истинных условных распределений классов).

продолжение см. ML2

Итоги

- Определите цели
- Измеряйте свое приближение к целям
- Ведите разведочный анализ, готовьте данные
- Выбирайте модели
- Обучайте модели
- Рассчитывайте метрики
- Внедряйте хорошие модели в бизнес-процессы
- Постоянно повышайте качество

Спасибо за внимание!